

基于风险修正与滚动时域优化的微网购电与储能调度

摘要

非孤岛式微网中，光伏发电功率与小区负载随时间变化，需借助储能和外部电网协调供需。本文以降低购电费用为目标，在满足供电需求和储能运行约束的条件下，研究确定性、预测偏差、日内信息更新及电价波动条件下的微网调度与储能优化问题。

针对问题一，将一天划分为 144 个 10 分钟时段，建立**购电—储能词典序线性规划模型**，先最小化购电费用，再在给定购电费用容差内减少储能充放电量，并满足供需、储能效率、容量、功率及日初日末储电量均为 6000 kWh 的约束。最优方案全天购电量为 59482.70 kWh，购电费用为 35126.95 元；与无储能方案相比，购电量减少 2307.24 kWh，费用降低 12925.10 元，降幅 26.90%，弃光量由 6247.96 kWh 降至 0。

针对问题二，考虑计划购电确定后负载与光伏发电预测仍存在偏差，建立**基于历史残差分位数修正的滚动时域线性规划模型**。利用历史同星期负载和近五个完整自然日光伏数据进行预测，并以历史净负载预测残差按 10 分钟时段进行 80% 分位数修正；储电量跨日连续，实际偏差优先由储能补偿，剩余缺口紧急购电。2025 年 2—12 月，正常购电量为 2164.7262 万 kWh，紧急购电量为 **12.3700** 万 kWh，总购电费用为 **1399.7990** 万元，其中正常购电费和紧急购电费分别为 1325.9252 万元和 73.8738 万元。与不作风险修正相比，总费用减少 149.9773 万元，紧急购电费减少 256.2844 万元，正常购电费增加 106.3071 万元，表明适当的风险修正能够通过减少高价紧急购电降低实际费用。

针对问题三，由于日内光伏预报持续更新，且实际负载可能偏离 0:00 预测，固定的 0:00 计划难以及时适应供需变化，因此建立**基于日内信息更新与负载反馈的滚动时域优化模型**。每日 0:00 制定初始计划，并在 6:00、12:00 和 18:00 结合最新光伏预报、实际储电量和已观测负载调整尚未执行的计划，调整量以 0:00 原计划为基准结算，下调取消部分退回原购电费用、并支付相当于该部分购电费用 50% 的违约费用，上调部分按交易时刻电价的 1.5 倍计价，实际偏差仍优先由储能调节，剩余缺口通过紧急购电补足。2025 年 2—12 月，最终方案上调购电 30.7206 万 kWh、下调购电 114.2817 万 kWh，紧急购电量为 **2.3340** 万 kWh，总购电费用为 **1359.9979** 万元；相较采用相同预测与结算规则、但不进行日内更新的对照方案，总购电费用减少 60.5864 万元，紧急购电量减少 9.6564 万 kWh，说明日内信息更新能够有效降低供需偏差及紧急购电需求。

针对问题四，考虑外部电网电价随日期和时段波动，将问题二、问题三模型分别扩展至波动电价情形。每日 0:00 根据历史价格预测当天电价并重新优化购电与储能计划，实际费用按各时段真实电价结算。仅在每日 0:00 制定计划时，总购电费用为 **1468.7087** 万元，紧急购电量为 **12.4870** 万 kWh，较直接沿用问题二原策略并按波动电价重新计费低 6.1255 万元；允许日内调整时，在 6:00、12:00 和 18:00 更新光伏预报、实际储电量和已观测负载，日内沿用 0:00 形成的日内电价预测曲线，总购电费用为 **1429.1323** 万元，紧急购电量为 **2.6947** 万 kWh，较直接沿用问题三原策略并按波动电价重新计费低 4.1338 万元，说明波动电价下重新优化购电与储能安排仍具有经济收益。

综合结果表明，历史预测、风险修正与日内信息更新能够协同降低紧急购电需求和购电费用，并保持储能运行约束下的方案可行性。

关键词：微网调度；储能优化；线性规划；风险修正；滚动时域优化

一、问题重述

1.1 问题背景

非孤岛式微网需要协调光伏发电、储能设备与外部电网，以满足小区负载需求。由于负载、光伏发电功率及外部电价均具有时变性，购电计划既要满足供电与储能运行约束，也要尽可能降低正常购电、调整购电及紧急购电产生的费用。

1.2 问题任务与关键条件

问题一：在单日负载、光伏发电功率预测和分时电价已知的条件下，确定各 10 分钟时段的正常购电量及储能充放电方案，使全天购电费用最低，并满足日初、日末储电量相同等运行约束。

问题二：考虑负载和光伏发电功率随日期变化，每日 0:00 提前制定购电计划。当储能调节后仍有供电缺口时，以同期正常电价的 5 倍进行紧急购电，求得统计期内购电与储能运行方案。

问题三：在问题二的基础上，利用 6:00、12:00 和 18:00 新发布的光伏发电功率预报调整尚未执行的购电计划，并计入上调、下调及紧急购电费用，比较日内调整前后的经济性，评价不同更新时间的信息价值。

问题四：将固定分时电价扩展为随日期和时段变化的实际电价，分别重新研究仅在 0:00 制定计划和允许日内调整两种情形，比较波动电价下不同购电与储能策略的费用表现。

各问题均需满足题目规定的储能设备容量、储电量范围、最大充放电功率与充放电效率等运行约束；结果以 10 分钟为时段，多日结果覆盖 2025 年 2 月 1 日至 12 月 31 日，并按题目附件 5 中表 1—表 3 的格式给出指定时段与指定日期的结果。

二、问题分析

2.1 问题一：确定性条件下的购电与储能调度分析

问题一中，负载、光伏发电功率和分时电价均已知，核心是协调外部购电与储能充放电，在满足供需平衡、容量和功率约束的同时降低全天购电费用。由于同一最低费用可能对应多组调度方案，本文采用**词典序线性规划**：先最小化购电费用，再以储能吞吐量作为给定购电费用容差范围内方案的择优准则，并通过无储能基准和约束复算检验方案的经济性与可行性。

2.2 问题二：预测偏差下的计划购电与紧急购电分析

问题二需要在每日 0:00 提前制定购电计划，而当天实际负载和光伏发电功率尚不可知，预测偏低还会引发高价紧急购电。为此，先依据历史数据预测负载与光伏发电功率，再利用**历史净负载预测残差的分位数**对计划需求进行风险修正，并通过跨日滚动优化确定正常购电和储能安排。实际运行中的剩余偏差优先由储能调节，无法补足的部分再通过紧急购电解决。

2.3 问题三：日内预报更新与购电调整分析

问题二的购电计划在 0:00 一次确定，无法利用当天随后到达的新信息。问题三进一步引入 6:00、12:00 和 18:00 的光伏发电功率预报、实际储电量及已观测负载，仅对尚未执行的时段重新优化，并通过**负载反馈**修正后续需求预测。模型同时计入购电调整费用和紧急购电费用，通过不同更新时间组合的比较评价日内信息更新的经济价值。

2.4 问题四：波动电价下的策略优化分析

问题四将前三问的固定分时电价扩展为随日期和时段变化的波动电价，因此购电与储能计划还需考虑未来价格的不确定性。本文利用历史电价形成预测价格，并在实际交付后按真实电价结算；分别在问题二和问题三模型基础上重新优化“仅 0:00 制定计划”和“允许日内调整”两种策略，再与原策略仅按波动电价重新计费的结果比较，以评估主动重优化带来的收益。

2.5 总体求解流程

四个问题由确定性调度逐步扩展至预测偏差、日内信息更新和波动电价，前一问的模型结构与结果作为后一问的基础，形成连续的购电—储能优化体系。各问题之间的联系及总体求解流程如图 1 所示。

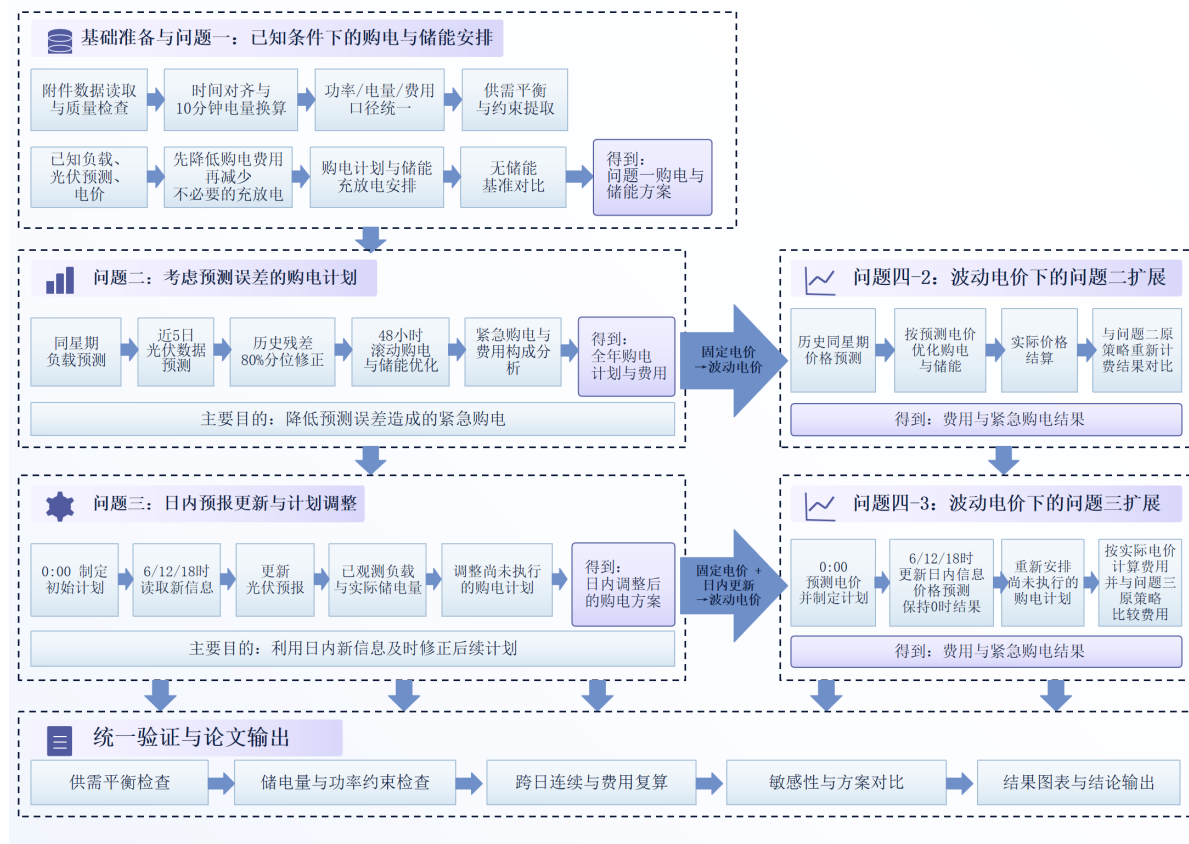


图 1: 总体求解流程

三、模型假设

题目已给出的储能容量、充放电功率、运行效率及电价倍率等参数，均直接写入相应约束或目标函数，不再作为模型假设重复列出。为便于模型建立，作如下假设：

假设 1：将附件中各时刻给出的功率数据作为相应 10 分钟区间的右端点离散值，并以右端点矩形规则近似计算时段电量，即取 $\Delta t = 1/6 \text{ h}$ 。该处理仅作为本文的离散化口径，不视为对连续功率曲线的精确积分；储电量在相邻时段之间按状态方程连续递推。

假设 2：研究期内不存在影响数据可比性的重大设备改造或统计、计量方式变化，负载、光伏发电和电价数据按统一口径进行处理，使不同时期的数据能够在相同时间尺度下比较。

假设 3: 负载、光伏发电及其预测误差在相近日期和相同时段具有一定延续性，已观测的历史信息能够为后续预测及风险修正提供参考。

假设 4: 除题目给定的充、放电效率外，不额外考虑储能自放电、线路损耗和电池老化成本；储能运行仍须满足容量范围、充放电功率上限及储电量递推关系。

假设 5: 题目未给出外部电网购电容量上限，因此本文不额外设置网侧购电功率上限，即认为该约束在本题研究范围内不构成限制。

假设 6: 光伏发电在满足小区负载和储能充电需求后若仍有剩余，允许弃用；题目未规定余电上网价格，因此不考虑向外部电网售电及相应收益。

以上假设用于统一模型中的现实简化。历史样本长度、残差分位水平、负载反馈强度、滚动时域长度以及电价预测方式等均属于具体模型设定，将在相应问题的模型建立过程中分别说明。

四、符号说明

表 1: 主要符号说明

符号	含义	单位	使用范围	备注
i	日期索引	—	全文	问题一中表示指定日期
t	日内时段索引	—	全文	每日共 144 个 10 分钟时段
v	购电计划的制定或更新时间	—	问题三、问题四 日内调整情形	$v \in \{0, 6, 12, 18\}$
Δt	单个时段长度	h	全文	$\Delta t = 1/6$
$L_{i,t}, \hat{L}_{i,t}^{(v)}$	实际负载功率、时刻 v 的负载预测值	kW	实际值用于全文；预测值用于问题二至问题四	预测仅使用当时已获得的信息
$G_{i,t}, \hat{G}_{i,t}^{(v)}$	实际光伏发电功率、时刻 v 可获得的光伏预测值	kW	实际值用于全文；预测值用于问题二至问题四	日内预报只更新尚未执行时段
$p_{i,t}, \hat{p}_{i,t}$	实际电价、预测电价	元/kWh	实际电价用于全文；预测电价用于问题四	问题四用预测电价制定计划，按实际电价结算
η_c, η_d	储能充电效率、放电效率	—	全文	由题目给定
$E_{\min}, E_{\max}$	储电量下限、上限	kWh	全文	分别为 1200、10800
$P_{\max}$	最大充电功率和最大放电功率	kW	全文	$P_{\max} = 5000$ ，单时段上限为 $P_{\max}\Delta t = 5000/6 \approx 833.3333$ kWh
κ	紧急购电价格倍数	—	问题二至问题四	$\kappa = 5$
$q_{i,t}^{(v)}$	时刻 v 确定的正常购电计划量	kWh	全文	$q_{i,t}^{(0)}$ 表示 0:00 计划
$q_{i,t}^f$	第 t 时段最终生效的正常购电量	kWh	问题三、问题四 日内调整情形	用于最终费用结算
$c_{i,t}, r_{i,t}$	第 t 时段的储能充电量、放电量	kWh	全文	$r_{i,t}$ 为储能向微网释放的电量，二者不同时为正

符号	含义	单位	使用范围	备注
$e_{i,t}$	紧急购电量	kWh	问题二至问题四	储能调节后仍有缺口时购入
$u_{i,t}^+, u_{i,t}^-$	相对 0:00 计划的上调量、下调量	kWh	问题三、问题四 日内调整情形	用于计算购电调整费用
$E_{i,t}$	第 t 时段结束时的储电量	kWh	全文	在相邻时段及日期之间连续传递
$w_{i,t}$	供需平衡中的未利用富余电量	kWh	全文	不区分富余电量来源，不直接作为弃光量统计

注： $r_{i,t}$ 表示储能放电量，不用于表示预测残差；后文将预测残差记为 $\varepsilon_{i,t}$ 。历史误差分位数、负载反馈系数、目标函数值及各项费用等局部符号，在首次出现时另行定义。表中符号按“索引—已知参数—预测参数—决策变量—状态及辅助变量”的顺序排列。

五、数据预处理与探索性分析

5.1 附件关系与数据质量检查

五份附件分别给出问题一的单日基准数据、全年实际负载与实际光伏发电功率、每日 0:00、6:00、12:00 和 18:00 发布的光伏预报、全年实际电价以及结果填报模板。除问题一外，结果统计区间均为 2025 年 2 月 1 日至 12 月 31 日，1 月数据用于形成首日预测所需的历史信息。

对附件 1—4 的数据规模、时间索引及有效性进行核查，结果见表 2。各表均未发现缺失值、无效值及异常负值，因此不进行样本删除、插补或平滑处理。

表 2: 附件 1—4 数据规模及质量核查结果

附件	原始数据规模	处理后规模	核查结果
附件 1	144 个 10 分钟时段	144 个时段	时间严格递增，电价、负载和光伏预测数据完整
附件 2	365 天 $\times$ 144 时段 $\times$ 2 类功率	保持不变	日期、时段完全对应，105120 个功率值完整有效
附件 3	365 天 $\times$ 4 次发布 $\times$ 24 小时预报	保持不变	1460 条预报记录、35040 个预报值完整有效
附件 4	365 天 $\times$ 144 时段	保持不变	52560 个电价值均为正，范围为 0.0076~1.7936 元/kWh

附件 2 与附件 4 采用相同的日期和 10 分钟时段索引。附件 3 中同日后续三次预报的日期标签空白仅作前向填充处理，不修改预测数值；经时间映射后仍超出 2025 年数据覆盖范围的年末预测目标不参与预报误差检验。附件 5 的个别时段标签与自然日边界存在轻微错位，内部计算统一采用每日 144 个连续 10 分钟时段，并按既定对应关系写回模板。

5.2 时间映射与功率—电量换算

附件中的负载、光伏发电功率和电价数据均按 10 分钟时段记录。为统一各附件的时间索引，本文以自然日为单位，将每天划分为 144 个连续时段。以 0:00 为起点，第 t 个时段记为 $[10(t-1), 10t] \text{ min}$, $t = 1, 2, \dots, 144$ 。对于附件中按时刻给出的负载和光伏发电功率，本文采用右端点映射，将该时刻数值作为前一 10 分钟区间的代表值。例如，0:10 对应 00:00—00:10，0:00+1 对应 23:50—24:00。

负载和光伏发电功率的单位为 kW，而购电量、充放电量和储电量均以 kWh 计。单个时段长度为

$$\Delta t = \frac{10}{60} = \frac{1}{6} \text{ h}. \quad (1)$$

因此，第 i 日第 t 个时段的负载电量和光伏发电量分别为

$$l_{i,t} = L_{i,t} \Delta t, \quad g_{i,t} = G_{i,t} \Delta t, \quad (2)$$

其中， $L_{i,t}$ 和 $G_{i,t}$ 分别表示该时段的负载功率和光伏发电功率。电价单位为元/kWh，可直接与购电量相乘计算费用。

储电量按时段末记录，并按本时段充放电逐时段递推；前一日 24:00 的储电量作为下一日 0:00 的初始储电量，保证跨日连续。

5.3 负载、光伏发电功率与电价的时序特征

负载、光伏发电功率与电价采用相同的 10 分钟时间网格。光伏发电功率具有明显的昼夜规律，全年 52560 个时段中有 23540 个时段光伏发电功率为 0，其余时段还受天气与季节影响；小区负载则全天持续存在并随日期和时段波动。全年共有 10130 个时段光伏发电功率高于同期负载，存在可供储能吸收的富余电量。

储能可将富余光伏或低价时段的电量转移至供电缺口时段，从而减少外部购电支出；电价在 0.0076~1.7936 元/kWh 之间变化，且日内与跨日差异明显，说明购电与储能充放电需按供需关系和电价变化统一安排。

5.4 预测发布时间与可用信息范围

为避免提前使用未来信息，本文按各类数据的实际可获得时点组织模型输入。问题一中，当日负载、光伏发电功率预测和电价均已知；问题二中，每日 0:00 仅利用此前已完成日期的负载和光伏发电功率制定当日计划。附件 3 中的光伏预报分别在 0:00、6:00、12:00 和 18:00 发布，日内调整仅使用对应时点已发布的最新预报、实际储电量和已观测负载。实际负载、实际光伏发电功率和实际电价只在发生后参与实际运行、反馈或费用结算；对于波动电价情形，计划阶段利用历史价格形成预测电价。

六、问题一：确定性条件下的购电与储能联合优化

附件 1 给出了单日各时段的小区负载、光伏发电功率和分时电价。在上述信息均已知的条件下，问题一需要协调外部购电与储能充放电，在满足各时段负载需求及储能运行约束的同时降低全天购电费用。考虑到最低购电费用可能对应多组可行解，本文建立**购电—储能词典序线性规划模型**^[1-2]：先最小化全天购电费用，再在给定购电费用容差内最小化电池充放电吞吐量，作为该范围内方案的稳定择优规则。

求解时先由负载、光伏与电价数据建立时段供需平衡和储能状态递推关系，再完成词典序优化、基准比较与可行性检验，整体流程如图 2 所示。

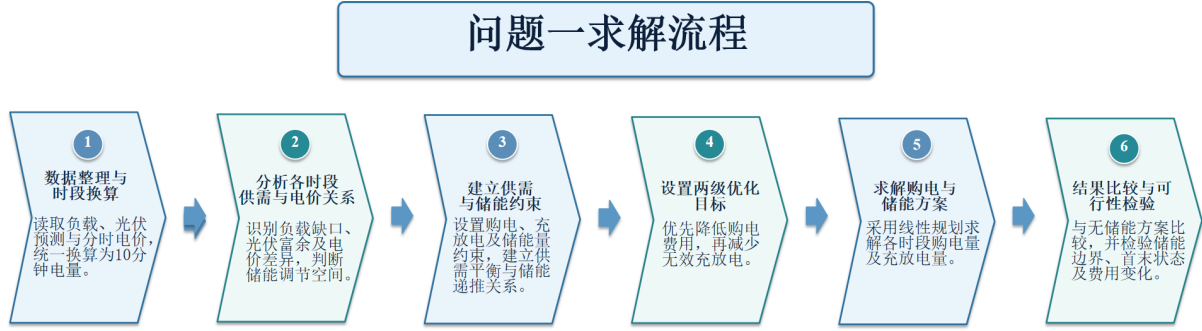


图 2: 问题一购电—储能联合优化流程图

6.1 时段供需平衡与储能状态递推

将一天划分为 144 个长度为 $\Delta t = 1/6 \text{ h}$ 的时段，记 $t = 1, 2, \dots, 144$ 。设第 t 个时段的负载功率和光伏发电功率分别为 L_t 和 G_t ，则相应的负载电量与光伏发电电量分别为 $L_t \Delta t$ 和 $G_t \Delta t$ 。

记 q_t 为正常购电量， c_t 和 r_t 分别为储能在母线侧的充电量和放电量， w_t 为供需平衡中的未利用富余电量。各时段的能量供需满足

$$q_t + G_t \Delta t + r_t = L_t \Delta t + c_t + w_t, \quad t = 1, 2, \dots, 144. \quad (3)$$

式 (3) 左侧为外部购电、光伏发电和储能放电共同形成的可用电量，右侧对应小区负载、储能充电以及未被利用的富余电量。当光伏发电功率高于同期负载时，剩余电量可用于储能充电；若储能受容量或功率限制无法继续吸收，则形成弃光。本文将可用光伏电量中未供给负载、未用于储能充电且未向外网出售的部分称为弃光量，单位为 kWh。 w_t 为供需平衡的富余项，未区分其来源，可能包含未利用的正常购电，因此一般不等同于弃光量；无储能基准中的弃光量按光伏优先供给负载后剩余的光伏电量单独计算。由此，外部购电、光伏利用和储能调节被统一纳入同一时段的供需平衡关系。

记 E_t 为第 t 个时段结束时的电池内部储电量，则 E_{t-1} 同时表示本时段开始时的储电状态。设充、放电效率分别为 η_c 和 η_d 。充电过程中仅有 $\eta_c c_t$ 实际存入电池，而向微网释放 r_t 电量需要消耗电池内部 r_t / η_d 的能量，因此储电量按照

$$E_t = E_{t-1} + \eta_c c_t - \frac{r_t}{\eta_d}, \quad t = 1, 2, \dots, 144 \quad (4)$$

逐时段递推。

题设给定 $\eta_c = \eta_d = 0.9$ ，储能额定容量为 12000 kWh，允许储电量保持在额定容量的 10% ~ 90% 范围内。问题一还要求日初与日末储电量均为 6000 kWh，故有

$$1200 \leq E_t \leq 10800, \quad E_0 = E_{144} = 6000. \quad (5)$$

储能最大充、放电功率均为 5000 kW。按单个时段长度 $\Delta t = 1/6 \text{ h}$ 换算，每个 10 分钟时段允许的最大充电量和放电量均为

$$5000 \times \frac{1}{6} = \frac{5000}{6} \approx 833.3333 \text{ kWh}.$$

因此，各决策变量还需满足

$$0 \leq c_t, r_t \leq \frac{5000}{6}, \quad q_t, w_t \geq 0. \quad (6)$$

从实际运行角度，同一时段内储能不应同时充电和放电，其物理互斥关系可表示为

$$c_t r_t = 0. \quad (7)$$

需要指出的是，式（7）若直接写入优化模型会使原问题转化为非线性模型。为保持主模型的线性结构，本文在正式线性规划中不额外加入该乘积约束，而是在求解后逐时段核验充、放电状态；144 个时段的正式最优解均未出现同时充放电现象。进一步采用含二元互斥变量的混合整数线性规划模型^[3]进行交叉验证，所得最优费用与调度结果保持一致，说明在本题数据条件下，采用线性规划不会改变问题一的最优结论。

至此，各约束条件共同确定了问题一的基本可行域，并通过充放电互斥的事后核验保证储能运行满足实际要求，为后续词典序目标函数的建立与求解提供基础。

6.2 购电费用与电池吞吐量的词典序优化

满足供需平衡、储电状态递推、容量边界和充放电功率约束的方案通常不唯一。本文将购电费用作为首要优化目标，并将储能总充放电量作为第二级目标，在给定购电费用容差范围内对方案进行统一选择。

记决策向量为 $\mathbf{x}$ ，其包含各时段的正常购电量 q_t 、充电量 c_t 、放电量 r_t 、储电量 E_t 及富余电量 w_t ；记满足上述线性约束的可行解集合为 $\mathcal{F}$ 。第一阶段最小化全天正常购电费用：

$$C^* = \min_{\mathbf{x} \in \mathcal{F}} \sum_{t=1}^{144} p_t q_t, \quad (8)$$

其中， p_t 为第 t 个时段的购电电价， C^* 为第一阶段得到的最低购电费用。

在一级购电费用给定容差范围内，第二阶段以储能能在母线侧的总充放电量为目标：

$$\begin{aligned} \min_{\mathbf{x} \in \mathcal{F}} \quad & B = \sum_{t=1}^{144} (c_t + r_t), \\ \text{s.t.} \quad & \sum_{t=1}^{144} p_t q_t \leq C^* + \varepsilon, \end{aligned} \quad (9)$$

其中， B 为全天储能总吞吐量， ε 为考虑求解器数值精度设置的容差。第二阶段允许购电费用相对一级最优值的增加不超过给定容差，并在此范围内最小化储能总吞吐量。

与将购电费用和储能吞吐量直接加权求和相比，上述处理无需人为设定两类目标之间的权重，能够保证购电费用的最高优先级。第二级目标的作用主要是为满足一级购电费用给定容差的方案提供统一的选择准则，而不作为降低购电费用之外的独立性能改进指标。

6.3 最优购电计划与储能运行方案

求解词典序线性规划模型后，得到全天正常购电量为 **59482.6990 kWh**，购电费用为 **35126.9486 元**。按照题目指定的结果格式，表 3 给出了六个时段的购电量以及全天购电量和购电费用。其中，10:00—10:10、14:00—14:10 和 20:00—20:10 时段无需从外部电网购电，而 12:00—12:10、16:00—16:10 和 18:00—18:10 时段仍存在一定的外部购电需求。

储能全天累计充电 20740.6661 kWh，累计放电 16799.9396 kWh。按照题目要求，将 144 个 10 分钟时段汇总为六个 4 小时时段，结果见表 4。20:00—24:00 区间内同时出现

表 3: 微网在指定时间段的购电量及全天购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.0000	12:00–12:10	480.4125	14:00–14:10	0.0000
16:00–16:10	445.4317	18:00–18:10	531.8941	20:00–20:10	0.0000
全天购电量		59482.6990	全天购电费		35126.9486

充电量和放电量，是由于该区间包含多个 10 分钟时段，不同时段采取了不同的充放电动作，并非储能在同一时段同时充电和放电；逐时段核验表明，所有时段均满足充放电互斥要求。

表 4: 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	4500.0000	0.0000	04:00–08:00	833.3333	6365.8412
08:00–12:00	4787.9643	1702.9970	12:00–16:00	5286.0352	91.1014
16:00–20:00	0.0000	5780.1319	20:00–24:00	5333.3333	2859.8681
0:00 储电量		6000.0000	24:00 储电量		6000.0000

表 3 和表 4 给出了题目要求的指定时段结果，但分段汇总难以完整反映购电、光伏发电功率与储能状态在一天内的协同变化。为观察各时段的调度特征，进一步绘制日内供需与储能联合运行情况，如图 3 所示。图中上层展示负载、光伏发电与外部购电的时序变化，中层给出储能充电量 c_t 和放电量 r_t ，下层为时段末储电量 E_t 的变化轨迹。

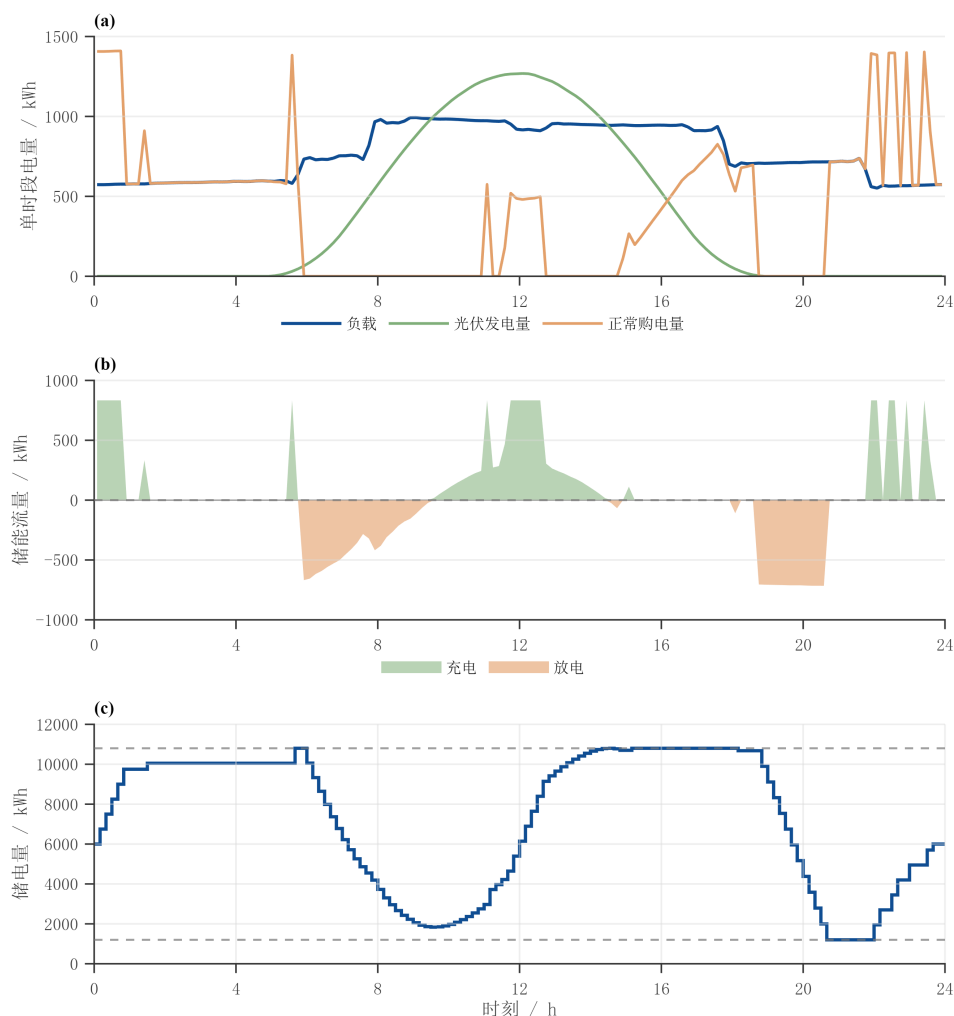


图 3: 问题一日内供需与储能联合运行情况

从日内运行结果可以看出，储能根据光伏发电功率、负载需求和分时电价在各时段间转移电量：光伏相对充裕或购电成本较低时储能倾向于充电，在光伏不足、负载需求较高或购电成本较高的时段释放电量，从而减少高成本外部购电。全天储电量始终满足 $1200 \leq E_t \leq 10800 \text{ kWh}$ ，且日初与日末储电量均为 6000 kWh ，说明最优方案在完成日内能量转移的同时满足题目规定的储能循环条件。

6.4 无储能基准比较与方案可行性核验

为量化储能参与调度后带来的经济性与光伏消纳改善，构造无储能基准方案：各时段光伏发电优先满足小区负载，富余光伏直接弃置，供电缺口全部由外部电网补足。基准方案与正式模型采用相同的负载、光伏和电价数据。

图 4 对比了有、无储能两种方案的购电量、购电费用和弃光量。无储能时，全天购电量为 61789.9354 kWh ，购电费用为 48052.0466 元 ，弃光量为 6247.9630 kWh ；引入储能后，全天购电量降至 59482.6990 kWh ，购电费用降至 35126.9486 元 ，弃光量降为 0。相较基准方案，购电量减少 2307.2364 kWh (3.7340%)，购电费用减少 12925.0980 元 (26.8981%)，费用降幅明显高于购电量降幅，说明储能的主要收益来自跨时段削减高价购电，并提高光伏利用水平。

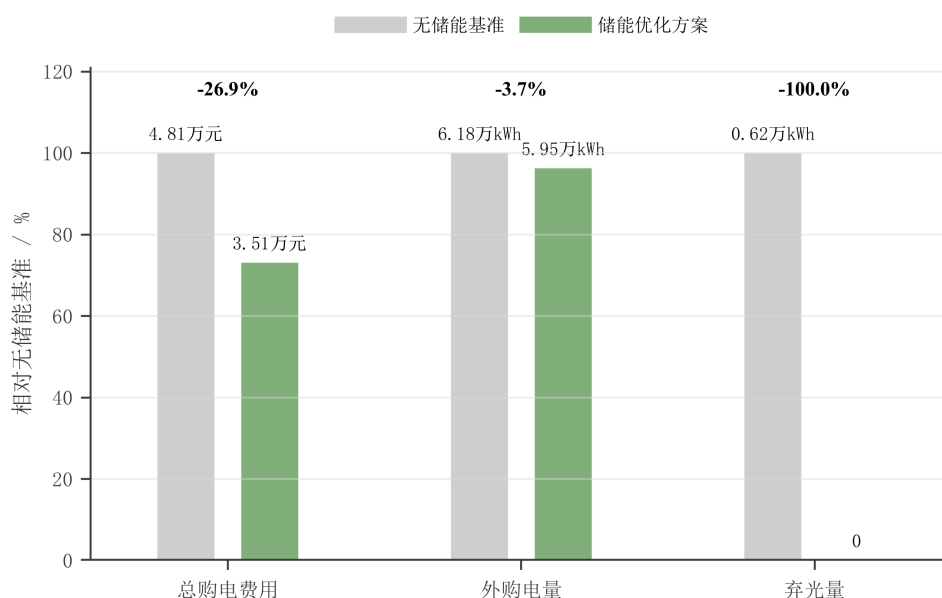


图 4: 储能方案相对无储能基准的改善效果

为检验词典序模型中第二级目标的实际作用，将仅最小化购电费用的单目标线性规划与正式词典序线性规划进行比较，结果见表 5。

表 5: 单目标线性规划与词典序线性规划结果对比

模型	购电费用 (元)	电池总吞吐量 (kWh)	同时充放电时段数	储电量轨迹最大差 (kWh)
仅最小化购电费用	35126.94858929	37540.60569835	0	300.0000
词典序线性规划	35126.94858964	37540.60569044	0	0.0000

单目标与词典序模型的购电费用仅相差 $3.51 \times 10^{-7} \text{ 元}$ 、储能吞吐量仅相差 $7.92 \times 10^{-6} \text{ kWh}$ ，二者均处于数值求解容差量级，但储电量轨迹最大相差 300 kWh 。因此，二级目标在本算例中主要用于对费用等价方案进一步择优，而非实质降低储能吞吐量。

问题一的正式模型未显式引入充放电互斥的二元变量。为验证这一简化是否影响最优结果，进一步建立含充放电互斥二元变量的**混合整数线性规划 (MILP) 模型**进行交叉求解，并与正式模型的**线性规划 (LP)** 结果对比，结果见表 6。

显式互斥 MILP 与正式 LP 的购电费用、购电量、储能吞吐量及储电量轨迹均一致，同时充放电时段数为 0，说明线性规划处理未改变本算例的最优结果。

表 6: LP 与 MILP 充放电互斥交叉验证结果

模型	最优购电费用 (元)	购电量 (kWh)	电池总吞吐量 (kWh)	同时充放电时段数	储电量轨迹最大差 (kWh)
正式 LP	35126.94858964	59482.69899752	37540.60569044	0	0.0000
显式互斥 MILP	35126.94858964	59482.69899752	37540.60569044	0	0.0000

最终方案的供需平衡、储电量递推及储能运行约束均通过独立复算，统一核验结果见第 10.1 节。

七、问题二：预测偏差下的风险修正购电策略

问题一在负载、光伏发电功率和电价均已知的条件下完成确定性调度，而问题二需要在每日 0:00 提前制定购电计划，此时当天实际负载与光伏发电功率尚不可知。若计划供电不足，剩余缺口只能以 5 倍电价紧急购电，因此预测偏低带来的经济损失不能忽略。为此，在沿用问题一供需平衡、储能状态递推及运行约束的基础上，本文建立**基于历史残差分位数修正的滚动时域线性规划模型**^[4-6]：先利用历史数据预测负载和光伏发电功率，再通过历史净负载残差修正计划需求，并在实际运行中由储能优先吸收预测偏差，剩余缺口通过紧急购电补足。

为直观说明预测、风险修正与滚动调度之间的关系，问题二的求解过程如图 5 所示。

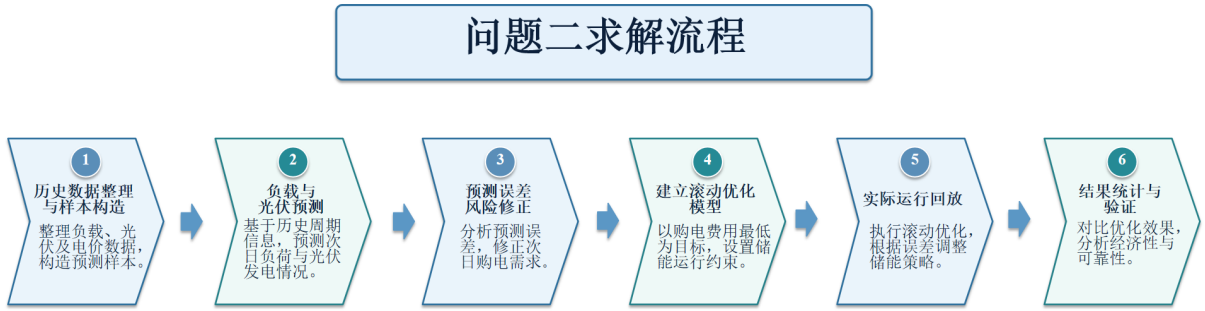


图 5: 问题二风险修正滚动购电策略求解流程

7.1 历史负载与光伏的短期预测

每日 0:00 制定购电计划时，当天及后续时段的实际负载和实际光伏发电功率尚未获得，因此预测过程仅使用当前时刻以前已经完整记录的历史数据。模型采用 48 h 滚动时域进行计划求解，但每天只执行前 24 h 的调度结果；进入下一日 0:00 后，再纳入最新完成日期的数据重新预测并优化，从而保证整个计算过程不使用未来实际信息。

小区负载具有较明显的周内周期。对于计划窗口中的目标日期 d ($d = i$ 或 $d = i + 1$)，从当前决策日 i 之前最近 28 个完整自然日中筛选与目标日期星期相同的历史日期，构成集合

$$\mathcal{H}_{i,d} = \{j \mid i - 28 \leq j < i, \text{weekday}(j) = \text{weekday}(d)\}. \quad (10)$$

将集合中的日期按时间先后排列为 $\{j_1, j_2, \dots, j_m\}$ ，近期日期取更大权重，第 d 日第 t 个

10 分钟时段的负载功率预测值为

$$\hat{L}_{d,t}^{(0)} = \sum_{k=1}^m \frac{2k}{m(m+1)} L_{jk,t}, \quad t = 1, 2, \dots, 144. \quad (11)$$

该方法既保留了不同星期属性下的负载差异，又提高了近期历史数据在预测中的作用，使预测能够随负载近期变化进行调整。

光伏发电功率受近期天气变化影响较明显，因此不再按星期属性分类，而采用当前决策时刻以前最近 5 个完整自然日的同一时段数据进行预测。第 t 个时段的光伏发电功率预测值为

$$\hat{G}_{i,t}^{(0)} = \frac{1}{5} \sum_{k=1}^5 G_{i-k,t}, \quad t = 1, 2, \dots, 144. \quad (12)$$

每日 0:00 根据已有历史数据生成光伏发电功率预测曲线，并用于当前 48 h 计划窗口；进入下一日后，再利用新增的完整日数据重新计算。当天尚未发生的实际光伏发电功率始终不参与计划制定。

在负载和光伏发电功率预测的基础上，定义基础净负载预测为

$$\hat{N}_{i,t}^{(0)} = \hat{L}_{i,t}^{(0)} - \hat{G}_{i,t}^{(0)}. \quad (13)$$

其中， $\hat{N}_{i,t}^{(0)} > 0$ 表示该时段预计存在供电缺口， $\hat{N}_{i,t}^{(0)} < 0$ 表示预计存在光伏富余。将预测功率乘以 $\Delta t = 1/6$ h 后，即得到进入购电—储能优化模型的基础净用电量。由于点预测仍可能低估实际净负载，下一节进一步利用历史预测残差对其进行分位数修正。

7.2 基于历史残差分位数的净负载修正

基础净负载预测仍存在一定误差。为刻画预测偏低的风险，记第 j 日第 t 个时段的实际净负载为

$$\begin{aligned} N_{j,t} &= L_{j,t} - G_{j,t}, \\ \varepsilon_{j,t} &= N_{j,t} - \hat{N}_{j,t}^{(0)}, \end{aligned} \quad (14)$$

其中， $L_{j,t}$ 和 $G_{j,t}$ 分别为实际负载功率和实际光伏发电功率；残差 $\varepsilon_{j,t}$ 为实际净负载减去由第 j 日 0:00 形成的基础净负载预测，其正负号表示基础预测偏低或偏高。当 $\varepsilon_{j,t} > 0$ 时，实际净负载高于基础预测值，即原计划低估了该时段的供电需求；当 $\varepsilon_{j,t} < 0$ 时，则表示基础预测偏高。只有在第 j 日的实际负载和实际光伏发电功率全部获得后，该日残差才进入后续预测，因此修正过程不会使用尚未发生的实际信息。

在第 i 日 0:00 制定购电计划时，从此前最近 56 个自然日中选取已经具有完整基础预测和实际观测记录的日期，构成历史残差样本集合

$$\mathcal{R}_i = \left\{ j \mid i - 56 \leq j < i, \hat{N}_{j,\cdot}^{(0)} \text{ 已形成} \right\}. \quad (15)$$

研究期初若可用历史不足 56 日，则从首个能够形成完整预测残差的日期开始，使用当时已有的全部样本，不对缺失历史进行插值或人为补充。对 $\mathcal{R}_i$ 中相同 10 分钟时段的残差分别计算经验分布的 80% 分位数^[4]，得到时段 t 的风险修正量

$$u_{i,t} = Q_{0.8}(\{\varepsilon_{j,t} \mid j \in \mathcal{R}_i\}). \quad (16)$$

修正量 $u_{i,t}$ 完全由历史预测误差计算得到，并随日期和时段滚动更新。进入下一日后，最新完成日期的残差加入样本窗口，超过 56 日的历史残差相应退出，从而使修正量能够随近期预测误差特征变化。

将 $u_{i,t}$ 加入 7.1 的基础净负载预测，得到购电计划实际采用的修正后净负载

$$\tilde{N}_{d,t}^{(0)} = \hat{N}_{d,t}^{(0)} + u_{i,t}, \quad d \in \{i, i+1\}. \quad (17)$$

其中， $d = i$ 和 $d = i+1$ 分别对应 48 h 计划窗口中的第 1 天和第 2 天；同一决策时刻得到的残差分位数修正量用于当前滚动窗口内相应时段的需求修正。经 $\Delta t = 1/6$ h 换算后， $\tilde{N}_{d,t}^{(0)} \Delta t$ 作为购电—储能优化中的预测净用电量，用于确定正常购电和储能充放电计划。

这种处理并不改变 7.1 的基础预测结构，而是利用历史中“实际净负载高于预测值”的信息调整计划需求，使购电策略对预测偏低造成的供电缺口保留适当余量。图 6 以典型日期展示基础净负载预测、实际净负载与 80% 分位数修正量之间的关系。

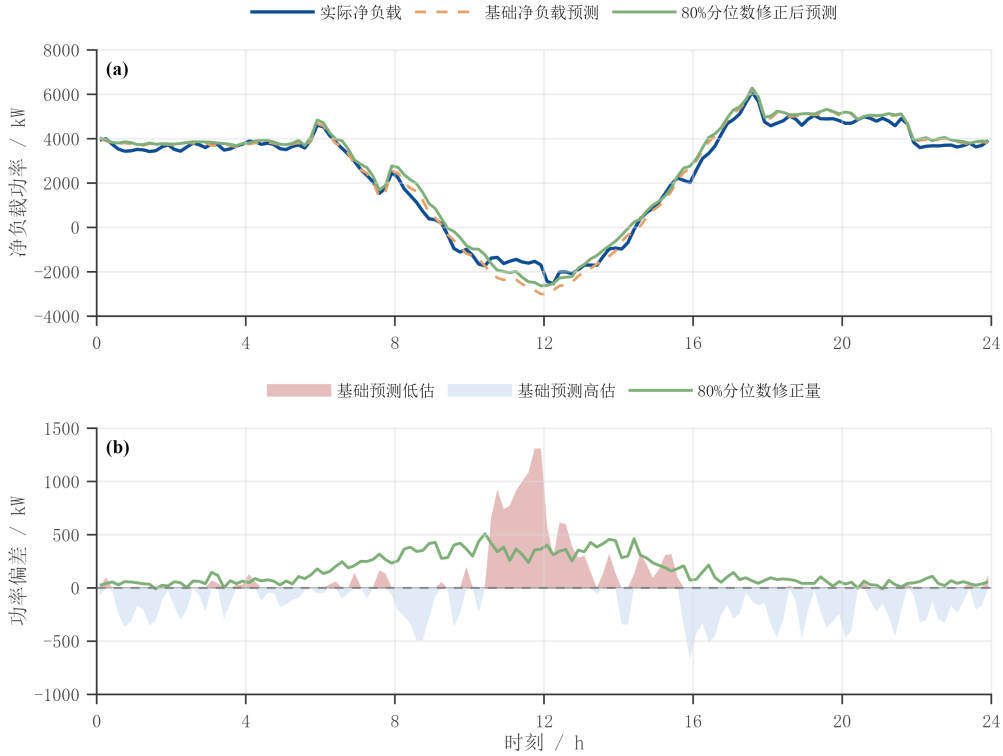


图 6: 典型日期的基础净负载预测与 80% 分位数修正

风险修正后的预测结果将在下一节进入跨日连续的滚动优化；随后按历史实际数据逐时段进行运行模拟。

7.3 跨日滚动计划与实际储能反馈

经 7.2 修正后，净负载预测需要进一步转化为可执行的购电与储能计划。本文采用 **48 h 滚动时域线性规划**^[5-6]：每日 i 的 0:00 同时优化当天和次日共 288 个 10 分钟时段，仅执行前 24 h；进入下一日后，以实际期末储电量作为新的初始状态重新求解。

按式 (2) 的功率—电量换算关系，修正后净负载对应的预测电量为 $\tilde{N}_{d,t}^{(0)} \Delta t$ ， $d \in \{i, i+1\}$ 。计划阶段的决策变量包括正常购电量 $q_{d,t}^p$ 、充电量 $c_{d,t}^p$ 、放电量 $r_{d,t}^p$ 、未利用富余电量 $w_{d,t}^p$ 和储电量 $E_{d,t}^p$ 。以 48 h 正常购电费用最小为目标：

$$\min \sum_{d=i}^{i+1} \sum_{t=1}^{144} p_t q_{d,t}^p. \quad (18)$$

相应的供需平衡、储电量递推、跨日衔接与运行边界为

$$\begin{aligned}
q_{d,t}^p + r_{d,t}^p &= \tilde{N}_{d,t}^{(0)} \Delta t + c_{d,t}^p + w_{d,t}^p, \\
E_{d,t}^p &= E_{d,t-1}^p + \eta_c c_{d,t}^p - \frac{r_{d,t}^p}{\eta_d}, \\
E_{i+1,0}^p &= E_{i,144}^p, \\
E_{i,0}^p &= E_{i,0}, \quad E_{i+1,144}^p = 6000, \\
1200 &\leq E_{d,t}^p \leq 10800, \quad 0 \leq c_{d,t}^p, r_{d,t}^p \leq \frac{5000}{6} \text{ kWh}, \quad q_{d,t}^p, w_{d,t}^p \geq 0.
\end{aligned} \tag{19}$$

其中, 前两式对 $d \in \{i, i+1\}$ 与 $t = 1, \dots, 144$ 成立, 因此计划窗口内部两日的储电量通过 $E_{i+1,0}^p = E_{i,144}^p$ 直接衔接, 不另设中断。滚动窗口以当天实际起始储电量为初值, 并将 48 h 末端储电量固定为 6000 kWh; 进入下一日后, 以实际期末储电量作为新窗口的初始状态重新求解, 使计划状态与实际状态的口径保持一致。

求得计划后, 仅执行当天正常购电量 $q_{i,t}^p$, 记为 $q_{i,t}$ 。实际负载与实际光伏发电功率到达后, 定义储能调节前的电量余缺

$$a_{i,t} = q_{i,t} + G_{i,t} \Delta t - L_{i,t} \Delta t. \tag{20}$$

当 $a_{i,t} \geq 0$ 时, 富余电量优先用于充电:

$$c_{i,t} = \min \left\{ a_{i,t}, \frac{5000}{6}, \frac{10800 - E_{i,t-1}}{\eta_c} \right\}, \quad w_{i,t} = a_{i,t} - c_{i,t}. \tag{21}$$

当 $a_{i,t} < 0$ 时, 储能优先放电, 仍未补足的部分通过紧急购电解决:

$$r_{i,t} = \min \left\{ -a_{i,t}, \frac{5000}{6}, \eta_d (E_{i,t-1} - 1200) \right\}, \quad e_{i,t} = \max \{0, -a_{i,t} - r_{i,t}\}. \tag{22}$$

因此, 同一时段只会执行充电或放电中的一种操作。运行模拟中的实际供需平衡为

$$q_{i,t} + G_{i,t} \Delta t + r_{i,t} + e_{i,t} = L_{i,t} \Delta t + c_{i,t} + w_{i,t}, \tag{23}$$

实际储电量按第 6 章式 (4) 递推, 增加日期索引, 并采用实际充放电量。当天结束后, 实际储电量直接传递至下一日:

$$E_{i+1,0} = E_{i,144}. \tag{24}$$

正常购电按分时电价 p_t 结算, 紧急购电按同期电价的 5 倍结算, 因此第 i 日实际购电费为

$$C_i = \sum_{t=1}^{144} p_t q_{i,t} + \sum_{t=1}^{144} 5p_t e_{i,t}. \tag{25}$$

由此, 计划阶段负责确定正常购电与储能安排, 运行模拟阶段则利用储能反馈吸收预测偏差, 并以紧急购电补足剩余缺口; 储电量按照运行模拟结果跨日连续传递。下一节将基于统计期内运行模拟结果, 比较不同预测与风险处理方式下的费用和紧急购电表现。

7.4 全期购电结果与预测—风险关键模块有效性分析

按照前述预测、风险修正和滚动调度方法, 对 2025 年 2 月 1 日至 12 月 31 日共 334 天进行逐日运行模拟。考虑到四个指定日期的结果表结构相同, 为避免正文中重复表格

占用过多篇幅，本文选取 2025 年 3 月 20 日作为代表日期，在正文中展示其指定时段正常购电量、全天购电量与购电费用，以及储能设备分时段充放电量和 0:00、24:00 的储电量；其余指定日期的同类结果统一列于附录，完整逐时段结果见对应结果文件。表 7 和表 8 分别按题目表 1、表 2 的格式给出该代表日期的结果；问题二其余三个指定日期的同类结果见附录 A.1。表 9 按题目表 3 的格式集中列出四个指定日期的紧急购电事件。

表 7: 微网在指定时间段的正常购电量及全天购电量和购电费

2025 年 3 月 20 日					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.0000	12:00–12:10	582.7096	14:00–14:10	0.0000
16:00–16:10	479.5529	18:00–18:10	701.7312	20:00–20:10	0.0000
全天购电量		63841.8084		全天购电费	
				39430.3622	

表 8: 储能设备在指定时间段的充放电电量及 0:00、24:00 的储电量

2025 年 3 月 20 日					
时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	1411.2592	190.5300	4:00–8:00	1082.4118	5299.0706
8:00–12:00	6188.4915	2777.3772	12:00–16:00	3811.3211	474.3713
16:00–20:00	230.6784	5855.4450	20:00–24:00	4213.6414	2830.3274
0:00 储电量		9024.4209	24:00 储电量		4904.9757

表 9: 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
20:30–20:40	178.4605	—	0.0000	20:20–20:30	6.3750	—	0.0000
—	—	—	—	20:50–21:10	36.7777	—	—
—	—	—	—	21:20–21:50	23.8190	—	—

四个指定日期的全天正常购电量分别为 63841.8084、42190.0469、69318.8824 和 97884.6594 kWh。其中，3 月 20 日发生 178.4605 kWh 的紧急购电；9 月 23 日共有三个连续缺口事件，紧急购电量合计 66.9717 kWh；6 月 21 日和 12 月 21 日的供需偏差均可由储能在容量和功率约束内完成补偿，未触发紧急购电。四日的日初、日末储电量并不要求分别相等，而是按照运行模拟结果跨日连续传递，与问题一的单日闭环条件有所不同。

对全部 334 天结果进行汇总，总购电费用为 1399.7990 万元，具体购电量及费用构成见表 10。

表 10: 2025 年 2 月 1 日至 12 月 31 日购电结果汇总

正常购电量 (kWh)	紧急购电量 (kWh)	正常购电费 (元)	紧急购电费 (元)	总购电费 (元)
21647262.3826	123699.5020	13259252.2664	738737.8952	13997990.1616

紧急购电量仅占正常购电与紧急购电总量的 0.568%，但由于其单位电价为正常购电的 5 倍，对应费用占总购电费用的 5.278%。因此，即使紧急购电在统计期购电量中所占比例较低，减少由净负载低估造成的短时供电缺口仍能对统计期费用产生明显影响。

为检验历史残差分位数修正的实际作用，在保持负载和光伏发电功率预测方法、48 h 滚动窗口、储能执行规则及费用口径不变的条件下，设置完全关闭残差修正的对照方案，结果见表 11。

表 11: 历史残差分位数修正的组件对照结果

方案	正常购电费 (元)	紧急购电费 (元)	总购电费 (元)	紧急购电量 (kWh)
无残差修正	12196180.8903	3301581.8627	15497762.7530	545200.1094
80% 分位数修正	13259252.2664	738737.8952	13997990.1616	123699.5020
变化量	+1063071.3761	-2562843.9675	-1499772.5914	-421500.6074

关闭残差修正时，总购电费用为 1549.7763 万元。其中紧急购电费用达到 330.1582 万元，采用 80% 分位数修正后，正常购电费用增加 106.3071 万元，但紧急购电费用减少 256.2844 万元，使总费用净下降 149.9773 万元 (9.68%)，紧急购电量同时减少 42.1501 万 kWh。可见，适量增加计划购电能够有效规避 5 倍电价的紧急购电。

图 7 比较两种方案每日紧急购电费用的分布及较大费用出现的情况。

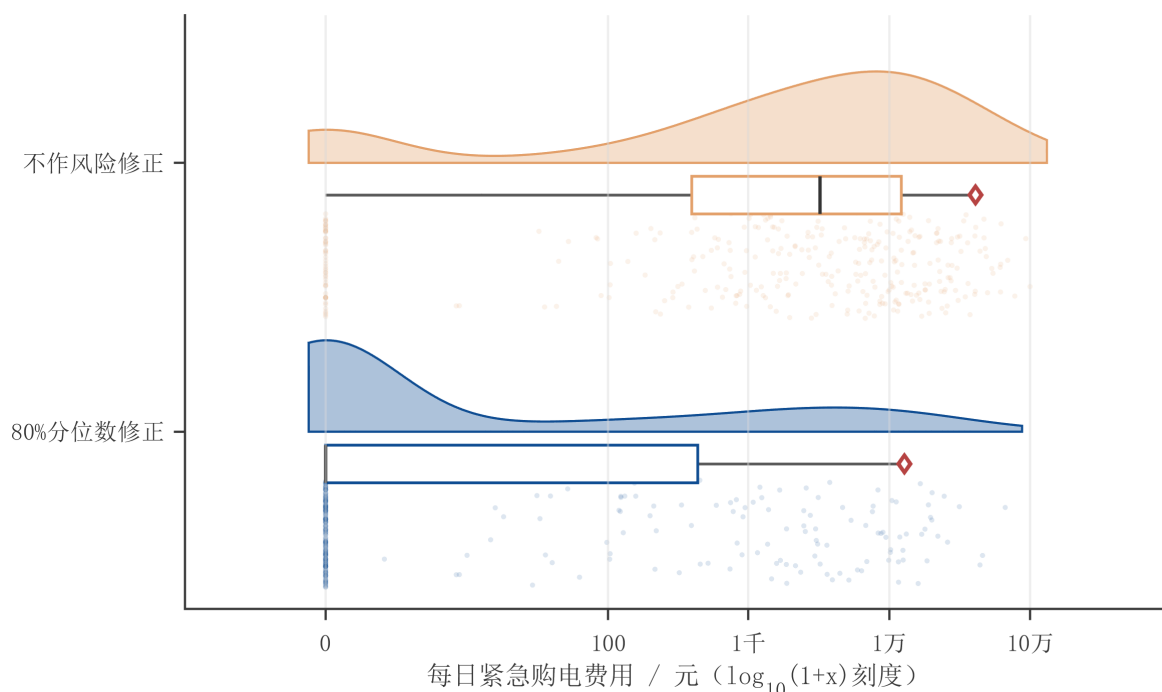


图 7: 无残差修正与 80% 分位数修正下的紧急购电分布

进一步保持风险修正机制不变，仅改变历史残差经验分布的分位水平

$$\tau \in \{0, 0.6, 0.7, 0.8, 0.9, 0.95\},$$

得到图 8 所示的敏感性结果。此处 $\tau = 0$ 表示采用残差经验分布的 0 分位数作为修正量，并不等同于表 11 中完全关闭残差修正的对照方案。

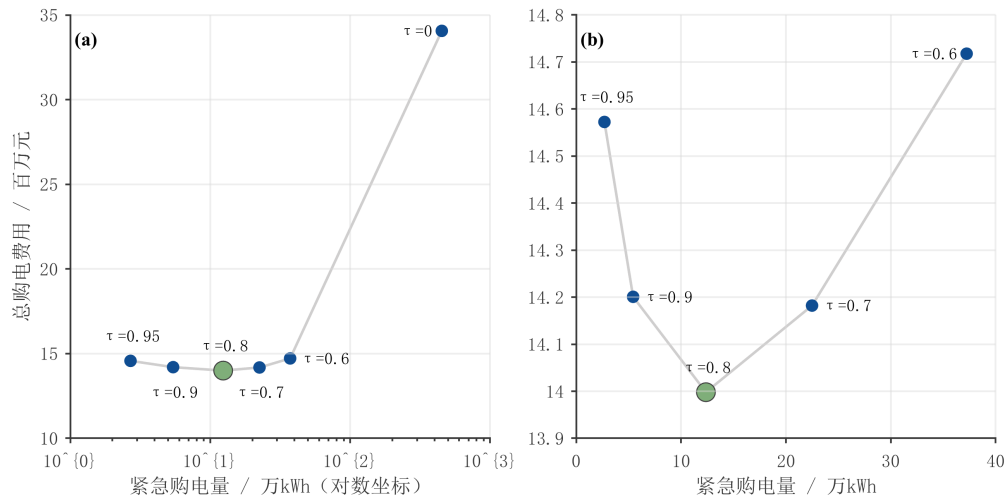


图 8: 不同历史残差分位水平下的费用—紧急购电量权衡

在检验的六个分位水平中，80% 对应的总费用最低；进一步提高分位水平虽减少紧急购电，但正常购电费用增加。

由此，历史残差分位数修正在保持滚动调度可行的同时，有效降低了预测偏低所引起的紧急购电成本。

八、问题三：日内预报更新与负载反馈下的滚动优化

问题二的购电计划在每日 0:00 一次确定，无法利用当天随后获得的新预报和实际运行信息。问题三进一步允许在 6:00、12:00 和 18:00 对尚未执行的购电计划进行调整，因此需要在保持固定电价、储能运行约束和紧急购电规则不变的基础上，将日内信息更新引入滚动调度过程。

本文建立**基于日内信息更新与负载反馈的滚动时域优化模型**^[7]。每日 0:00 先形成初始购电与储能计划；在 6:00、12:00 和 18:00，利用当时最新的光伏发电功率预报、实际储电量和已观测负载，对尚未执行的时段重新优化，并采用反馈系数 $\alpha = 0.5$ 修正后续负载预测。已执行时段保持不变，实际供需偏差仍优先由储能调节，剩余缺口通过紧急购电补足。问题三的求解流程如图 9 所示。

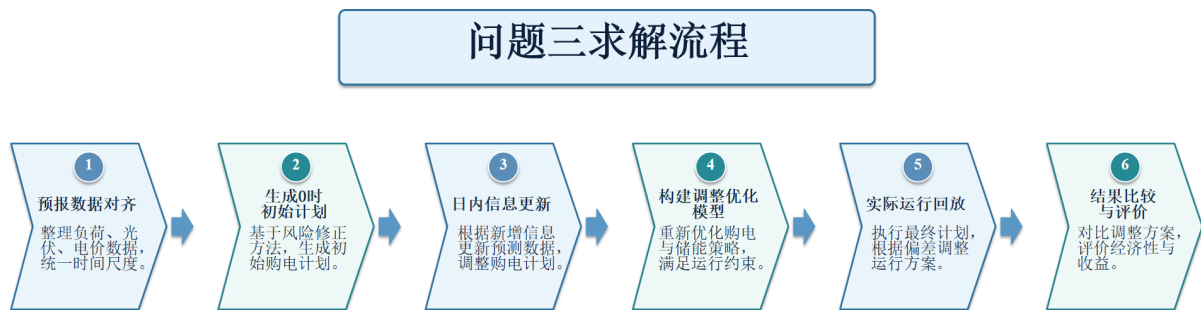


图 9: 问题三日内预报更新与购电调整流程

8.1 光伏预报对齐与因果负载反馈

为与本文采用的 10 分钟调度尺度一致，将预报发布时间记为 v ，其取值为每日 0:00、6:00、12:00 和 18:00。每次预报均以发布时间对应的实际光伏发电功率作为起点，其中 0:00 的起点取前一日 24:00 的实际观测值，其余时刻取当日 v :00 的实际观测值。设发布时间 v 后第 h 小时的光伏发电功率预报为 $F_{i,h}^{(v)}$ ($h = 1, \dots, 24$)，起点记为 $F_{i,0}^{(v)}$ 。

对于发布后的第 s 个 10 分钟时段，令 $x_s = s/6$ ，采用分段线性插值将逐小时预报映射到 10 分钟时间网格：

$$\begin{aligned}\widehat{G}_{i,s}^{(v)} &= (1 - \lambda_s)F_{i,m_s}^{(v)} + \lambda_s F_{i,m_s+1}^{(v)}, \\ m_s &= \lfloor x_s \rfloor, \quad \lambda_s = x_s - m_s.\end{aligned}\tag{26}$$

当 $x_s = 24$ 时，直接取 $F_{i,24}^{(v)}$ 。该处理利用相邻整点预报进行线性过渡，使逐小时光伏发电功率预报能够平滑对应到各 10 分钟调度时段。

在时刻 v 更新计划时，当日已有 $6v$ 个 10 分钟时段执行完毕，因此新获得的光伏发电功率预报只作用于尚未执行的 $144 - 6v$ 个时段。已执行时段的购电计划和实际运行结果保持不变；预报窗口中超出当日范围的部分仅用于滚动优化中的后续时段，不修改已执行时段的计划。

负载预测则以 0:00 形成的历史同星期加权预测 $\widehat{L}_{i,t}^{(0)}$ 为基准。6:00、12:00 和 18:00 更新时，利用已发生时段的实际负载与基准预测之间的偏差修正当日剩余时段。定义已执行时段的累计负载比值为

$$R_i^{(v)} = \frac{\sum_{t=1}^{6v} L_{i,t}}{\sum_{t=1}^{6v} \widehat{L}_{i,t}^{(0)}}.\tag{27}$$

正式模型取负载反馈系数 $\alpha = 0.5$ ，将该比值按反馈强度作用于当日尚未执行时段的负载预测，得

$$\widehat{L}_{i,t}^{(v)} = \left[1 + \alpha \left(R_i^{(v)} - 1\right)\right] \widehat{L}_{i,t}^{(0)}, \quad v \in \{6, 12, 18\}, \quad t = 6v + 1, \dots, 144.\tag{28}$$

0:00 时尚无当日实际负载可供反馈，因此直接采用基准预测；6:00、12:00 和 18:00 的反馈修正仅使用已经观测到的负载，并只作用于当日剩余时段。跨越至次日的负载预测仍采用原有历史预测结果，不使用当日尚未发生的实际信息。

由此，在每个日内更新时间均可获得与当前信息一致的负载和光伏发电功率预测，为后续未执行时段的购电计划调整提供输入。

8.2 购电调整费用与未执行时段约束

在 6:00、12:00 和 18:00 获得新信息后，仅允许调整尚未执行的购电计划，已经完成交付的时段保持不变。记 $q_{i,t}^{(v)}$ 为第 i 日在时刻 $v \in \{0, 6, 12, 18\}$ 确定的第 t 时段正常购电量。时刻 v 到来时，当日已有 $6v$ 个 10 分钟时段执行完毕，可调整时段集合为

$$\mathcal{T}_v = \{6v + 1, 6v + 2, \dots, 144\}.\tag{29}$$

对于已执行时段，购电量维持上一版本不变：

$$q_{i,t}^{(v)} = q_{i,t}^{(\text{prev}(v))}, \quad t \leq 6v,\tag{30}$$

其中， $\text{prev}(v)$ 表示时刻 v 之前最近一次计划更新时间。尚未执行的时段则依据最新光伏发电功率预报、负载预测和当前储电量重新优化。因此，当日各时段最终生效的正常购电量为

$$q_{i,t}^f = \begin{cases} q_{i,t}^{(0)}, & 1 \leq t \leq 36, \\ q_{i,t}^{(6)}, & 37 \leq t \leq 72, \\ q_{i,t}^{(12)}, & 73 \leq t \leq 108, \\ q_{i,t}^{(18)}, & 109 \leq t \leq 144. \end{cases} \quad (31)$$

每次更新仍从当前时刻向后保留 24 h 优化窗口，其中超出当日的部分仅用于约束后续储能安排，不修改已执行时段的计划。

题目规定，日内购电调整统一以每日 0:00 形成的原始计划 $q_{i,t}^{(0)}$ 为结算基准。定义上调量和下调量分别为

$$\begin{aligned} u_{i,t}^+ &= \max(q_{i,t}^f - q_{i,t}^{(0)}, 0), & u_{i,t}^- &= \max(q_{i,t}^{(0)} - q_{i,t}^f, 0), \\ q_{i,t}^f &= q_{i,t}^{(0)} + u_{i,t}^+ - u_{i,t}^-. \end{aligned} \quad (32)$$

正常购电结算费用 C_i^N 由 0:00 原计划购电费用与购电调整费用构成；再加紧急购电费用，得到总购电费用。中间各次调整不重复计费，而以最终正常购电量与 0:00 原始计划之间的差额统一结算。若下调 $u_{i,t}^-$ ，取消部分退回原购电费用，同时支付该部分购电费用 50% 的违约费用；若上调 $u_{i,t}^+$ ，新增部分按对应时段电价的 1.5 倍计费。因此，第 i 日正常购电结算费用为

$$C_i^N = \sum_{t=1}^{144} \left[p_t q_{i,t}^{(0)} - 0.5 p_t u_{i,t}^- + 1.5 p_t u_{i,t}^+ \right]. \quad (33)$$

其中， $-0.5 p_t u_{i,t}^-$ 表示下调后相对于原计划减少的净支出，并非额外收益。

运行模拟沿用第 7.3 节的储能反馈与紧急购电规则，并在式 (23) 中采用最终正常购电量 $q_{i,t}^f$ 。储能仍无法补足的缺口 $e_{i,t}$ 按同期电价的 5 倍购入，第 i 日总购电费用为

$$C_i = C_i^N + \sum_{t=1}^{144} 5 p_t e_{i,t}. \quad (34)$$

上述结算方式使日内更新带来的正常购电调整成本与实际运行中的紧急购电成本能够在同一费用口径下比较，为后续评价日内预报更新的经济收益提供基础。

8.3 多时点滚动求解与日内更新收益

在每日 0:00 制定初始购电计划后，分别于 6:00、12:00 和 18:00 利用新发布的光伏发电功率预报、已观测负载偏差及实际储电量，对尚未执行时段重新求解。以下结果均采用最终确定的负载反馈系数 $\alpha = 0.5$ ，并按 8.2 节的购电调整与紧急购电结算规则统计。

表 12、表 13 展示 3 月 20 日的购电与储能结果，表 14 汇总四个指定日期的紧急购电事件；其他指定日期结果见附录 A.2。

表中“全天购电量”为最终正常购电量；“全天购电费”为 0:00 原计划购电费用、购电调整费用与紧急购电费用之和。

表 12: 微网在指定时间段的正常购电量及全天购电量和购电费

2025 年 3 月 20 日					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.0000	12:00–12:10	552.9497	14:00–14:10	0.0000
16:00–16:10	532.7965	18:00–18:10	703.0559	20:00–20:10	0.0000
全天购电量 (kWh)		63013.1004		全天购电费 (元)	
				41059.8250	

表 13: 储能设备在指定时间段的充放电量及 0:00、24:00 的储电量

2025 年 3 月 20 日					
时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	1372.8360	190.5300	4:00–8:00	934.3725	5813.6763
8:00–12:00	4340.0468	2777.3772	12:00–16:00	6268.4752	287.2112
16:00–20:00	256.5956	5637.9852	20:00–24:00	4501.4306	3002.6417
0:00 储电量 (kWh)		9059.0018	24:00 储电量 (kWh)		5288.2477

表 14: 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
—	0.0000	—	0.0000	20:20–20:30	6.3750	7:50–8:00	1.6725
—	—	—	—	—	—	—	—
—	—	—	—	—	—	—	—

四个指定日期中，3 月 20 日和 6 月 21 日的供需偏差均由储能完成补偿，未发生紧急购电；9 月 23 日和 12 月 21 日分别仅发生 6.3750 kWh 与 1.6725 kWh 的紧急购电。各日储电量不再要求回到固定初值，而是按实际运行状态跨日传递。

表 15 汇总统计期结果。日内更新后，最终正常购电量为 21120897.4341 kWh，紧急购电量为 23339.6808 kWh。上调、下调购电量及费用分项亦列于表中；下调净退费是取消原购电费用并支付该部分 50% 违约费用后的净额。

表 15: 问题三统计期内购电量与费用汇总

最终正常购电量 (kWh)	上调购电量 (kWh)	下调购电量 (kWh)	紧急购电量 (kWh)
21120897.4341	307205.5101	1142817.3223	23339.6808
0:00 原计划购电费用 (元)		下调净退费 (元)	上调购电费用 (元)
13482376.9554		368942.6983	354960.6433
正常购电结算费用 (元)		紧急购电费用 (元)	总购电费用 (元)
13468394.9004		131583.9912	13599978.8916

为展示 0:00 初始计划在各次更新后对尚未执行时段的调整，图 10 给出典型日期的购电计划及相对初始计划的变化。

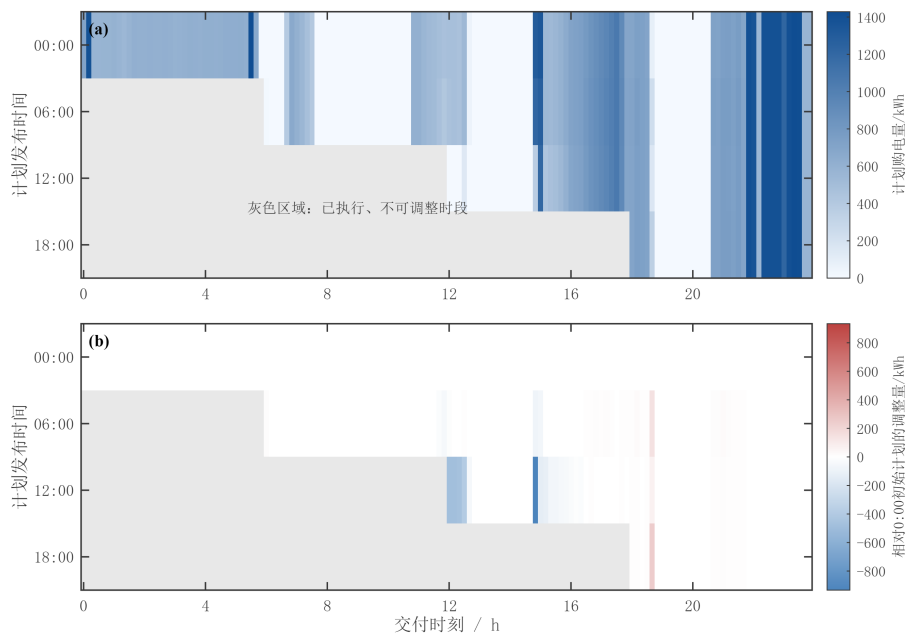


图 10: 不同发布时间下的购电计划及相对 0:00 计划的调整量

为公平衡量日内更新的作用,将最终方案与“仅在 0:00 制定计划、日内不再更新”的对照方案比较。后者沿用相同的预测、储能约束和费用规则,仅关闭 6:00、12:00 与 18:00 的计划更新。引入三次日内更新后,统计期总购电费用由 1420.5843 万元降至 1359.9979 万元,节省 60.5864 万元;紧急购电量由 11.9904 万 kWh 降至 2.3340 万 kWh,减少 9.6564 万 kWh。

进一步对 {6:00, 12:00, 18:00} 的全部八种更新时间组合进行运行模拟;完整费用、紧急购电量及基于 **Shapley 值**^[8] 的分解结果见附录 A.3。其中节省费用均相对于仅在 0:00 制定计划的对照方案计算。

如图 11 所示,八种组合中,6:00、12:00、18:00 全部更新时总费用最低。仅在 12:00 更新时节省 41.8790 万元,其 Shapley 平均边际费用贡献为 28.6008 万元,均为三个时点中最高;6:00 平均减少紧急购电 4.6473 万 kWh,为三个时点中最多,说明各更新时间的作用具有互补性。

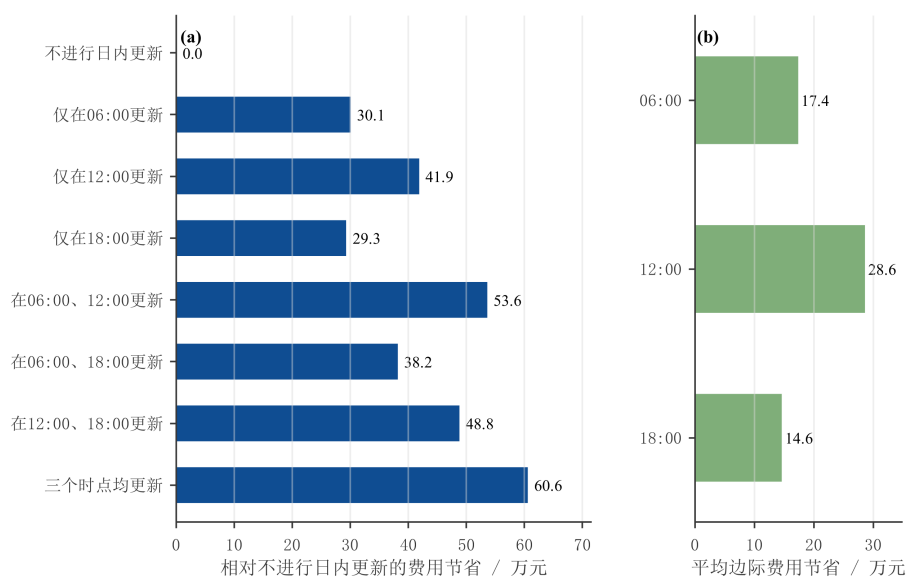


图 11: 日内更新时间组合的费用与紧急购电表现

8.4 信息模块贡献与负载反馈强度分析

8.3 已比较不同更新时间组合的作用。为了区分日内更新收益的具体来源，在固定 6:00、12:00 和 18:00 三个更新时间的条件下，分别控制“实际储电量更新”“新光伏发电功率预报”和“负载反馈”三个信息模块的启用状态，形成完整的 2^3 组合对照实验。各组实验均保持购电调整规则、储能约束、风险修正和跨日储电量传递方式不变，完整组合结果见附录 A.3。

三类信息全部启用时，总购电费用较全部关闭减少 60.5864 万元，紧急购电量由 11.9904 万 kWh 降至 2.3340 万 kWh。单独启用实际储电量更新、新光伏发电功率预报和负载反馈时的节省费用分别为 38.3232、15.5837 和 13.6381 万元。为避免加入顺序影响评价，采用 Shapley 值计算平均边际费用贡献。实际储电量更新、新光伏发电功率预报和负载反馈分别贡献 28.7126、18.4259 和 13.4479 万元，其中实际储电量更新贡献最大；完整结果见附录 A.3。

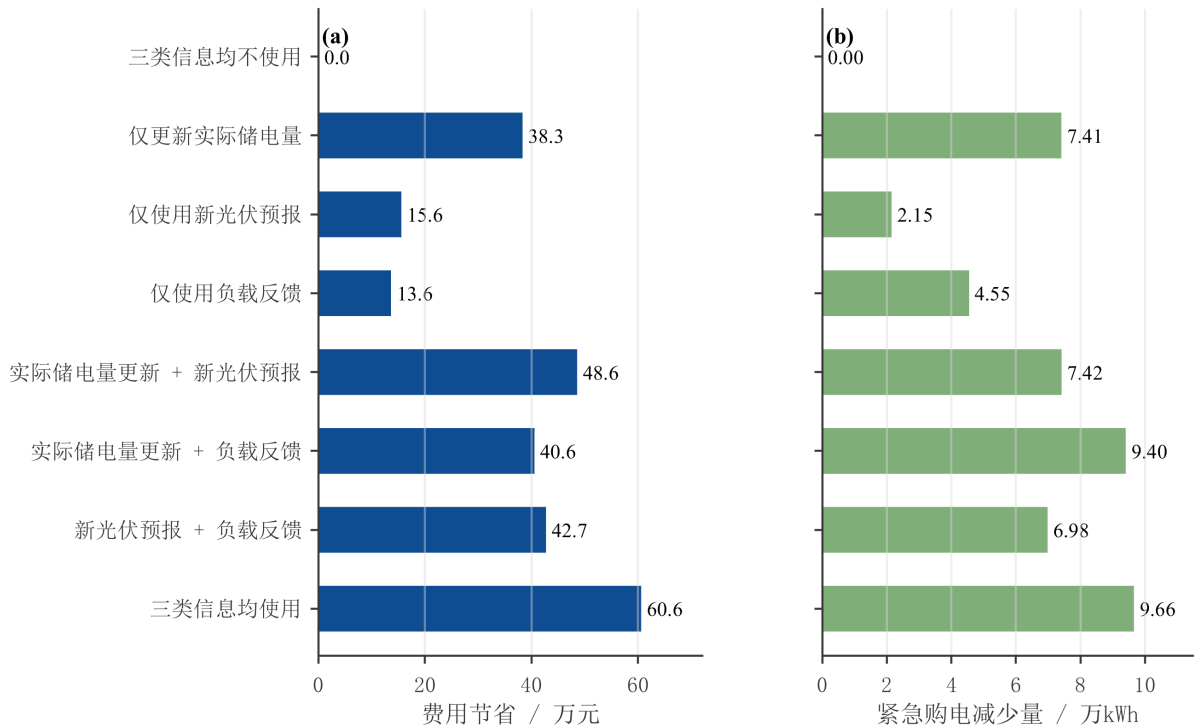


图 12: 三类日内信息模块组合的费用与紧急购电表现

图 12 表明，三个模块共同启用时费用节省最大，且实际储电量更新在单独启用时已带来最显著改善，和 Shapley 分解的排序一致。

分别取 $\alpha = 0$ 、0.25、0.50、0.75 和 1.00 五个取值，完整敏感性数值见附录 A.3。

如图 13 所示，在五个取值中，紧急购电量随 α 增大而下降，总费用则先降后升， $\alpha = 0.50$ 时最低，为 1359.9979 万元。继续提高反馈强度虽减少紧急购电，却增加购电调整净支出，使总费用回升。

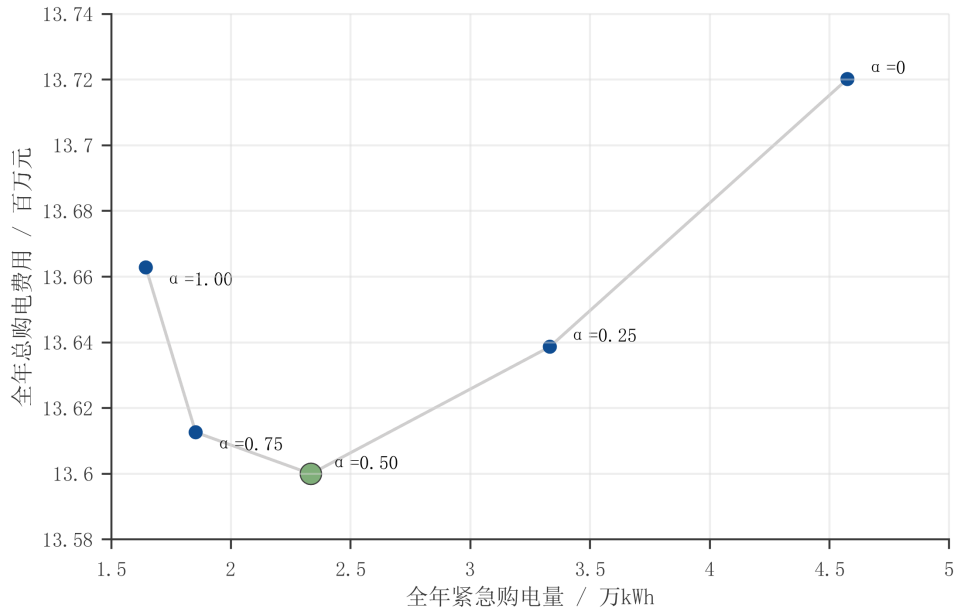


图 13: 不同负载反馈系数下的费用—紧急购电量权衡

时间顺序检验中, $\alpha = 0.62$ 在验证期比 0.50 节省 3286.23 元, 但在后续留出期和波动电价场景中分别多支出 3516.12 元和 3775.16 元, 其优势未能持续, 因此仍采用 $\alpha = 0.50$; 样本划分、候选网格与后续检验用途见附录 A.3。

九、问题四：波动电价下的购电策略重优化

前三问采用固定分时电价, 问题四进一步考虑电价随日期和时段波动, 因此需在计划阶段形成预测电价, 并按实际电价结算。

对于仅在 0:00 制定计划的情形, 沿用问题二的信息边界和风险修正机制; 对于允许日内调整的情形, 则继承问题三在 6:00、12:00 和 18:00 的光伏预报更新、实际储电量传递及 $\alpha = 0.5$ 的负载反馈, 日内不再更新 0:00 形成的价格预测。最后, 将重新优化方案与“保持原策略、仅按波动电价重新计费”的结果比较, 以衡量主动调整购电策略带来的收益。问题四的求解流程如图 14 所示。



图 14: 问题四波动电价下的购电策略重优化流程

9.1 历史电价预测与实际交付结算

每日 0:00 制定购电计划时，当日及次日实际电价尚不可知，因此计划阶段只能依据历史价格形成预测电价；实际运行结束后，再按照各 10 分钟时段的实际电价结算。考虑到电价通常具有明显的日内和周内周期性^[9]，本文以同星期历史电价曲线构造预测价格。

设第 i 日第 t 个时段的实际电价为 $p_{i,t}$ 。对目标日期 i ，从此前 28 个完整自然日中筛选与目标日期星期相同的历史日期，记为 $\mathcal{H}_i$ ，并以对应时段的历史均值作为电价预测：

$$\hat{p}_{i,t} = \frac{1}{|\mathcal{H}_i|} \sum_{j \in \mathcal{H}_i} p_{j,t}, \quad \mathcal{H}_i = \{j \mid \max(1, i - 28) \leq j < i, \text{weekday}(j) = \text{weekday}(i)\}. \quad (35)$$

$\hat{p}_{i,t}$ 仅由计划时刻以前已经获得的价格记录计算。48 h 滚动窗口中的次日电价采用相同方法预测，不使用第 i 日及第 $i + 1$ 日尚未发生的实际价格。

采用平均绝对误差（MAE）和均方根误差（RMSE）评价预测准确性^[10]，并补充报告加权绝对百分比误差（WAPE）。滚动验证中，三项指标分别为 **0.0489 元/kWh**、**0.0696 元/kWh** 和 **6.45%**，均低于前一日同一时段延续预测，完整结果及指标定义见附录 A.4。图 15 进一步给出了预测值与实际值的对应关系。

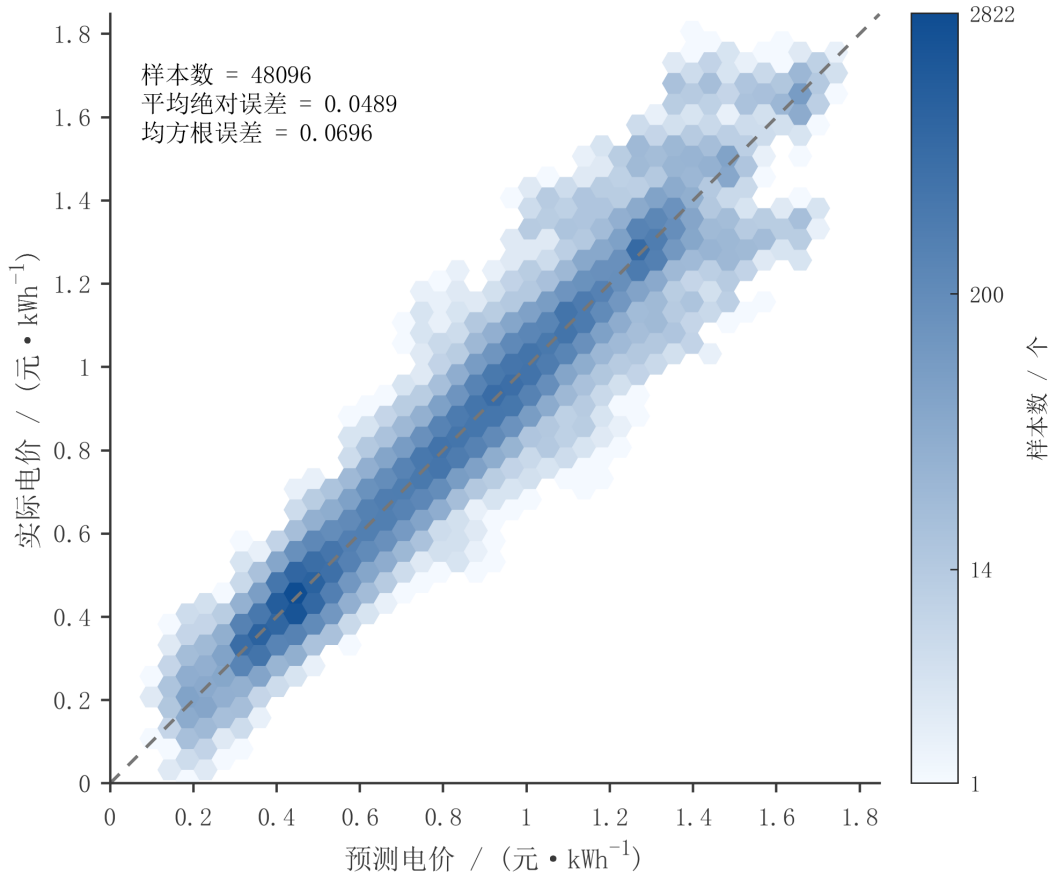


图 15: 历史同星期电价预测与实际电价的对应关系

全年实际电价介于 0.0076 ~ 1.7936 元/kWh，均值为 0.7575 元/kWh。较明显的日内及跨日价格差异为重新安排购电与储能时段提供了空间。

实际交付时，费用统一按照对应时段的实际电价计算。对于仅在 0:00 制定计划的情形，第 i 日购电费用为

$$C_i^{(0)} = \sum_{t=1}^{144} p_{i,t} (q_{i,t}^0 + 5e_{i,t}), \quad (36)$$

其中, $q_{i,t}^0$ 和 $e_{i,t}$ 分别为正常购电量和紧急购电量。预测电价只用于计划优化, 实际电价仅用于最终费用结算, 从而保证决策过程中不使用未来价格信息。

预测电价随后作为购电与储能重优化的价格输入。

9.2 波动电价下的计划购电策略重新优化

在仅于每日 0:00 制定计划的情形下, 继续沿用问题二的负载预测、光伏发电功率预测、历史净负载残差 80% 分位数修正、48 h 滚动计划以及跨日储电量连续规则, 仅将固定分时电价替换为式 (35) 得到的预测电价。每次求解同时形成未来 48 h 的购电与储能计划, 但只执行当日 144 个时段, 下一日再以实际期末储电量为初始状态重新计算。

按预测电价重新优化后, 统计期总购电费用为 1468.7087 万元, 其中正常购电费用为 1392.7742 万元, 紧急购电费用为 75.9346 万元。为区分电价变化本身与主动调整策略带来的影响, 将问题二原有购电与储能方案保持不变, 仅按问题四的实际波动电价重新计费, 得到 1474.8342 万元的比较基准。主动重新优化使正常购电费用减少 5.1081 万元, 紧急购电费用减少 1.0174 万元, 统计期总费用节省 6.1255 万元, 降幅约为 0.42%。紧急购电量虽增加 1170.25 kWh, 但对应费用下降, 说明其电量加权平均结算电价有所降低。完整购电量与费用分项见附录 A.4。

上述策略仅利用每日 0:00 已获得的信息。进一步允许日内更新后, 可在新的光伏发电功率预报和实际运行状态到达时重新调整尚未执行的购电计划。

9.3 波动电价下的日内购电调整策略

日内调整情形沿用问题三的光伏发电功率预报更新、实际储电量更新及 $\alpha = 0.5$ 的负载反馈机制, 并继续只调整尚未执行的时段。问题四在此基础上的主要变化是引入波动电价, 而日内价格预测仍保持 0:00 形成的结果不变。

对于尚未执行的时段, 有

$$\hat{p}_{i,t}^{(6)} = \hat{p}_{i,t}^{(12)} = \hat{p}_{i,t}^{(18)} = \hat{p}_{i,t}^{(0)}. \quad (37)$$

因此, 6:00、12:00 和 18:00 的计划更新只吸收新的光伏发电功率预报、实际储电量和负载反馈信息, 价格预测始终保持 0:00 的结果不变。

调整购电仍以 0:00 原始计划为结算基准, 最终费用按照各时段实际电价计算。统计期内的购电量与费用构成见表 16。

表 16: 波动电价下日内购电调整的统计期结果

指标	数值	指标	数值
原始正常购电量/kWh	21887910.2506	最终正常购电量/kWh	21051179.4837
上调购电量/kWh	311674.8853	下调购电量/kWh	1148405.6523
紧急购电量/kWh	26946.6585	原计划购电费用/元	14166202.73
下调购电结算抵扣/元	-391246.20	上调购电费用/元	377813.31
正常购电结算费用/元	14152769.83	紧急购电费用/元	138553.49
总购电费用/元	14291323.32		

下面将其与保持问题三原策略、仅按实际波动电价重新计费的基准方案比较, 以评价主动重新优化的经济收益。

9.4 重新优化收益与负载反馈作用分析

两种情形下重新优化结果与相应基准方案的完整费用比较见附录 A.4。仅在 0:00 制定计划时，主动重新优化节省 6.1255 万元；允许日内调整时，主动重新优化节省 4.1338 万元。

由于上述比较均采用相同的实际电价进行结算，费用差异主要来自购电与储能时段的重新安排。仅在 0:00 制定计划时，正常购电费用和紧急购电费用均有所下降；允许日内调整时，正常购电结算费用减少 4.5552 万元，而紧急购电费用增加 0.4214 万元，二者合计后统计期总费用仍减少 4.1338 万元。因此，波动电价下重新优化的主要收益来自正常购电与储能运行时段的重新配置，紧急购电量或紧急购电费用并不必然同步下降。两类情形中“原策略重新计价”与“主动重新优化”的费用变化如图 16 所示。

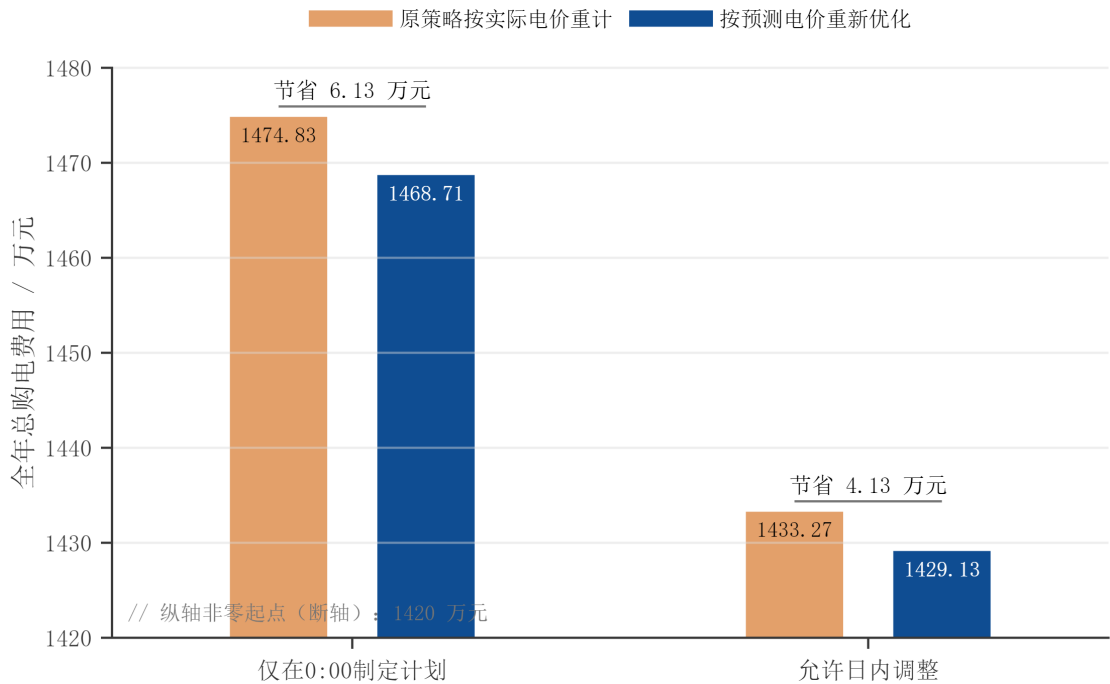


图 16: 原策略重新计价与主动重新优化的费用比较

进一步考察负载反馈和日内电价再修正对结果的影响。保持新光伏发电功率预报、实际储电量更新、费用结算和储能约束不变，分别改变负载反馈系数以及是否利用日内已实现价格再次修正后续预测；再修正公式、历史窗口与参数见附录 A.4。结果见表 17。

表 17: 负载反馈与日内电价再修正的对照结果

负载反馈系数	日内电价再修正	总购电费用（元）	紧急购电量（kWh）
0	关闭	14421755.29	49580.83
0	启用	14421117.79	49163.54
0.5	关闭	14291323.32	26946.66
0.5	启用	14293311.43	26988.87

关闭日内电价再修正时， $\alpha = 0.5$ 使总费用减少 13.0432 万元、紧急购电量减少 2.2634 万 kWh；启用价格再修正后，负载反馈仍可节省 12.7806 万元。相比之下，日内价格再修正在 $\alpha = 0$ 时仅节省 637.50 元，在 $\alpha = 0.5$ 时反而增加 1988.11 元，因此正式方案不再日内修正电价。

十、模型独立核验与结果可比性分析

前文给出了不同条件下的购电与储能调度结果。为检验结果的可复现性及不同方案之间的可比性，本章分别复算费用与运行约束、检查决策信息，并校正期末储电量差异。

10.1 费用与储能运行约束的独立复算

将最终逐时段结果与原始数据重新匹配，在不调用原求解程序的条件下，依据既定结算规则独立复算费用，并检查供需平衡、储电量递推及储能运行约束，结果见表 18。问题一、问题二记录汇总费用复算差，问题三和问题四记录逐时段费用公式的最大差。

表 18: 各问题费用独立复算与储能运行综合核验

情形	费用复算差 (元)	复算范围	供需平衡最大 残差 (kWh)	储电量递推最 大残差 (kWh)	容量/功率/同时充放 电违规数
问题一	1.60×10^{-10}	汇总	1.00×10^{-10}	1.22×10^{-10}	0/0/0
问题二	1.86×10^{-9}	汇总	6.96×10^{-13}	4.55×10^{-12}	0/0/0
问题三	6.82×10^{-13}	逐时段	6.82×10^{-13}	4.55×10^{-12}	0/0/0
问题四：仅 0:00 计划	1.82×10^{-12}	逐时段	6.96×10^{-13}	4.55×10^{-12}	0/0/0
问题四：日内调 整	9.09×10^{-13}	逐时段	6.82×10^{-13}	4.55×10^{-12}	0/0/0

表中 0/0/0 依次表示储电量越界次数、充放电功率超限次数和同时充放电时段数。五种情形均未发生储电量越界、充放电功率超限或同时充放电；供需平衡与储电量递推的最大残差均处于数值误差量级。费用独立复算结果与正式结果一致，说明各方案的费用计算和储能运行状态均可由最终逐时段结果独立复现。

10.2 信息截止、时段锁定与跨日连续性核查

数值复算能够验证费用与约束是否一致，但还需检查滚动决策过程中是否使用了决策时刻之后的真实信息。为此，在四个季节代表日的相应决策时刻保持已有历史数据、预测信息和实际储电量不变，仅扰动此后尚未发生的实际负载、光伏发电功率或实际电价，并比较扰动前后的购电计划。

表 19: 信息截止、时段锁定与跨日连续性核验

情形	扰动测试数	最大计划 变化 (kWh)	信息越界 次数	跨日储电量 最大断点 (kWh)	已执行时段锁定情 况
问题二	8	0	0	0	仅在 0:00 制定计划
问题三	32	0	0	0	6:00、12:00、18:00 均无改动
问题四：仅 0:00 计划	12	0	0	0	仅在 0:00 制定计划
问题四：日内调 整	48	0	0	0	6:00、12:00、18:00 均无改动

100 项未来真实信息扰动测试中，当前购电计划最大变化均为 0，未发现未来真实负载、光伏发电功率或实际电价提前进入决策。问题三和问题四的日内更新均未修改已

执行时段，问题二至问题四的跨日储电量最大断点均为 0，说明信息截止、时段锁定和跨日连续性均满足要求。

10.3 期末储电量价值校正与方案排序

不同策略在统计期末可能保留不同的储电量。若仅比较现金购电费用，期末储电量较低的方案可能因提前释放更多电能而获得额外的账面优势。为检验这一差异是否会改变方案排序，对期末储电量进行价值折算。

设对照方案与待比较方案的期末储电量分别为 E_T^{ref} 和 E_T^{new} ，现金费用节省额为 ΔC_{cash} 。以剩余储电量按平均电价释放后的可利用价值进行校正，可得

$$\Delta C_{\text{inv}} = \Delta C_{\text{cash}} + \eta_d \bar{p} (E_T^{\text{new}} - E_T^{\text{ref}}), \quad (38)$$

其中， $\eta_d = 0.9$ 为放电效率， $\bar{p}$ 为相应情形下的平均电价。当待比较方案的期末储电量较低时，其少保留的储能价值将从现金费用优势中扣除；期末储电量较高时则作相反调整。

为检验方案排序在更严格估值下是否稳定，再以 5 倍最大电价对期末储电量差异进行保守折算：

$$\Delta C_{\text{cons}} = \Delta C_{\text{cash}} - \eta_d \cdot 5p_{\max} |E_T^{\text{new}} - E_T^{\text{ref}}|. \quad (39)$$

两种校正方式下的完整比较结果见附录 A.5，各组方案均具有相同的初始储电量。

标准价值校正后，问题三和问题四日内调整方案仍分别节省 12.0067 万元和 12.9702 万元；采用更保守的估值后仍保持正收益，方案排序均未改变，完整结果见表 31。

期末储电量价值校正仅用于检验相同任务、相同初始状态下不同策略的公平可比性。问题一至问题四报告的正式购电费用仍采用实际费用结算结果，不以校正后的数值替代。

十一、模型评价、改进推广与结论

11.1 模型优点

(1) **优化目标与调度要求相匹配。**词典序优化先最小化购电费用，再以储能总吞吐量在给定购电费用容差范围内进一步择优。与显式设置充放电互斥约束的混合整数线性规划交叉验证表明，两种模型的费用与调度结果一致。

(2) **预测误差直接进入购电决策。**风险修正使计划需求在正常购电成本与高价紧急购电风险之间取得平衡。残差修正的对照结果显示，80% 分位数在所检验的分位水平中对应最低总费用。

(3) **能够利用日内新增信息动态调整计划。**信息模块对照与 Shapley 分解表明，在本算例中，实际储电量更新、光伏发电功率预报与负载反馈的 Shapley 平均边际费用贡献均为正，联合使用时总费用最低。

(4) **决策过程具有明确的信息边界和核验机制。**各时刻仅使用当时已获得的信息，且储电量在各时段与日期之间连续传递。100 项未来真实信息扰动测试均未改变当前决策；费用独立复算与原结果一致，期末储电量价值校正后方案排序不变。

11.2 模型局限

(1) 负载、光伏发电功率和电价预测主要依赖历史周期性与近期数据，对节假日、极端天气及价格结构突变等异常情形的刻画仍较有限。

(2) 储能模型主要考虑容量、充放电功率和转换效率，未进一步计入自放电、电池老化及温度变化等因素，因此当前目标函数主要反映购电与调整费用，尚未完整描述长

期运行成本。

(3) 模型将微网视为单一供需节点，未进一步考虑线路容量、网损、电压约束及多个微网之间的能量交换，因此结果更适用于题目给定的汇总能量调度场景。

11.3 改进与推广

可引入天气、节假日等外生变量，在保持信息边界的条件下开展概率预测与多场景滚动优化。储能模型可补充老化、自放电及温度影响，兼顾短期购电费用与长期使用成本。获得网络参数后，可进一步考虑线路容量、网损和电压约束，并扩展至多微网或共享储能场景。

11.4 主要结论

(1) 在负载、光伏发电功率和电价均已知的条件下，词典序线性规划得到全天正常购电量 59482.6990 kWh，购电费用 35126.95 元。与无储能方案相比，购电费用降低 26.90%，弃光量由 6247.9630 kWh 降至 0，表明储能能够通过跨时段转移电量提高光伏利用率并降低高价时段购电需求。

(2) 在每日 0:00 提前制定计划的条件下，采用历史净负载预测残差的 80% 分位数修正后，2025 年 2 月 1 日至 12 月 31 日总购电费用为 **1399.7990 万元**。与关闭残差修正相比，总费用降低约 **9.68%**，说明适度的风险修正能够有效减少预测偏低造成的高价紧急购电。

(3) 引入 6:00、12:00 和 18:00 的日内更新后，总购电费用降至 **1359.9979 万元**。相较采用相同预测与结算规则、但不进行日内更新的对照方案，总购电费用减少 **60.5864 万元**，紧急购电量减少 **9.6564 万 kWh**。负载反馈系数取 $\alpha = 0.5$ 时，在降低紧急购电与控制调整费用之间取得较好的平衡。

(4) 在波动电价条件下，仅在 0:00 制定计划和允许日内调整两种情形的总购电费用分别为 **1468.7087 万元** 和 **1429.1323 万元**。与保持原策略、仅按实际电价重新计费相比，主动重新优化分别节省 **6.1255 万元** 和 **4.1338 万元**。在是否进行日内电价再修正的两种条件下，负载反馈均降低了总购电费用，而日内再次修正价格预测未表现出稳定收益。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于论文语言润色、表达优化及部分代码与图表辅助，以提升论文的准确性和阅读体验；模型、数据、计算结果与结论均由参赛队员人工核验确认。AI 工具的具体使用情况、典型交互及人工修改与核验过程详见支撑材料《AI 工具使用详情.pdf》。

参考文献

- [1] BERTSIMAS D, TSITSIKLIS J N. Introduction to linear optimization[M]. Belmont: Athena Scientific, 1997.
- [2] EHRGOTT M. Multicriteria optimization[M]. 2nd ed. Berlin: Springer-Verlag, 2005.
- [3] NEMHAUSER G L, WOLSEY L A. Integer and combinatorial optimization[M]. New York: Wiley, 1988.
- [4] HYNDMAN R J, FAN Y. Sample quantiles in statistical packages[J]. The American Statistician, 1996, 50(4): 361–365.
- [5] MAYNE D Q, RAWLINGS J B, RAO C V, et al. Constrained model predictive control: Stability and optimality[J]. Automatica, 2000, 36(6): 789–814.
- [6] SILVENTE J, KOPANOS G M, PISTIKOPOULOS E N, et al. A rolling horizon optimization framework for the simultaneous energy supply and demand planning in microgrids[J]. Applied Energy, 2015, 155: 485–501.
- [7] PARISIO A, RIKOS E, GLIELMO L. A model predictive control approach to microgrid operation optimization[J]. IEEE Transactions on Control Systems Technology, 2014, 22(5): 1813–1827.
- [8] SHAPLEY L S. A value for n-person games[C]//KUHN H W, TUCKER A W, eds. Contributions to the Theory of Games II. Princeton: Princeton University Press, 1953: 307–317.
- [9] WERON R. Electricity price forecasting: A review of the state-of-the-art with a look into the future[J]. International Journal of Forecasting, 2014, 30(4): 1030–1081.
- [10] HYNDMAN R J, KOEHLER A B. Another look at measures of forecast accuracy[J]. International Journal of Forecasting, 2006, 22(4): 679–688.

附录 A：补充结果与图表

附录 A.1：问题二其余指定日期结果

表 20: 问题二其余指定日期微网正常购电结果

2025 年 6 月 21 日					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.0000	12:00–12:10	0.0000	14:00–14:10	0.0000
16:00–16:10	239.5252	18:00–18:10	0.0000	20:00–20:10	0.0000
全天购电量		42190.0469	全天购电费		23988.9139

2025 年 9 月 23 日					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.0000	12:00–12:10	498.6559	14:00–14:10	0.0000
16:00–16:10	533.3026	18:00–18:10	791.7373	20:00–20:10	0.0000
全天购电量		69318.8824	全天购电费		43133.8132

2025 年 12 月 21 日					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.0000	12:00–12:10	983.4198	14:00–14:10	0.0000
16:00–16:10	855.9184	18:00–18:10	731.1782	20:00–20:10	0.0000
全天购电量		97884.6594	全天购电费		63059.4744

表 21: 问题二其余指定日期储能设备运行结果

2025 年 6 月 21 日					
时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	611.2809	66.5687	4:00–8:00	486.5193	2520.3163
8:00–12:00	8991.6775	0.0000	12:00–16:00	0.0000	0.0000
16:00–20:00	190.1471	5231.4297	20:00–24:00	8974.1694	2485.8966
0:00 储电量		4593.7867	24:00 储电量		10473.0777

2025 年 9 月 23 日					
时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	1825.8687	22.8963	4:00–8:00	541.0884	6060.5489
8:00–12:00	5607.9336	1896.6157	12:00–16:00	4504.5986	541.8748
16:00–20:00	182.0335	5700.5625	20:00–24:00	8440.2965	2933.3395
0:00 储电量		8839.9721	24:00 储电量		8769.5676

2025 年 12 月 21 日					
时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	1767.6823	145.1295	4:00–8:00	1205.5377	1470.0630
8:00–12:00	5718.1534	7027.2864	12:00–16:00	6697.7901	2220.1094
16:00–20:00	3.6562	5627.3300	20:00–24:00	8442.0846	2842.5258
0:00 储电量		9019.2955	24:00 储电量		8990.2158

附录 A.2：问题三其余指定日期结果

表 22: 问题三其余指定日期微网正常购电结果

2025 年 6 月 21 日					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.0000	12:00–12:10	0.0000	14:00–14:10	0.0000
16:00–16:10	162.6189	18:00–18:10	0.0000	20:00–20:10	0.0000
全天购电量（kWh）		41593.0254	全天购电费（元）		23656.7882

2025 年 9 月 23 日					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.0000	12:00–12:10	40.9722	14:00–14:10	0.0000
16:00–16:10	538.4372	18:00–18:10	811.7140	20:00–20:10	0.0000
全天购电量（kWh）		68699.3401	全天购电费（元）		44431.5357

2025 年 12 月 21 日					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.0000	12:00–12:10	935.1050	14:00–14:10	0.0000
16:00–16:10	879.2543	18:00–18:10	729.7332	20:00–20:10	0.0000
全天购电量（kWh）		96923.9463	全天购电费（元）		63374.6243

表 23: 问题三其余指定日期储能设备运行结果

2025 年 6 月 21 日					
时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	611.2807	66.5685	4:00–8:00	582.9662	2506.9533
8:00–12:00	9026.0374	0.0000	12:00–16:00	0.0000	0.0000
16:00–20:00	207.9911	5246.6161	20:00–24:00	8953.5141	2485.8966
0:00 储电量（kWh）		4461.2128	24:00 储电量（kWh）		10453.6739

2025 年 9 月 23 日					
时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	1655.1687	22.8963	4:00–8:00	457.9779	6389.1675
8:00–12:00	6104.6729	1896.6157	12:00–16:00	4545.2207	442.2217
16:00–20:00	119.6002	5630.5020	20:00–24:00	8447.2735	2961.9895
0:00 储电量（kWh）		8993.6020	24:00 储电量（kWh）		8920.0882

2025 年 12 月 21 日					
时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	1705.8616	145.1295	4:00–8:00	1196.5267	1486.5790
8:00–12:00	5329.3511	7704.3271	12:00–16:00	7834.9017	2125.3843
16:00–20:00	0.0000	5588.1236	20:00–24:00	8444.1033	2842.5258
0:00 储电量（kWh）		9074.9341	24:00 储电量（kWh）		9032.3047

附录 A.3: 问题三补充对比与敏感性结果

表 24: 不同日内更新时间组合的费用与紧急购电结果

更新时间组合	总购电费用 (元)	紧急购电量 (kWh)	节省费用 (元)
$\emptyset$	14205843.32	119903.91	0.00
{6:00}	13905332.15	39415.08	300511.17
{12:00}	13787053.10	66740.57	418790.22
{18:00}	13912889.22	52735.09	292954.10
{6:00, 12:00}	13669742.05	37319.47	536101.27
{6:00, 18:00}	13823839.81	23391.26	382003.51
{12:00, 18:00}	13717736.05	52888.02	488107.27
{6:00, 12:00, 18:00}	13599978.89	23339.68	605864.43
Shapley 平均边际贡献			
更新时间	平均节省费用 (元)	平均减少紧急购电量 (kWh)	
6:00	173816.18	46473.21	
12:00	286007.59	18062.08	
18:00	146040.65	32028.93	

表 25: 三类信息模块的组合对照结果

实际储电量 更新	新光伏发电 功率预报	负载反馈	总购电费用 (元)	紧急购电量 (kWh)	较三类信息均不使用时 节省的费用 (元)
关	关	关	14205843.32	119903.91	0.00
开	关	关	13822611.17	45812.41	383232.16
关	开	关	14050006.57	98416.76	155836.75
关	关	开	14069462.81	74389.84	136380.51
开	开	关	13720153.48	45747.04	485689.84
开	关	开	13800257.57	25882.18	405585.75
关	开	开	13778596.30	50075.64	427247.02
开	开	开	13599978.89	23339.68	605864.43

表 26: 三类信息模块的 Shapley 平均边际贡献

模块	平均节省费用 (元)	平均减少紧急购电量 (kWh)
实际储电量更新	287126.24	50472.05
新光伏发电功率预报	184259.18	12073.14
负载反馈	134479.01	34019.03

表 27: 负载反馈系数的五点敏感性结果

α	0:00 原计划 购电费用 (元)	购电调整费用 (元)	紧急购电费用 (元)	总购电费用 (元)	紧急购电量 (kWh)
0.00	13475035.44	-8932.30	254050.34	13720153.48	45747.04
0.25	13478991.78	-26166.34	185851.46	13638676.90	33319.69
0.50	13482376.96	-13982.06	131583.99	13599978.89	23339.68
0.75	13484060.95	23407.39	105168.94	13612637.28	18527.18
1.00	13484954.79	83595.30	94265.50	13662815.60	16445.59

为检验 $\alpha = 0.50$ 的稳健性，补充校准将 2025 年 2 月 1 日至 6 月 30 日作为预热期，7 月 1 日至 9 月 30 日作为验证期，10 月 1 日至 12 月 31 日作为留出期。验证期先在 $\{0, 0.1, \dots, 1.0\}$ 中粗筛，再在 $\{0.50, 0.52, \dots, 0.70\}$ 中细筛， $\alpha = 0.62$ 的验证期费用最低；随后在留出期及波动电价下的日内调整分支中与 $\alpha = 0.50$ 对照。该检验用于评估验证期优势是否可延续，未替换正式方案的 $\alpha = 0.50$ 。

附录 A.4: 问题四补充比较结果

历史同星期均值预测与前一日同一时段延续预测的完整滚动验证结果见表 28。表中指标以 2025 年 2 月 1 日至 12 月 31 日共 $n = 48096$ 个 10 分钟电价点为样本，记实际电价与预测电价分别为 p_k 、 $\hat{p}_k$ ，则

$$\begin{aligned} \text{MAE} &= \frac{1}{n} \sum_{k=1}^n |p_k - \hat{p}_k|, & \text{RMSE} &= \sqrt{\frac{1}{n} \sum_{k=1}^n (p_k - \hat{p}_k)^2}, \\ \text{WAPE} &= \frac{\sum_{k=1}^n |p_k - \hat{p}_k|}{\sum_{k=1}^n p_k} \times 100\%. \end{aligned}$$

表 28: 两种电价预测方法的滚动验证结果

预测方法	MAE (元/kWh)	RMSE (元/kWh)	WAPE	平均偏差 (元/kWh)
前一日同一时段延续预测	0.0843	0.1238	11.13%	-0.00060
历史同星期均值预测	0.0489	0.0696	6.45%	-0.00003

日内电价再修正仅作为对照实验，正式方案仍采用 0:00 形成的预测电价。对更新时刻 $v \in \{6, 12, 18\}$ ，令 $m = 6v$ ，先由此前 28 个完整自然日内与目标日期星期相同的历史电价均值构造当天与次日的基准曲线，并拼接尚未到达的 24 h 基准向量 $\mathbf{b}_{i,v}$ 。利用当天已实现的前缀价格计算比例系数

$$\lambda_{i,v} = \frac{\sum_{t=1}^m p_{i,t}}{\sum_{t=1}^m \hat{p}_{i,t}}.$$

再以近 28 日残差 $e_{j,t} = p_{j,t} - \hat{p}_{j,t}$ 估计一阶自回归系数 $\rho_{i,v}$ ，并截断至 $[-0.99, 0.99]$ 。设 $r_{i,v} = p_{i,m} - \lambda_{i,v} \hat{p}_{i,m}$ ，则第 h 个后续时段的再修正预测为

$$\tilde{p}_{i,v,h} = \max \{0, \lambda_{i,v} b_{i,v,h} + r_{i,v} \rho_{i,v}^h\}, \quad h = 1, \dots, 144.$$

该式只使用更新时刻以前已实现的当天电价和历史残差；表 17 的“启用”即采用该方法。

表 29: 仅在 0:00 制定计划时的波动电价运行结果

指标	原策略仅按 实际电价重计费	按预测电价 重新优化	变化
正常购电量 (kWh)	21647262.3826	21581018.2339	-66244.1487
紧急购电量 (kWh)	123699.5020	124869.7518	1170.2497
正常购电费用 (元)	13978822.18	13927741.67	-51080.51
紧急购电费用 (元)	769519.96	759345.78	-10174.18
总购电费用 (元)	14748342.14	14687087.44	-61254.70

表 30: 原策略重新计价与主动重新优化的费用比较

情形	策略	正常购电 结算费用（元）	紧急购电 费用（元）	总购电 费用（元）
仅 0:00 制定计划	保持问题二原策略并重计价	13978822.18	769519.96	14748342.14
	按预测电价重新优化	13927741.67	759345.78	14687087.44
允许日内调整	保持问题三原策略并重计价	14198321.49	134339.97	14332661.46
	按预测电价重新优化	14152769.83	138553.49	14291323.32

附录 A.5：期末储电量价值校正结果

表 31: 期末储电量价值校正下的方案排序核验

比较情形	对照方案 期末储电 量 (kWh)	最终方案 期末储电 量	现金费用 节省	标准校正 后节省 (元)	保守校正 后节省	排序是 否变化
问题三最终方案 比较	8687.22	8531.12	120174.59	120066.94	119194.52	否
问题四日内调整 最终方案比较	8801.69	8666.32	129794.47	129702.18	128701.92	否

附录 A.6: 问题四日内调整的购电—储能联合运行情况

如图 17 所示, 主动重新优化后典型日内购电、储能充放电与储电量随预测电价相应调整。

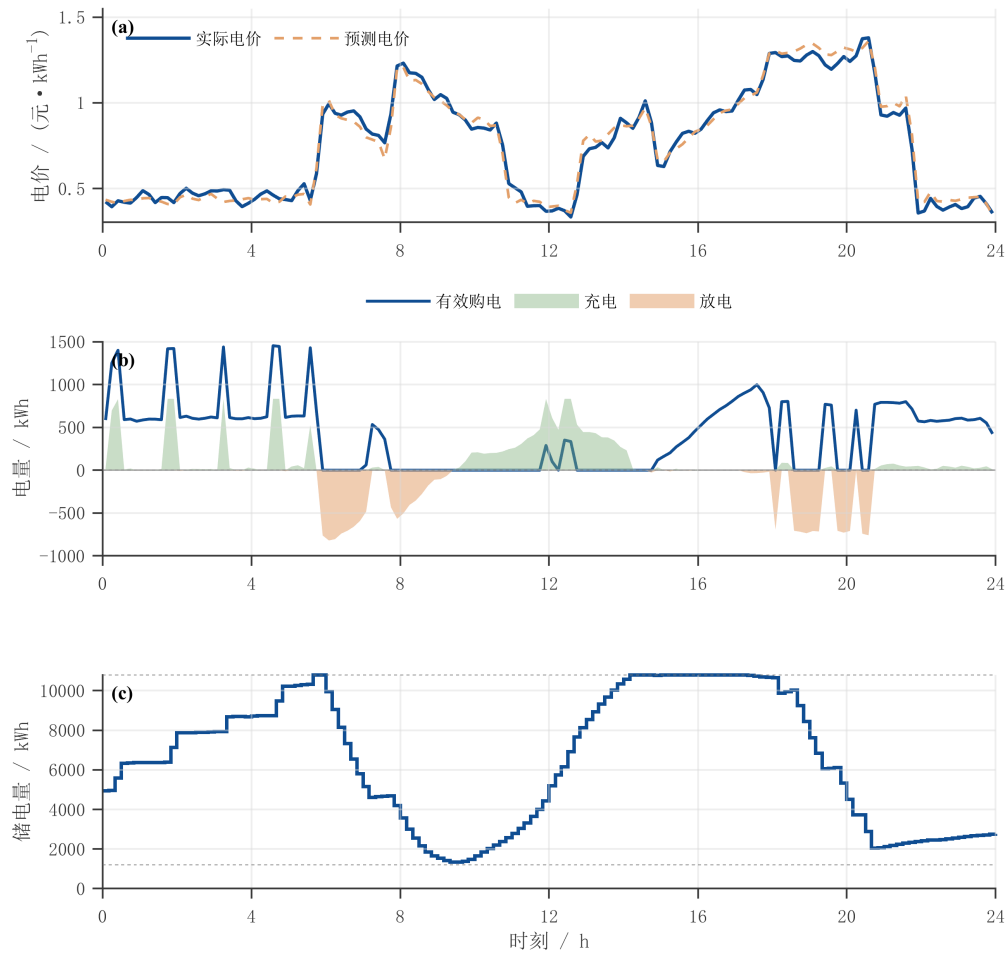


图 17: 波动电价下日内调整的购电—储能联合运行情况

附录 B：支撑材料文件目录

支撑材料按“完整计算结果、源程序、补充结果与图表”三类组织，当前分别包含 5、54 和 93 个文件，另含《AI 工具使用详情.pdf》，共 153 个纳入校验的支撑文件。表 32 以提交文件夹为根目录列出其层级与主要内容；逐文件的相对路径、文件大小和 SHA-256 校验值见根目录下的支撑材料清单 `_SHA256.csv`。

表 32: 支撑材料文件目录

序号	相对目录或文件	主要文件与内容
1	根目录	《AI 工具使用详情.pdf》；支撑材料清单 <code>_SHA256.csv</code> 。后者记录全部支撑文件的相对路径、大小和 SHA-256 校验值。
2	1、完整计算结果	<code>result1.xlsx</code> 、 <code>result2.xlsx</code> 、 <code>result3.xlsx</code> 、 <code>result4-2.xlsx</code> 、 <code>result4-3.xlsx</code> ，分别对应问题一至问题四的完整计算结果。
3	2、源程序	《README_程序运行说明.pdf》；下设“问题一”至“问题四”四个源程序目录。
4	2、源程序\问题一	<code>q1_solve.py</code> 为正式求解入口；同时包含公共数据读取、调度核心、结果工作簿生成和独立复算程序。
5	2、源程序\问题二	<code>q2_solve.py</code> 为正式求解入口；同时包含分量消融、候选稳健性、在线路线交叉核验、结果生成和独立校验程序。
6	2、源程序\问题三	<code>q3_solve.py</code> 为正式求解入口；同时包含负载反馈、信息模块与更新时间组合消融、反馈系数标定、结算敏感性、结果生成和独立校验程序。
7	2、源程序\问题四	<code>q4_solve.py</code> 为正式求解入口；同时包含波动电价迁移、负载反馈与电价再修正对照、期末储电量检验、结果生成和独立校验程序。
8	3、补充结果与图表	《图表说明与正文解读建议.md》；下设“其他较大篇幅中间结果”“求解流程图”“问题二补充结果”“问题三消融与敏感性结果”“问题四补充比较结果”五个目录。
9	3、补充结果与图表\其他较大篇幅中间结果	“验证记录”目录，包含四问独立复算、模型结果核验及五个结果工作簿核验的 JSON 文件。
10	3、补充结果与图表\求解流程图	总体流程图的 PNG、PPTX 文件，以及问题一至问题四求解流程图 PNG 和四张流程图的可编辑 PPTX 文件。
11	3、补充结果与图表\问题二补充结果	残差分位数敏感性、候选方案稳健性、分量消融、在线路线交叉核验结果，以及论文配图和 MATLAB 绘图脚本。
12	3、补充结果与图表\问题三消融与敏感性结果	负载反馈候选、反馈系数标定与五点敏感性、信息模块及更新时间组合消融结果，以及论文配图和 MATLAB 绘图脚本。
13	3、补充结果与图表\问题四补充比较结果	波动电价下重计价与重新优化、负载反馈与电价再修正、期末储电量检验等结果，以及论文配图和 MATLAB 绘图脚本。

附录 C：源程序代码

本附录仅保留问题一至问题四的正式求解入口，便于对应正文模型逐问查看。各入口依赖的公共数据读取、线性规划调度核心、结果工作簿生成、对比实验、独立复算，以及正文图像和绘图数据，均随完整源代码和图表数据一并提交至支撑材料，不再在论文中重复列示。

附录 C-1：问题一求解程序

该程序实现确定性购电—储能词典序线性规划，依次求解最低购电费用和最小储能吞吐量，并输出购电计划、储能运行结果及结果工作簿载荷。

问题一确定性优化

源文件：code/q1_solve.py

```
1  from __future__ import annotations
2
3  import csv
4  import json
5  import math
6  import platform
7  import sys
8  from dataclasses import dataclass
9  from pathlib import Path
10 from typing import Any
11
12 import numpy as np
13 import scipy
14 from scipy.optimize import linprog
15 from scipy.sparse import coo_matrix, vstack
16
17 from common_data import PROJECT_ROOT, load_attachment1, natural_slot_label, sha256
18
19
20 RESULTS = PROJECT_ROOT / "results"
21 VALIDATION = PROJECT_ROOT / "validation"
22 INTERMEDIATE = PROJECT_ROOT / "intermediate"
23
24 N = 144
25 DELTA_H = 1.0 / 6.0
26 SOC_MIN = 1200.0
27 SOC_MAX = 10800.0
28 SOC_INITIAL = 6000.0
29 SOC_TERMINAL = 6000.0
30 POWER_MAX_KW = 5000.0
31 FLOW_MAX_KWH = POWER_MAX_KW * DELTA_H
32 SOLVER_TOL_KWH = 1e-6
33 POSITIVE_TOL_KWH = 1e-7
34
35
36 @dataclass
37 class Solution:
38     scenario: str
39     eta_charge: float
40     eta_discharge: float
41     integration: str
42     q: np.ndarray
43     charge: np.ndarray
44     discharge: np.ndarray
45     spill: np.ndarray
46     soc: np.ndarray
47     load_kwh: np.ndarray
48     pv_kwh: np.ndarray
49     primary_optimum: float
50     actual_cost: float
51     solver_status_primary: int
52     solver_message_primary: str
53     solver_status_secondary: int
54     solver_message_secondary: str
```

```

55
56
57 def energy_from_power(power_kw: np.ndarray, method: str) -> np.ndarray:
58     if method == "right_endpoint_rectangle":
59         return power_kw * DELTA_H
60     if method == "trapezoidal_repeated_day":
61         previous = np.roll(power_kw, 1)
62         return 0.5 * (previous + power_kw) * DELTA_H
63     raise ValueError(f"未知积分口径: {method}")
64
65
66 def variable_slices() -> dict[str, slice]:
67     return {
68         "q": slice(0, N),
69         "charge": slice(N, 2 * N),
70         "discharge": slice(2 * N, 3 * N),
71         "spill": slice(3 * N, 4 * N),
72         "soc": slice(4 * N, 5 * N + 1),
73     }
74
75
76 def build_equalities(load_kwh: np.ndarray, pv_kwh: np.ndarray, eta_c: float, eta_d:
77     float):
78     idx = variable_slices()
79     rows: list[int] = []
80     cols: list[int] = []
81     data: list[float] = []
82     rhs = np.zeros(2 * N, dtype=float)
83     for t in range(N):
84         # q + PV + discharge = load + charge + spill
85         row = t
86         for col, value in (
87             (idx["q"].start + t, 1.0),
88             (idx["charge"].start + t, -1.0),
89             (idx["discharge"].start + t, 1.0),
90             (idx["spill"].start + t, -1.0),
91         ):
92             rows.append(row)
93             cols.append(col)
94             data.append(value)
95         rhs[row] = load_kwh[t] - pv_kwh[t]
96
97         # SOC[t+1] = SOC[t] + eta_c*charge - discharge/eta_d
98         row = N + t
99         for col, value in (
100             (idx["charge"].start + t, -eta_c),
101             (idx["discharge"].start + t, 1.0 / eta_d),
102             (idx["soc"].start + t, -1.0),
103             (idx["soc"].start + t + 1, 1.0),
104         ):
105             rows.append(row)
106             cols.append(col)
107             data.append(value)
108     matrix = coo_matrix((data, (rows, cols)), shape=(2 * N, 5 * N + 1)).tocsr()
109     return matrix, rhs
110
111 def solve_lp(
112     price: np.ndarray,
113     load_kw: np.ndarray,
114     pv_kw: np.ndarray,
115     *,
116     scenario: str,
117     eta_c: float,
118     eta_d: float,
119     integration: str,
120 ) -> Solution:
121     load_kwh = energy_from_power(load_kw, integration)
122     pv_kwh = energy_from_power(pv_kw, integration)
123     idx = variable_slices()
124     a_eq, b_eq = build_equalities(load_kwh, pv_kwh, eta_c, eta_d)
125
126     objective = np.zeros(5 * N + 1, dtype=float)
127     objective[idx["q"]] = price
128     bounds: list[tuple[float | None, float | None]] = []

```

```

129     bounds.extend([(0.0, None)] * N)
130     bounds.extend([(0.0, FLOW_MAX_KWH)] * N)
131     bounds.extend([(0.0, FLOW_MAX_KWH)] * N)
132     bounds.extend([(0.0, None)] * N)
133     bounds.append((SOC_INITIAL, SOC_INITIAL))
134     bounds.extend([(SOC_MIN, SOC_MAX)] * (N - 1))
135     bounds.append((SOC_TERMINAL, SOC_TERMINAL))
136
137     primary = linprog(objective, A_eq=a_eq, b_eq=b_eq, bounds=bounds, method="highs")
138     if not primary.success:
139         raise RuntimeError(f"Q1主LP失败: {primary.status} {primary.message}")
140
141     # Lexicographic tie-break: keep the purchase optimum, then minimize battery
142     ↪ throughput.
143     # This does not replace the primary cash-cost objective and removes degenerate
144     ↪ cycling.
145     secondary_objective = np.zeros_like(objective)
146     secondary_objective[idx["charge"]] = 1.0
147     secondary_objective[idx["discharge"]] = 1.0
148     cost_tolerance = max(1e-7, abs(float(primary.fun)) * 1e-11)
149     a_ub = coo_matrix(objective.reshape(1, -1)).tocsr()
150     b_ub = np.asarray([float(primary.fun) + cost_tolerance])
151     secondary = linprog(
152         secondary_objective,
153         A_ub=a_ub,
154         b_ub=b_ub,
155         A_eq=a_eq,
156         b_eq=b_eq,
157         bounds=bounds,
158         method="highs",
159     )
160     if not secondary.success:
161         raise RuntimeError(f"Q1二级LP失败: {secondary.status} {secondary.message}")
162
163     x = np.asarray(secondary.x, dtype=float)
164     x[np.abs(x) < 1e-10] = 0.0
165     return Solution(
166         scenario=scenario,
167         eta_charge=eta_c,
168         eta_discharge=eta_d,
169         integration=integration,
170         q=x[idx["q"]],
171         charge=x[idx["charge"]],
172         discharge=x[idx["discharge"]],
173         spill=x[idx["spill"]],
174         soc=x[idx["soc"]],
175         load_kwh=load_kwh,
176         pv_kwh=pv_kwh,
177         primary_optimum=float(primary.fun),
178         actual_cost=float(np.dot(price, x[idx["q"]])),
179         solver_status_primary=int(primary.status),
180         solver_message_primary=str(primary.message),
181         solver_status_secondary=int(secondary.status),
182         solver_message_secondary=str(secondary.message),
183     )
184
185 def validate(solution: Solution, price: np.ndarray) -> dict[str, Any]:
186     balance = solution.q + solution.pv_kwh + solution.discharge - solution.load_kwh -
187     ↪ solution.charge - solution.spill
188     state = solution.soc[1:] - solution.soc[:-1] - solution.eta_charge *
189     ↪ solution.charge + solution.discharge / solution.eta_discharge
190     simultaneous = np.minimum(solution.charge, solution.discharge)
191     independent_cost = float(np.sum(price * solution.q))
192     checks = {
193         "balance_max_abs_kwh": float(np.max(np.abs(balance))),
194         "state_max_abs_kwh": float(np.max(np.abs(state))),
195         "soc_min_kwh": float(np.min(solution.soc)),
196         "soc_max_kwh": float(np.max(solution.soc)),
197         "soc_open_kwh": float(solution.soc[0]),
198         "soc_close_kwh": float(solution.soc[-1]),
199         "charge_max_kwh_per_slot": float(np.max(solution.charge)),
200         "discharge_max_kwh_per_slot": float(np.max(solution.discharge)),
201         "simultaneous_positive_slot_count": int(np.sum(simultaneous >
202     ↪ POSITIVE_TOL_KWH)),

```

```

199     "simultaneous_max_kwh": float(np.max(simultaneous)),
200     "independent_cost_yuan": independent_cost,
201     "reported_cost_yuan": solution.actual_cost,
202     "cost_reconciliation_abs_yuan": float(abs(independent_cost -
    ↪ solution.actual_cost)),
203     "secondary_cost_above_primary_yuan": float(solution.actual_cost -
    ↪ solution.primary_optimum),
204 }
205 pass_flags = {
206     "balance": checks["balance_max_abs_kwh"] <= SOLVER_TOL_KWH,
207     "state": checks["state_max_abs_kwh"] <= SOLVER_TOL_KWH,
208     "soc_bounds": checks["soc_min_kwh"] >= SOC_MIN - SOLVER_TOL_KWH and
    ↪ checks["soc_max_kwh"] <= SOC_MAX + SOLVER_TOL_KWH,
209     "soc_cycle": abs(checks["soc_open_kwh"] - SOC_INITIAL) <= SOLVER_TOL_KWH and
    ↪ abs(checks["soc_close_kwh"] - SOC_TERMINAL) <= SOLVER_TOL_KWH,
210     "power": checks["charge_max_kwh_per_slot"] <= FLOW_MAX_KWH + SOLVER_TOL_KWH and
    ↪ checks["discharge_max_kwh_per_slot"] <= FLOW_MAX_KWH + SOLVER_TOL_KWH,
211     "nonnegative": bool(np.min(np.concatenate([solution.q, solution.charge,
    ↪ solution.discharge, solution.spill])) >= -SOLVER_TOL_KWH),
212     "mutual_exclusion": checks["simultaneous_positive_slot_count"] == 0,
213     "cost_reconciliation": checks["cost_reconciliation_abs_yuan"] <= 1e-6,
214 }
215 return {"metrics": checks, "pass_flags": pass_flags, "all_pass":
    ↪ all(pass_flags.values())}
216
217
218 def baseline(load_kwh: np.ndarray, pv_kwh: np.ndarray, price: np.ndarray) -> dict[str,
    ↪ float]:
219     q = np.maximum(load_kwh - pv_kwh, 0.0)
220     spill = np.maximum(pv_kwh - load_kwh, 0.0)
221     return {
222         "purchase_kwh": float(np.sum(q)),
223         "cost_yuan": float(np.sum(price * q)),
224         "pv_spill_kwh": float(np.sum(spill)),
225     }
226
227
228 def write_detail(solution: Solution, data: dict[str, Any], price: np.ndarray) -> Path:
229     path = RESULTS / "q1_detail.csv"
230     fields = [
231         "slot", "source_endpoint_label", "interval_start", "interval_end",
    ↪ "interval_label",
232         "price_yuan_per_kwh", "load_kw", "pv_kw", "load_kwh", "pv_kwh",
233         "normal_purchase_kwh", "charge_bus_kwh", "discharge_bus_kwh",
    ↪ "pv_spill_or_surplus_kwh",
234         "soc_open_kwh", "soc_close_kwh", "purchase_cost_yuan", "balance_residual_kwh",
    ↪ "state_residual_kwh",
235     ]
236     with path.open("w", encoding="utf-8-sig", newline="") as stream:
237         writer = csv.DictWriter(stream, fieldnames=fields)
238         writer.writeheader()
239         for t in range(N):
240             start, end, label = natural_slot_label(t)
241             balance = solution.q[t] + solution.pv_kwh[t] + solution.discharge[t] -
    ↪ solution.load_kwh[t] - solution.charge[t] - solution.spill[t]
242             state = solution.soc[t + 1] - solution.soc[t] - solution.eta_charge *
    ↪ solution.charge[t] + solution.discharge[t] / solution.eta_discharge
243             writer.writerow({
244                 "slot": t + 1,
245                 "source_endpoint_label": data["raw_endpoint_labels"][t],
246                 "interval_start": start,
247                 "interval_end": end,
248                 "interval_label": label,
249                 "price_yuan_per_kwh": f"{price[t]:.10f}",
250                 "load_kw": f"{data['load_kw'][t]:.10f}",
251                 "pv_kw": f"{data['pv_kw'][t]:.10f}",
252                 "load_kwh": f"{solution.load_kwh[t]:.10f}",
253                 "pv_kwh": f"{solution.pv_kwh[t]:.10f}",
254                 "normal_purchase_kwh": f"{solution.q[t]:.10f}",
255                 "charge_bus_kwh": f"{solution.charge[t]:.10f}",
256                 "discharge_bus_kwh": f"{solution.discharge[t]:.10f}",
257                 "pv_spill_or_surplus_kwh": f"{solution.spill[t]:.10f}",
258                 "soc_open_kwh": f"{solution.soc[t]:.10f}",
259                 "soc_close_kwh": f"{solution.soc[t + 1]:.10f}",
260                 "purchase_cost_yuan": f"{price[t] * solution.q[t]:.10f}",

```

```

261         "balance_residual_kwh": f"{balance:.12g}",
262         "state_residual_kwh": f"{state:.12g}",
263     })
264     return path
265
266
267 def four_hour_summary(solution: Solution) -> list[dict[str, float | str]]:
268     rows: list[dict[str, float | str]] = []
269     for block in range(6):
270         start = block * 24
271         stop = (block + 1) * 24
272         rows.append({
273             "interval": f"{block * 4:02d}:00-{(block + 1) * 4:02d}:00",
274             "charge_kwh": float(np.sum(solution.charge[start:stop])),
275             "discharge_kwh": float(np.sum(solution.discharge[start:stop])),
276         })
277     return rows
278
279
280 def write_four_hour(rows: list[dict[str, float | str]]) -> Path:
281     path = RESULTS / "q1_storage_4h.csv"
282     with path.open("w", encoding="utf-8-sig", newline="") as stream:
283         writer = csv.DictWriter(stream, fieldnames=["interval", "charge_kwh",
284             ↪ "discharge_kwh"])
285         writer.writeheader()
286         for row in rows:
287             writer.writerow({"interval": row["interval"], "charge_kwh":
288                 ↪ f"{float(row['charge_kwh']):.10f}", "discharge_kwh":
289                 ↪ f"{float(row['discharge_kwh']):.10f}"})
290     return path
291
292
293 def json_default(value: Any):
294     if isinstance(value, np.generic):
295         return value.item()
296     raise TypeError(type(value).__name__)
297
298
299 def main() -> None:
300     RESULTS.mkdir(parents=True, exist_ok=True)
301     VALIDATION.mkdir(parents=True, exist_ok=True)
302     INTERMEDIATE.mkdir(parents=True, exist_ok=True)
303     data = load_attachment1()
304     price = data["price_yuan_per_kwh"]
305
306     scenarios = [
307         ("base", 0.9, 0.9, "right_endpoint_rectangle"),
308         ("efficiency_round_trip_0.9_symmetric", math.sqrt(0.9), math.sqrt(0.9),
309             ↪ "right_endpoint_rectangle"),
310         ("trapezoidal_power_integration", 0.9, 0.9, "trapezoidal_repeated_day"),
311     ]
312     solved: list[Solution] = []
313     validations: dict[str, Any] = {}
314     for scenario, eta_c, eta_d, integration in scenarios:
315         solution = solve_lp(price, data["load_kw"], data["pv_kw"], scenario=scenario,
316             ↪ eta_c=eta_c, eta_d=eta_d, integration=integration)
317         solved.append(solution)
318         validations[scenario] = validate(solution, price)
319
320     base = solved[0]
321     base_baseline = baseline(base.load_kwh, base.pv_kwh, price)
322     detail_path = write_detail(base, data, price)
323     storage_rows = four_hour_summary(base)
324     storage_path = write_four_hour(storage_rows)
325     workbook_payload_path = INTERMEDIATE / "q1_workbook_payload.json"
326     workbook_payload_path.write_text(
327         json.dumps(
328             {
329                 "schema_version": 1,
330                 "question": "Q1",
331                 "time_label_mapping": "natural_day_00:00_to_24:00_corrected_from_shift",
332                 ↪ "ed_official_template",
333                 "purchase_plan": [
334                     {
335                         "slot": slot,

```

```

330         "interval": natural_slot_label(slot)[2],
331         "purchase_kwh": float(base.q[slot]),
332     }
333     for slot in range(len(base.q))
334 ],
335     "storage_four_hour_summary": storage_rows,
336     "soc_endpoints_kwh": {"0:00": float(base.soc[0]), "24:00":
        ↳ float(base.soc[-1])},
337 },
338     ensure_ascii=False,
339     indent=2,
340 ),
341     encoding="utf-8",
342 )
343
344 requested_starts = [10 * 60, 12 * 60, 14 * 60, 16 * 60, 18 * 60, 20 * 60]
345 specified = {}
346 for minute in requested_starts:
347     slot = minute // 10
348     specified[natural_slot_label(slot)[2]] = float(base.q[slot])
349
350 sensitivity = []
351 for solution in solved:
352     sensitivity.append({
353         "scenario": solution.scenario,
354         "eta_charge": solution.eta_charge,
355         "eta_discharge": solution.eta_discharge,
356         "integration": solution.integration,
357         "purchase_kwh": float(np.sum(solution.q)),
358         "cost_yuan": solution.actual_cost,
359         "charge_kwh": float(np.sum(solution.charge)),
360         "discharge_kwh": float(np.sum(solution.discharge)),
361         "pv_spill_or_surplus_kwh": float(np.sum(solution.spill)),
362         "cost_change_vs_base_yuan": float(solution.actual_cost - base.actual_cost),
363         "cost_change_vs_base_fraction": float(solution.actual_cost /
        ↳ base.actual_cost - 1.0),
364         "all_checks_pass": validations[solution.scenario]["all_pass"],
365     })
366
367 summary = {
368     "schema_version": 1,
369     "status": "SOLVED_AND_VALIDATED" if validations["base"]["all_pass"] else
        ↳ "FAILED_VALIDATION",
370     "question": "Q1",
371     "frozen_basis": {
372         "time": "natural_day_144_slots_right_endpoint_rectangle",
373         "efficiency": {"eta_charge": 0.9, "eta_discharge": 0.9},
374         "measurement_side": "microgrid_bus_side",
375         "soc_kwh": {"initial": SOC_INITIAL, "terminal": SOC_TERMINAL, "min":
        ↳ SOC_MIN, "max": SOC_MAX},
376         "flow_max_kwh_per_slot": FLOW_MAX_KWH,
377         "known_conflict": "A01 template labels are shifted by ten minutes; the
        ↳ generated result workbook uses corrected natural-day labels and a
        ↳ visible disclosure, without claiming an official erratum.",
378     },
379     "input": {"path": str(data["path"].relative_to(PROJECT_ROOT)), "sha256":
        ↳ data["sha256"]},
380     "baseline_no_storage_pv_first": base_baseline,
381     "optimized": {
382         "purchase_kwh": float(np.sum(base.q)),
383         "cost_yuan": base.actual_cost,
384         "cost_saving_vs_baseline_yuan": float(base_baseline["cost_yuan"] -
        ↳ base.actual_cost),
385         "cost_saving_vs_baseline_fraction": float(1.0 - base.actual_cost /
        ↳ base_baseline["cost_yuan"]),
386         "charge_kwh": float(np.sum(base.charge)),
387         "discharge_kwh": float(np.sum(base.discharge)),
388         "pv_spill_or_surplus_kwh": float(np.sum(base.spill)),
389         "soc_open_kwh": float(base.soc[0]),
390         "soc_close_kwh": float(base.soc[-1]),
391         "specified_interval_purchase_kwh": specified,
392         "storage_four_hour_summary": storage_rows,
393     },
394     "sensitivity": sensitivity,
395     "model_upgrade_decision": {

```

```

396         "milp_required": not validations["base"]["pass_flags"]["mutual_exclusion"],
397         "reason": "The lexicographic LP produced no simultaneous charge/discharge
↪ above the frozen tolerance; MILP is unnecessary." if
↪ validations["base"]["pass_flags"]["mutual_exclusion"] else
↪ "Simultaneous flow exceeded tolerance; upgrade to MILP before accepting
↪ Q1.",
398     },
399     "files": {
400         "detail_csv": str(detail_path.relative_to(PROJECT_ROOT)),
401         "storage_4h_csv": str(storage_path.relative_to(PROJECT_ROOT)),
402         "workbook_payload_json":
↪ str(workbook_payload_path.relative_to(PROJECT_ROOT)),
403     },
404 }
405
406 validation_payload = {
407     "schema_version": 1,
408     "question": "Q1",
409     "status": "PASS" if all(item["all_pass"] for item in validations.values()) else
↪ "FAIL",
410     "tolerances": {"solver_balance_state_kwh": SOLVER_TOL_KWH, "positive_flow_kwh":
↪ POSITIVE_TOL_KWH},
411     "scenarios": validations,
412     "independent_recalculation": {
413         "detail_rows_expected": 144,
414         "detail_rows_written": 144,
415         "daily_purchase_from_detail_kwh": float(np.sum(base.q)),
416         "daily_cost_from_detail_yuan": float(np.sum(price * base.q)),
417         "four_hour_charge_sum_kwh": float(sum(float(row["charge_kwh"]) for row in
↪ storage_rows)),
418         "four_hour_discharge_sum_kwh": float(sum(float(row["discharge_kwh"]) for
↪ row in storage_rows)),
419     },
420     "environment": {
421         "python": sys.version,
422         "python_executable": sys.executable,
423         "platform": platform.platform(),
424         "numpy": np.__version__,
425         "scipy": scipy.__version__,
426         "solver": "scipy.optimize.linprog(method='highs')",
427     },
428 }
429
430 summary_path = RESULTS / "q1_summary.json"
431 summary_path.write_text(json.dumps(summary, ensure_ascii=False, indent=2,
↪ default=json_default), encoding="utf-8")
432 validation_path = VALIDATION / "q1_validation.json"
433 validation_path.write_text(json.dumps(validation_payload, ensure_ascii=False,
↪ indent=2, default=json_default), encoding="utf-8")
434
435 run_manifest = {
436     "question": "Q1",
437     "status": summary["status"],
438     "input_sha256": data["sha256"],
439     "code_sha256": {
440         "common_data.py": sha256(Path(__file__).with_name("common_data.py")),
441         "q1_solve.py": sha256(Path(__file__)),
442     },
443     "outputs_sha256": {
444         str(path.relative_to(PROJECT_ROOT)): sha256(path)
445         for path in (detail_path, storage_path, workbook_payload_path,
↪ summary_path, validation_path)
446     },
447 }
448 manifest_path = INTERMEDIATE / "q1_run_manifest.json"
449 manifest_path.write_text(json.dumps(run_manifest, ensure_ascii=False, indent=2),
↪ encoding="utf-8")
450 print(json.dumps({
451     "status": summary["status"],
452     "baseline_cost_yuan": base_baseline["cost_yuan"],
453     "optimized_cost_yuan": base.actual_cost,
454     "saving_yuan": base_baseline["cost_yuan"] - base.actual_cost,
455     "validation": validation_payload["status"],
456     "milp_required": summary["model_upgrade_decision"]["milp_required"],
457 }, ensure_ascii=False))

```

```

458
459
460 if __name__ == "__main__":
461     main()

```

附录 C-2: 问题二求解程序

该程序实现历史同星期负载预测、近五个完整自然日光伏预测、分时段残差分位数修正和跨日连续储能仿真，生成问题二统计期运行结果。

问题二风险修正滚动优化

源文件: code/q2_solve.py

```

1  from __future__ import annotations
2
3  import csv
4  import json
5  import platform
6  import sys
7  from dataclasses import asdict
8  from datetime import date, timedelta
9  from pathlib import Path
10 from typing import Any
11
12 import numpy as np
13 import scipy
14
15 from common_data import PROJECT_ROOT, load_attachment1, load_attachment2,
16 ↪ natural_slot_label, sha256
17 from dispatch_core import (
18     ETA_CHARGE,
19     ETA_DISCHARGE,
20     FLOW_MAX_KWH,
21     POSITIVE_TOL_KWH,
22     SOC_INITIAL,
23     SOC_MAX,
24     SOC_MIN,
25     SOLVER_TOL_KWH,
26     execute_fixed_plan,
27     maximal_positive_events,
28     solve_deterministic_plan,
29 )
30
31 RESULTS = PROJECT_ROOT / "results"
32 VALIDATION = PROJECT_ROOT / "validation"
33 INTERMEDIATE = PROJECT_ROOT / "intermediate"
34 N = 144
35 EVAL_START = 31 # 2025-02-01; January is historical warm-up with idle storage.
36 LOOKBACK_DAYS = 28
37 RESIDUAL_WINDOW_DAYS = 56
38 RISK_QUANTILE = 0.8
39 EMERGENCY_MULTIPLIER = 5.0
40
41
42 def json_default(value: Any):
43     if isinstance(value, (date,)):
44         return value.isoformat()
45     if isinstance(value, np.generic):
46         return value.item()
47     raise TypeError(type(value).__name__)
48
49
50 def weekday_mean_forecast(
51     values_kw: np.ndarray,
52     dates: list[date],
53     cutoff_idx: int,
54     target_date: date,
55     lookback_days: int = LOOKBACK_DAYS,
56 ) -> np.ndarray:

```

```

57     known_start = max(0, cutoff_idx - lookback_days)
58     candidates = [
59         idx
60         for idx in range(known_start, cutoff_idx)
61         if dates[idx].weekday() == target_date.weekday()
62     ]
63     if not candidates:
64         if cutoff_idx <= 0:
65             raise ValueError("没有任何历史数据可生成预测")
66         candidates = [cutoff_idx - 1]
67     return np.mean(values_kw[candidates], axis=0)
68
69
70 def make_historical_weekday_cache(values_kw: np.ndarray, dates: list[date]) ->
    ↪ np.ndarray:
71     cache = np.full_like(values_kw, np.nan, dtype=float)
72     for idx in range(1, len(dates)):
73         cache[idx] = weekday_mean_forecast(values_kw, dates, idx, dates[idx])
74     return cache
75
76
77 def weekday_weighted_forecast(
78     values_kw: np.ndarray,
79     dates: list[date],
80     cutoff_idx: int,
81     target_date: date,
82     weight_kind: str,
83 ) -> np.ndarray:
84     candidates = [
85         idx for idx in range(max(0, cutoff_idx - LOOKBACK_DAYS), cutoff_idx)
86         if dates[idx].weekday() == target_date.weekday()
87     ]
88     if not candidates:
89         if cutoff_idx <= 0:
90             raise ValueError("没有任何历史数据可生成同星期预测")
91         candidates = [cutoff_idx - 1]
92     if weight_kind == "equal":
93         weights = np.ones(len(candidates), dtype=float)
94     elif weight_kind == "linear":
95         weights = np.arange(1, len(candidates) + 1, dtype=float)
96     elif weight_kind == "exp2w":
97         ages_weeks = (cutoff_idx - 1 - np.asarray(candidates, dtype=float)) / 7.0
98         weights = np.power(0.5, ages_weeks / 2.0)
99     else:
100         raise ValueError(f"未知同星期权重: {weight_kind}")
101     return np.average(values_kw[candidates], axis=0, weights=weights)
102
103
104 def candidate_kind(weight_kind: str, pv_window_days: int) -> str:
105     if weight_kind not in {"equal", "linear", "exp2w"} or pv_window_days not in {3, 5}:
106         raise ValueError("候选只允许equal/linear/exp2w与PV3/5日")
107     return f"candidate_{weight_kind}_pv{pv_window_days}"
108
109
110 def make_point_forecast_cache(
111     kind: str, load_kw: np.ndarray, pv_kw: np.ndarray, dates: list[date]
112 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
113     load_cache = np.full_like(load_kw, np.nan, dtype=float)
114     pv_cache = np.full_like(pv_kw, np.nan, dtype=float)
115     for idx in range(1, len(dates)):
116         forecast_load, forecast_pv = forecast_two_days(kind, load_kw, pv_kw, dates,
117             ↪ idx)
118         load_cache[idx] = forecast_load[0]
119         pv_cache[idx] = forecast_pv[0]
120     return load_cache, pv_cache, load_cache - pv_cache
121
122
123 def residual_quantile_kw(
124     actual_net_kw: np.ndarray,
125     weekday_net_cache_kw: np.ndarray,
126     decision_idx: int,
127     quantile: float,
128 ) -> tuple[np.ndarray, int, int]:
129     start = max(1, decision_idx - RESIDUAL_WINDOW_DAYS)
130     historical_indices = np.arange(start, decision_idx, dtype=int)

```

```

130     valid = historical_indices[np.isfinite(weekday_net_cache_kw[historical_indices]).a]
131     ↪ ll(axis=1)]
132     if len(valid) == 0:
133         raise ValueError(f"{decision_idx=} 前无可用历史残差")
134     residuals = actual_net_kw[valid] - weekday_net_cache_kw[valid]
135     return np.quantile(residuals, quantile, axis=0), int(valid[0]), int(valid[-1])
136
137 def forecast_two_days(
138     kind: str,
139     load_kw: np.ndarray,
140     pv_kw: np.ndarray,
141     dates: list[date],
142     decision_idx: int,
143 ) -> tuple[np.ndarray, np.ndarray]:
144     if kind == "persistence":
145         if decision_idx <= 0:
146             raise ValueError("持久性预测至少需要一天历史")
147         return (
148             np.vstack([load_kw[decision_idx - 1], load_kw[decision_idx - 1]]),
149             np.vstack([pv_kw[decision_idx - 1], pv_kw[decision_idx - 1]]),
150         )
151     if kind == "weekday_mean_28d":
152         target_dates = [dates[decision_idx], dates[decision_idx] + timedelta(days=1)]
153         return (
154             np.vstack([weekday_mean_forecast(load_kw, dates, decision_idx, target) for
155                 ↪ target in target_dates]),
156             np.vstack([weekday_mean_forecast(pv_kw, dates, decision_idx, target) for
157                 ↪ target in target_dates]),
158         )
159     if kind == "weighted_weekday_load_pv3":
160         kind = "candidate_linear_pv3"
161     if kind.startswith("candidate_"):
162         parts = kind.split("_")
163         if len(parts) != 3 or not parts[2].startswith("pv"):
164             raise ValueError(f"候选预测名称格式错误: {kind}")
165         weight_kind = parts[1]
166         pv_window = int(parts[2][2:])
167         candidate_kind(weight_kind, pv_window)
168         target_dates = [dates[decision_idx], dates[decision_idx] + timedelta(days=1)]
169         load_forecast = np.vstack([
170             weekday_weighted_forecast(load_kw, dates, decision_idx, target,
171                 ↪ weight_kind)
172             for target in target_dates
173         ])
174         pv_start = max(0, decision_idx - pv_window)
175         pv_candidates = np.arange(pv_start, decision_idx, dtype=int)
176         if len(pv_candidates) == 0:
177             raise ValueError("滚动光伏均值至少需要一个已完成日")
178         pv_average = np.mean(pv_kw[pv_candidates], axis=0)
179         return load_forecast, np.vstack([pv_average, pv_average])
180     raise ValueError(f"未知预测类型: {kind}")
181
182 def forecast_metrics(actual: np.ndarray, forecast: np.ndarray) -> dict[str, float]:
183     error = forecast - actual
184     denominator = float(np.sum(np.abs(actual)))
185     return {
186         "mae_kw": float(np.mean(np.abs(error))),
187         "rmse_kw": float(np.sqrt(np.mean(error**2))),
188         "wape": float(np.sum(np.abs(error)) / denominator) if denominator > 0 else 0.0,
189         "bias_kw": float(np.mean(error)),
190     }
191
192 def run_strategy(
193     *,
194     name: str,
195     forecast_kind: str,
196     quantile: float | None,
197     terminal_mode: str,
198     load_kw: np.ndarray,
199     pv_kw: np.ndarray,
200     dates: list[date],
201     price: np.ndarray,

```

```

201     actual_net_kw: np.ndarray,
202     weekday_net_cache_kw: np.ndarray,
203     raw_endpoint_labels: list[str],
204     keep_detail: bool,
205     actual_price_by_date: np.ndarray | None = None,
206     planning_price_kind: str | None = None,
207 ) -> dict[str, Any]:
208     soc = SOC_INITIAL
209     daily_rows: list[dict[str, Any]] = []
210     detail_rows: list[dict[str, Any]] = []
211     storage_rows: list[dict[str, Any]] = []
212     event_rows: list[dict[str, Any]] = []
213     plan_rows: list[dict[str, Any]] = []
214     max_plan_balance = 0.0
215     max_plan_state = 0.0
216     plan_simultaneous = 0
217     secondary_days = 0
218     max_actual_balance = 0.0
219     max_actual_state = 0.0
220     min_soc = soc
221     max_soc = soc
222     max_crossday_gap = 0.0
223     previous_close: float | None = None
224
225     for day_idx in range(EVAL_START, len(dates)):
226         day = dates[day_idx]
227         train_end = dates[day_idx - 1]
228         if actual_price_by_date is None:
229             plan_price_2d = np.tile(price, 2)
230             settlement_price = price
231         else:
232             if planning_price_kind == "weekday_mean_28d":
233                 plan_price_2d = np.concatenate([
234                     weekday_mean_forecast(actual_price_by_date, dates, day_idx, day),
235                     weekday_mean_forecast(actual_price_by_date, dates, day_idx, day +
236                                             ↳ timedelta(days=1)),
237                 ])
238             elif planning_price_kind == "persistence":
239                 plan_price_2d = np.tile(actual_price_by_date[day_idx - 1], 2)
240             else:
241                 raise ValueError("实时价格模式必须明确 planning_price_kind")
242             settlement_price = actual_price_by_date[day_idx]
243             forecast_load_2d, forecast_pv_2d = forecast_two_days(
244                 forecast_kind, load_kw, pv_kw, dates, day_idx
245             )
246             point_net_2d_kw = forecast_load_2d - forecast_pv_2d
247             risk_uplift_kw = np.zeros(N, dtype=float)
248             residual_first_idx = day_idx - 1
249             residual_last_idx = day_idx - 1
250             if quantile is not None:
251                 risk_uplift_kw, residual_first_idx, residual_last_idx =
252                 ↳ residual_quantile_kw(
253                     actual_net_kw, weekday_net_cache_kw, day_idx, quantile
254                 )
255             effective_net_2d_kwh = (point_net_2d_kw + np.vstack([risk_uplift_kw,
256                 ↳ risk_uplift_kw])) / 6.0
257             terminal_soc = SOC_INITIAL if terminal_mode == "fixed_6000_at_48h" else soc
258             plan = solve_deterministic_plan(
259                 effective_net_2d_kwh.reshape(-1),
260                 plan_price_2d,
261                 soc,
262                 terminal_soc,
263             )
264             q = plan.q[:N].copy()
265             max_plan_balance = max(max_plan_balance, plan.balance_max_abs_kwh)
266             max_plan_state = max(max_plan_state, plan.state_max_abs_kwh)
267             plan_simultaneous += plan.simultaneous_positive_slots
268             secondary_days += int(plan.secondary_used)
269
270             load_kwh = load_kw[day_idx] / 6.0
271             pv_kwh = pv_kw[day_idx] / 6.0
272             execution = execute_fixed_plan(load_kwh, pv_kwh, q, soc)
273             if previous_close is not None:
274                 max_crossday_gap = max(max_crossday_gap, abs(soc - previous_close))

```

```

273 balance = (
274     q
275     + pv_kwh
276     + execution.discharge
277     + execution.emergency
278     - load_kwh
279     - execution.charge
280     - execution.surplus
281 )
282 state_residual = (
283     execution.soc[1:]
284     - execution.soc[:-1]
285     - ETA_CHARGE * execution.charge
286     + execution.discharge / ETA_DISCHARGE
287 )
288 max_actual_balance = max(max_actual_balance, float(np.max(np.abs(balance))))
289 max_actual_state = max(max_actual_state, float(np.max(np.abs(state_residual))))
290 min_soc = min(min_soc, float(np.min(execution.soc)))
291 max_soc = max(max_soc, float(np.max(execution.soc)))
292
293 normal_cost = float(np.dot(settlement_price, q))
294 emergency_cost = float(np.dot(EMERGENCY_MULTIPLIER * settlement_price,
295     ↪ execution.emergency))
296 daily = {
297     "date": day.isoformat(),
298     "train_end_date": train_end.isoformat(),
299     "residual_history_start_date": dates[residual_first_idx].isoformat(),
300     "residual_history_end_date": dates[residual_last_idx].isoformat(),
301     "soc_open_kwh": float(execution.soc[0]),
302     "soc_close_kwh": float(execution.soc[-1]),
303     "normal_purchase_kwh": float(np.sum(q)),
304     "emergency_purchase_kwh": float(np.sum(execution.emergency)),
305     "normal_cost_yuan": normal_cost,
306     "emergency_cost_yuan": emergency_cost,
307     "total_cost_yuan": normal_cost + emergency_cost,
308     "charge_kwh": float(np.sum(execution.charge)),
309     "discharge_kwh": float(np.sum(execution.discharge)),
310     "surplus_discard_kwh": float(np.sum(execution.surplus)),
311 }
312 daily_rows.append(daily)
313
314 if keep_detail:
315     plan_rows.append({
316         "date": day.isoformat(),
317         "purchase_kwh": [float(value) for value in q],
318         "total_purchase_kwh": daily["normal_purchase_kwh"],
319         "total_cost_yuan": daily["total_cost_yuan"],
320     })
321     for slot in range(N):
322         start, end, label = natural_slot_label(slot)
323         detail_rows.append({
324             "date": day.isoformat(),
325             "slot": slot + 1,
326             "source_endpoint_label": raw_endpoint_labels[slot],
327             "interval_start": start,
328             "interval_end": end,
329             "interval_label": label,
330             "plan_created_at": day.isoformat() + "T00:00:00",
331             "train_end_date": train_end.isoformat(),
332             "forecast_load_kw": float(forecast_load_2d[0, slot]),
333             "forecast_pv_kw": float(forecast_pv_2d[0, slot]),
334             "forecast_point_net_kw": float(point_net_2d_kw[0, slot]),
335             "risk_uplift_kw": float(risk_uplift_kw[slot]),
336             "forecast_price_yuan_per_kwh": float(plan_price_2d[slot]),
337             "actual_price_yuan_per_kwh": float(settlement_price[slot]),
338             "effective_forecast_net_kwh": float(effective_net_2d_kwh[0, slot]),
339             "actual_load_kw": float(load_kw[day_idx, slot]),
340             "actual_pv_kw": float(pv_kw[day_idx, slot]),
341             "load_kwh": float(load_kwh[slot]),
342             "pv_kwh": float(pv_kwh[slot]),
343             "normal_purchase_kwh": float(q[slot]),
344             "charge_bus_kwh": float(execution.charge[slot]),
345             "discharge_bus_kwh": float(execution.discharge[slot]),
346             "emergency_purchase_kwh": float(execution.emergency[slot]),
347             "surplus_discard_kwh": float(execution.surplus[slot]),

```

```

347         "soc_open_kwh": float(execution.soc[slot]),
348         "soc_close_kwh": float(execution.soc[slot + 1]),
349         "normal_cost_yuan": float(settlement_price[slot] * q[slot]),
350         "emergency_cost_yuan": float(EMERGENCY_MULTIPLIER *
351         ↪ settlement_price[slot] * execution.emergency[slot]),
352         "balance_residual_kwh": float(balance[slot]),
353         "state_residual_kwh": float(state_residual[slot]),
354     })
355     for block in range(6):
356         left = block * 24
357         right = (block + 1) * 24
358         storage_rows.append({
359             "date": day.isoformat(),
360             "block": block + 1,
361             "interval": f"{block * 4}:00-{(block + 1) * 4}:00",
362             "charge_kwh": float(np.sum(execution.charge[left:right])),
363             "discharge_kwh": float(np.sum(execution.discharge[left:right])),
364             "soc_open_day_kwh": float(execution.soc[0]),
365             "soc_close_day_kwh": float(execution.soc[-1]),
366         })
367     for event_id, (start_slot, end_slot, amount) in
368     ↪ enumerate(maximal_positive_events(execution.emergency), start=1):
369         event_rows.append({
370             "date": day.isoformat(),
371             "event_id": event_id,
372             "start_slot": start_slot + 1,
373             "end_slot_exclusive": end_slot + 1,
374             "interval": f"{natural_slot_label(start_slot)[0]}-{natural_slot_la
375             ↪ bel(end_slot - 1)[1]}",
376             "purchase_kwh": amount,
377         })
378     previous_close = float(execution.soc[-1])
379     soc = previous_close
380
381     aggregate = {
382         "strategy": name,
383         "forecast_kind": forecast_kind,
384         "risk_quantile": quantile,
385         "residual_window_days": RESIDUAL_WINDOW_DAYS if quantile is not None else None,
386         "terminal_mode": terminal_mode,
387         "days": len(daily_rows),
388         "normal_purchase_kwh": float(sum(row["normal_purchase_kwh"] for row in
389         ↪ daily_rows)),
390         "emergency_purchase_kwh": float(sum(row["emergency_purchase_kwh"] for row in
391         ↪ daily_rows)),
392         "normal_cost_yuan": float(sum(row["normal_cost_yuan"] for row in daily_rows)),
393         "emergency_cost_yuan": float(sum(row["emergency_cost_yuan"] for row in
394         ↪ daily_rows)),
395         "total_cost_yuan": float(sum(row["total_cost_yuan"] for row in daily_rows)),
396         "charge_kwh": float(sum(row["charge_kwh"] for row in daily_rows)),
397         "discharge_kwh": float(sum(row["discharge_kwh"] for row in daily_rows)),
398         "surplus_discard_kwh": float(sum(row["surplus_discard_kwh"] for row in
399         ↪ daily_rows)),
400         "soc_initial_kwh": SOC_INITIAL,
401         "soc_final_kwh": soc,
402         "soc_min_kwh": min_soc,
403         "soc_max_kwh": max_soc,
404         "max_plan_balance_residual_kwh": max_plan_balance,
405         "max_plan_state_residual_kwh": max_plan_state,
406         "plan_simultaneous_positive_slots": plan_simultaneous,
407         "plan_secondary_days": secondary_days,
408         "max_actual_balance_residual_kwh": max_actual_balance,
409         "max_actual_state_residual_kwh": max_actual_state,
410         "max_crossday_soc_gap_kwh": max_crossday_gap,
411     }
412     return {
413         "aggregate": aggregate,
414         "daily": daily_rows,
415         "detail": detail_rows,
416         "storage": storage_rows,
417         "events": event_rows,
418         "plans": plan_rows,
419     }

```

```

415
416 def write_csv(path: Path, rows: list[dict[str, Any]]) -> None:
417     if not rows:
418         raise ValueError(f"拒绝写入无表头空CSV: {path.name}")
419     with path.open("w", encoding="utf-8-sig", newline="") as stream:
420         writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
421         writer.writeheader()
422         writer.writerows(rows)
423
424
425 def main() -> None:
426     RESULTS.mkdir(parents=True, exist_ok=True)
427     VALIDATION.mkdir(parents=True, exist_ok=True)
428     INTERMEDIATE.mkdir(parents=True, exist_ok=True)
429     fixed = load_attachment1()
430     annual = load_attachment2()
431     dates = annual["dates"]
432     load_kw = annual["load_kw"]
433     pv_kw = annual["pv_kw"]
434     actual_net_kw = load_kw - pv_kw
435     price = fixed["price_yuan_per_kwh"]
436
437     config_path = INTERMEDIATE / "q2_main_config.json"
438     if config_path.exists():
439         main_config = json.loads(config_path.read_text(encoding="utf-8"))
440     else:
441         main_config = {"status": "KEEP_OLD_MAIN", "selected_forecast_kind":
442             ↪ "weekday_mean_28d"}
443     selected_kind = str(main_config["selected_forecast_kind"])
444
445     weekday_load_cache = make_historical_weekday_cache(load_kw, dates)
446     weekday_pv_cache = make_historical_weekday_cache(pv_kw, dates)
447     weekday_net_cache = weekday_load_cache - weekday_pv_cache
448     selected_load_cache, selected_pv_cache, selected_net_cache =
449     ↪ make_point_forecast_cache(
450         selected_kind, load_kw, pv_kw, dates
451     )
452     eval_slice = slice(EVAL_START, len(dates))
453     persistence_load = load_kw[EVAL_START - 1 : len(dates) - 1]
454     persistence_pv = pv_kw[EVAL_START - 1 : len(dates) - 1]
455     seasonal_load = weekday_load_cache[eval_slice]
456     seasonal_pv = weekday_pv_cache[eval_slice]
457     forecast_diagnostics = {
458         "persistence_ld": {
459             "load": forecast_metrics(load_kw[eval_slice], persistence_load),
460             "pv": forecast_metrics(pv_kw[eval_slice], persistence_pv),
461             "net": forecast_metrics(actual_net_kw[eval_slice], persistence_load -
462                 ↪ persistence_pv),
463         },
464         "weekday_mean_28d": {
465             "load": forecast_metrics(load_kw[eval_slice], seasonal_load),
466             "pv": forecast_metrics(pv_kw[eval_slice], seasonal_pv),
467             "net": forecast_metrics(actual_net_kw[eval_slice],
468                 ↪ weekday_net_cache[eval_slice]),
469         },
470         "selected_main_point_forecast": {
471             "forecast_kind": selected_kind,
472             "load": forecast_metrics(load_kw[eval_slice],
473                 ↪ selected_load_cache[eval_slice]),
474             "pv": forecast_metrics(pv_kw[eval_slice], selected_pv_cache[eval_slice]),
475             "net": forecast_metrics(actual_net_kw[eval_slice],
476                 ↪ selected_net_cache[eval_slice]),
477         },
478     }
479     q80_effective: list[np.ndarray] = []
480     for idx in range(EVAL_START, len(dates)):
481         uplift, _, _ = residual_quantile_kw(actual_net_kw, selected_net_cache, idx,
482             ↪ RISK_QUANTILE)
483         q80_effective.append(selected_net_cache[idx] + uplift)
484     q80_effective_array = np.asarray(q80_effective)
485     forecast_diagnostics["selected_main_plus_q80_residual"] = {
486         "forecast_kind": selected_kind,
487         "net": forecast_metrics(actual_net_kw[eval_slice], q80_effective_array),
488         "empirical_one_sided_coverage": float(np.mean(actual_net_kw[eval_slice] <=
489             ↪ q80_effective_array)),

```

```

482     "target_quantile": RISK_QUANTILE,
483 }
484
485 specs = [
486     ("baseline_persistence", "persistence", weekday_net_cache, None,
487      ↪ "fixed_6000_at_48h", False),
488     ("old_seasonal_q80", "weekday_mean_28d", weekday_net_cache, 0.8,
489      ↪ "fixed_6000_at_48h", False),
490     ("selected_point", selected_kind, selected_net_cache, None,
491      ↪ "fixed_6000_at_48h", False),
492     ("seasonal_q80_main", selected_kind, selected_net_cache, 0.8,
493      ↪ "fixed_6000_at_48h", True),
494     ("seasonal_q90_risk", selected_kind, selected_net_cache, 0.9,
495      ↪ "fixed_6000_at_48h", False),
496     ("seasonal_q80_terminal_hold", selected_kind, selected_net_cache, 0.8,
497      ↪ "initial_soc_at_48h", False),
498 ]
499 runs: dict[str, dict[str, Any]] = {}
500 for name, kind, point_cache, quantile, terminal_mode, keep_detail in specs:
501     print(json.dumps({"event": "q2_strategy_start", "strategy": name},
502      ↪ ensure_ascii=False), flush=True)
503     runs[name] = run_strategy(
504         name=name,
505         forecast_kind=kind,
506         quantile=quantile,
507         terminal_mode=terminal_mode,
508         load_kw=load_kw,
509         pv_kw=pv_kw,
510         dates=dates,
511         price=price,
512         actual_net_kw=actual_net_kw,
513         weekday_net_cache_kw=point_cache,
514         raw_endpoint_labels=annual["raw_endpoint_labels"],
515         keep_detail=keep_detail,
516     )
517     print(json.dumps({"event": "q2_strategy_complete", **runs[name]["aggregate"]},
518      ↪ ensure_ascii=False), flush=True)
519
520 main_run = runs["seasonal_q80_main"]
521 detail_path = RESULTS / "q2_detail.csv"
522 daily_path = RESULTS / "q2_daily.csv"
523 storage_path = RESULTS / "q2_storage_4h.csv"
524 events_path = RESULTS / "q2_emergency_events.csv"
525 write_csv(detail_path, main_run["detail"])
526 write_csv(daily_path, main_run["daily"])
527 write_csv(storage_path, main_run["storage"])
528 if main_run["events"]:
529     write_csv(events_path, main_run["events"])
530 else:
531     events_path.write_text("date,event_id,start_slot,end_slot_exclusive,interval,p
532     ↪ urchase_kwh\n", encoding="utf-8-sig")
533
534 selected_dates = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
535 selected_events = {
536     target: [row for row in main_run["events"] if row["date"] == target]
537     for target in selected_dates
538 }
539 aggregates = {name: run["aggregate"] for name, run in runs.items()}
540 baseline_cost = aggregates["baseline_persistence"]["total_cost_yuan"]
541 main_cost = aggregates["seasonal_q80_main"]["total_cost_yuan"]
542 summary = {
543     "schema_version": 1,
544     "status": "SOLVED_AND_VALIDATED",
545     "question": "Q2",
546     "evaluation_period": {"start": dates[EVAL_START].isoformat(), "end":
547     ↪ dates[-1].isoformat(), "days": len(dates) - EVAL_START},
548     "frozen_basis": {
549         "time": "natural_day_144_slots_right_endpoint_rectangle",
550         "information": "00:00 plan uses only prior-day-and-earlier history;
551         ↪ current-slot storage feedback uses no future slot",
552         "storage": "cross-day continuous; January idle gives 2025-02-01 opening SOC
553         ↪ 6000 kWh",
554         "emergency": "shortfall-only and charged at five times the delivery-slot
555         ↪ fixed daily price",

```

```

543         "known_conflict": "A01 remains; result workbook will use corrected
        ↪ natural-day labels with visible disclosure",
544     },
545     "model_settings": {
546         "baseline_forecast": "previous-day same-slot persistence",
547         "old_point_forecast": "mean of available same-weekday load and PV profiles
        ↪ in prior 28 days",
548         "upgraded_point_forecast": selected_kind,
549         "upgrade_gate": main_config["status"],
550         "residual_uplift": "slotwise causal trailing-56-day empirical 0.8
        ↪ quantile",
551         "risk_quantile_basis": "minimizes  $p*q + 5*p*$ shortage pointwise; critical
        ↪ fractile is 0.8",
552         "planning_horizon": "48 hours",
553         "terminal_main": "SOC=6000 kWh at the end of the moving 48-hour planning
        ↪ horizon; only first day is executed",
554         "execution": "fixed normal purchase plus causal greedy charge/discharge,
        ↪ then emergency for remaining shortage",
555     },
556     "forecast_diagnostics": forecast_diagnostics,
557     "strategies": aggregates,
558     "main_improvement_vs_persistence": {
559         "cost_saving_yuan": baseline_cost - main_cost,
560         "cost_saving_fraction":  $1.0 - \text{main\_cost} / \text{baseline\_cost}$ ,
561         "emergency_reduction_kwh":
        ↪ aggregates["baseline_persistence"]["emergency_purchase_kwh"] -
        ↪ aggregates["seasonal_q80_main"]["emergency_purchase_kwh"],
562     },
563     "main_improvement_vs_old_q80": {
564         "cost_saving_yuan": aggregates["old_seasonal_q80"]["total_cost_yuan"] -
        ↪ main_cost,
565         "cost_saving_fraction":  $1.0 - \text{main\_cost} /$ 
        ↪ aggregates["old_seasonal_q80"]["total_cost_yuan"],
566         "emergency_reduction_kwh":
        ↪ aggregates["old_seasonal_q80"]["emergency_purchase_kwh"] -
        ↪ aggregates["seasonal_q80_main"]["emergency_purchase_kwh"],
567     },
568     "selected_date_emergency_events": selected_events,
569     "files": {
570         "detail_csv": str(detail_path.relative_to(PROJECT_ROOT)),
571         "daily_csv": str(daily_path.relative_to(PROJECT_ROOT)),
572         "storage_4h_csv": str(storage_path.relative_to(PROJECT_ROOT)),
573         "emergency_events_csv": str(events_path.relative_to(PROJECT_ROOT)),
574     },
575 }
576 summary_path = RESULTS / "q2_summary.json"
577 summary_path.write_text(json.dumps(summary, ensure_ascii=False, indent=2,
        ↪ default=json_default), encoding="utf-8")
578
579 main_agg = main_run["aggregate"]
580 cost_from_daily = float(sum(row["total_cost_yuan"] for row in main_run["daily"]))
581 emergency_charge_overlap = sum(
582     1
583     for row in main_run["detail"]
584     if row["emergency_purchase_kwh"] > POSITIVE_TOL_KWH and row["charge_bus_kwh"] >
        ↪ POSITIVE_TOL_KWH
585 )
586 simultaneous_actual = sum(
587     1
588     for row in main_run["detail"]
589     if row["charge_bus_kwh"] > POSITIVE_TOL_KWH and row["discharge_bus_kwh"] >
        ↪ POSITIVE_TOL_KWH
590 )
591 information_violations = sum(
592     1 for row in main_run["daily"] if row["train_end_date"] >= row["date"] or
        ↪ row["residual_history_end_date"] >= row["date"]
593 )
594 pass_flags = {
595     "evaluation_days": len(main_run["daily"]) == 334,
596     "detail_rows": len(main_run["detail"]) == 334 * 144,
597     "storage_rows": len(main_run["storage"]) == 334 * 6,
598     "actual_balance": main_agg["max_actual_balance_residual_kwh"] <=
        ↪ SOLVER_TOL_KWH,
599     "actual_state": main_agg["max_actual_state_residual_kwh"] <= SOLVER_TOL_KWH,

```

```

600     "soc_bounds": main_agg["soc_min_kwh"] >= SOC_MIN - SOLVER_TOL_KWH and
        ↳ main_agg["soc_max_kwh"] <= SOC_MAX + SOLVER_TOL_KWH,
601     "crossday_soc": main_agg["max_crossday_soc_gap_kwh"] <= SOLVER_TOL_KWH,
602     "mutual_exclusion_actual": simultaneous_actual == 0,
603     "emergency_not_used_to_charge": emergency_charge_overlap == 0,
604     "information_cutoff": information_violations == 0,
605     "plan_lp_balance": main_agg["max_plan_balance_residual_kwh"] <= SOLVER_TOL_KWH,
606     "plan_lp_state": main_agg["max_plan_state_residual_kwh"] <= SOLVER_TOL_KWH,
607     "plan_lp_mutual_exclusion": main_agg["plan_simultaneous_positive_slots"] == 0,
608     "cost_reconciliation": abs(cost_from_daily - main_agg["total_cost_yuan"]) <=
        ↳ 1e-6,
609 }
610 validation = {
611     "schema_version": 1,
612     "question": "Q2",
613     "status": "PASS" if all(pass_flags.values()) else "FAIL",
614     "tolerances": {"solver_kwh": SOLVER_TOL_KWH, "positive_kwh": POSITIVE_TOL_KWH},
615     "pass_flags": pass_flags,
616     "metrics": {
617         "information_violations": information_violations,
618         "simultaneous_actual_slots": simultaneous_actual,
619         "emergency_charge_overlap_slots": emergency_charge_overlap,
620         "cost_from_daily_yuan": cost_from_daily,
621         **main_agg,
622     },
623     "all_strategy_aggregates": aggregates,
624     "environment": {
625         "python": sys.version,
626         "python_executable": sys.executable,
627         "platform": platform.platform(),
628         "numpy": np.__version__,
629         "scipy": scipy.__version__,
630         "solver": "scipy.optimize.linprog(method='highs')",
631     },
632 }
633 validation_path = VALIDATION / "q2_validation.json"
634 validation_path.write_text(json.dumps(validation, ensure_ascii=False, indent=2,
        ↳ default=json_default), encoding="utf-8")
635
636 workbook_payload = {
637     "schema_version": 1,
638     "question": "Q2",
639     "time_label_mapping":
        ↳ "natural_day_00:00_to_24:00_corrected_from_shifted_official_template",
640     "plans": main_run["plans"],
641     "storage_four_hour": main_run["storage"],
642     "emergency_events": main_run["events"],
643 }
644 workbook_payload_path = INTERMEDIATE / "q2_workbook_payload.json"
645 workbook_payload_path.write_text(json.dumps(workbook_payload, ensure_ascii=False,
        ↳ indent=2), encoding="utf-8")
646
647 run_manifest = {
648     "question": "Q2",
649     "status": validation["status"],
650     "input_sha256": {"attachment1": fixed["sha256"], "attachment2":
        ↳ annual["sha256"]},
651     "code_sha256": {
652         name: sha256(Path(__file__).with_name(name))
653         for name in ("common_data.py", "dispatch_core.py", "q2_solve.py")
654     },
655     "outputs_sha256": {
656         str(path.relative_to(PROJECT_ROOT)): sha256(path)
657         for path in (
658             detail_path,
659             daily_path,
660             storage_path,
661             events_path,
662             summary_path,
663             validation_path,
664             workbook_payload_path,
665         )
666     },
667 }
668 manifest_path = INTERMEDIATE / "q2_run_manifest.json"

```

```

669     manifest_path.write_text(json.dumps(run_manifest, ensure_ascii=False, indent=2),
        ↪ encoding="utf-8")
670     print(json.dumps({
671         "status": validation["status"],
672         "main_total_cost_yuan": main_cost,
673         "baseline_total_cost_yuan": baseline_cost,
674         "saving_fraction": 1.0 - main_cost / baseline_cost,
675         "main_emergency_kwh": main_agg["emergency_purchase_kwh"],
676         "main_final_soc_kwh": main_agg["soc_final_kwh"],
677     }, ensure_ascii=False))
678     if validation["status"] != "PASS":
679         raise RuntimeError("Q2主结果未通过约束核验")
680
681
682 if __name__ == "__main__":
683     main()

```

附录 C-3：问题三求解与负载反馈程序

该程序实现 0:00 初始计划与 6:00、12:00、18:00 日内滚动调整，并以因果负载反馈构造最终采用的 $\alpha = 0.5$ 预测修正。

问题三日内滚动优化

源文件：code/q3_solve.py

```

1  from __future__ import annotations
2
3  import csv
4  import json
5  import platform
6  import sys
7  from datetime import date, timedelta
8  from pathlib import Path
9  from typing import Any
10
11 import numpy as np
12import scipy
13
14from common_data import PROJECT_ROOT, load_attachment1, load_attachment2,
    ↪ load_attachment3, natural_slot_label, sha256
15from dispatch_core import (
16    ETA_CHARGE,
17    ETA_DISCHARGE,
18    FLOW_MAX_KWH,
19    POSITIVE_TOL_KWH,
20    SOC_INITIAL,
21    SOC_MAX,
22    SOC_MIN,
23    SOLVER_TOL_KWH,
24    execute_fixed_plan,
25    maximal_positive_events,
26    solve_adjusted_plan,
27    solve_deterministic_plan,
28)
29import q2_solve as q2
30
31
32RESULTS = PROJECT_ROOT / "results"
33VALIDATION = PROJECT_ROOT / "validation"
34INTERMEDIATE = PROJECT_ROOT / "intermediate"
35N = 144
36EVAL_START = 31
37LOOKBACK_DAYS = 28
38RESIDUAL_WINDOW_DAYS = 56
39RISK_QUANTILE = 0.8
40EMERGENCY_MULTIPLIER = 5.0
41ISSUE_HOURS = [0, 6, 12, 18]
42
43
44def weekday_mean(values_kw: np.ndarray, dates: list[date], cutoff_idx: int,
    ↪ target_date: date) -> np.ndarray:

```

```

45     candidates = [
46         idx for idx in range(max(0, cutoff_idx - LOOKBACK_DAYS), cutoff_idx)
47         if dates[idx].weekday() == target_date.weekday()
48     ]
49     if not candidates:
50         if cutoff_idx <= 0:
51             raise ValueError("没有历史数据可生成负载日型预测")
52         candidates = [cutoff_idx - 1]
53     return np.mean(values_kw[candidates], axis=0)
54
55
56 def causal_price_forecast_horizon(
57     actual_price_by_date: np.ndarray,
58     dates: list[date],
59     day_idx: int,
60     issue_hour: int,
61     kind: str,
62 ) -> tuple[np.ndarray, float]:
63     """Forecast the 24 hours after an issue using history and the observed same-day
64     ↪ prefix only."""
65     start = issue_hour * 6
66     if kind in {"weekday_mean_28d", "weekday_mean_28d_intraday_scale",
67     ↪ "weekday_mean_28d_intraday_scale_ar1"}:
68         today = weekday_mean(actual_price_by_date, dates, day_idx, dates[day_idx])
69         tomorrow = weekday_mean(actual_price_by_date, dates, day_idx, dates[day_idx] +
70     ↪ timedelta(days=1))
71     elif kind == "persistence":
72         today = actual_price_by_date[day_idx - 1]
73         tomorrow = actual_price_by_date[day_idx - 1]
74     else:
75         raise ValueError("实时价格模式必须明确 planning_price_kind")
76     scale = 1.0
77     if kind in {"weekday_mean_28d_intraday_scale",
78     ↪ "weekday_mean_28d_intraday_scale_ar1"} and start > 0:
79         denominator = float(np.sum(today[:start]))
80         if denominator <= 0.0:
81             raise ValueError("历史价格日型前缀和非正，无法进行因果比例修正")
82         scale = float(np.sum(actual_price_by_date[day_idx, :start]) / denominator)
83     horizon = np.concatenate([today[start:], tomorrow[:start]]) * scale
84     if kind == "weekday_mean_28d_intraday_scale_ar1" and start > 0:
85         residual_rows = []
86         for history_idx in range(max(7, day_idx - 28), day_idx):
87             history_base = weekday_mean(actual_price_by_date, dates, history_idx,
88             ↪ dates[history_idx])
89             residual_rows.append(actual_price_by_date[history_idx] - history_base)
90         residual = np.asarray(residual_rows)
91         lag = residual[:, :-1].reshape(-1)
92         lead = residual[:, 1:].reshape(-1)
93         denominator = float(np.dot(lag, lag))
94         rho = 0.0 if denominator <= 0.0 else float(np.clip(np.dot(lag, lead) /
95     ↪ denominator, -0.99, 0.99))
96         last_residual = float(actual_price_by_date[day_idx, start - 1] - today[start -
97     ↪ 1] * scale)
98         horizon = horizon + last_residual * rho ** np.arange(1, len(horizon) + 1,
99     ↪ dtype=float)
100     return np.maximum(horizon, 0.0), scale
101
102 def target_profile(values_kw: np.ndarray, dates: list[date], cutoff_idx: int,
103     ↪ issue_hour: int) -> np.ndarray:
104     start = issue_hour * 6
105     today = q2.weekday_weighted_forecast(values_kw, dates, cutoff_idx,
106     ↪ dates[cutoff_idx], "linear")
107     tomorrow = q2.weekday_weighted_forecast(values_kw, dates, cutoff_idx,
108     ↪ dates[cutoff_idx] + timedelta(days=1), "linear")
109     return np.concatenate([today[start:], tomorrow[:start]])
110
111 def actual_target(values_kw: np.ndarray, day_idx: int, issue_hour: int) -> np.ndarray:
112     start = issue_hour * 6
113     return np.concatenate([values_kw[day_idx, start:], values_kw[day_idx + 1, :start]])
114
115 def pv_forecast_10min(
116     forecast_leads_kw: np.ndarray,

```

```

109     boundary_kw: float,
110     interpolation: str,
111 ) -> np.ndarray:
112     leads = np.asarray(forecast_leads_kw, dtype=float)
113     if leads.shape != (24,):
114         raise ValueError("光伏预报必须包含提前1至24小时")
115     x = np.arange(1, 145, dtype=float) / 6.0
116     if interpolation == "linear":
117         return np.interp(x, np.arange(25, dtype=float), np.concatenate([[boundary_kw],
118                                     ↳ leads]))
119     if interpolation == "step_hold":
120         result = np.empty(144, dtype=float)
121         for idx, hour in enumerate(x):
122             if hour < 1.0:
123                 result[idx] = boundary_kw
124             else:
125                 lead_idx = min(int(np.floor(hour)) - 1, 23)
126                 result[idx] = leads[lead_idx]
127         return result
128     raise ValueError(f"未知插值方式: {interpolation}")
129
130 def issue_boundary_pv(pv_kw: np.ndarray, day_idx: int, issue_hour: int) -> float:
131     if issue_hour == 0:
132         return float(pv_kw[day_idx - 1, -1])
133     return float(pv_kw[day_idx, issue_hour * 6 - 1])
134
135 def issue_forecast(
136     *,
137     load_kw: np.ndarray,
138     pv_kw: np.ndarray,
139     forecast3_kw: np.ndarray,
140     dates: list[date],
141     day_idx: int,
142     issue_idx: int,
143     interpolation: str,
144 ) -> tuple[np.ndarray, np.ndarray]:
145     issue_hour = ISSUE_HOURS[issue_idx]
146     load = target_profile(load_kw, dates, day_idx, issue_hour)
147     pv = pv_forecast_10min(
148         forecast3_kw[day_idx, issue_idx],
149         issue_boundary_pv(pv_kw, day_idx, issue_hour),
150         interpolation,
151     )
152     return load, pv
153
154
155 def build_forecast_caches(
156     load_kw: np.ndarray,
157     pv_kw: np.ndarray,
158     forecast3_kw: np.ndarray,
159     dates: list[date],
160     interpolation: str,
161 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
162     residual = np.full((len(dates), 4, N), np.nan, dtype=float)
163     point_pv = np.full_like(residual, np.nan)
164     point_net = np.full_like(residual, np.nan)
165     for day_idx in range(1, len(dates) - 1):
166         for issue_idx, issue_hour in enumerate(ISSUE_HOURS):
167             forecast_load, forecast_pv = issue_forecast(
168                 load_kw=load_kw,
169                 pv_kw=pv_kw,
170                 forecast3_kw=forecast3_kw,
171                 dates=dates,
172                 day_idx=day_idx,
173                 issue_idx=issue_idx,
174                 interpolation=interpolation,
175             )
176             actual_net = actual_target(load_kw, day_idx, issue_hour) -
177                 ↳ actual_target(pv_kw, day_idx, issue_hour)
178             point_pv[day_idx, issue_idx] = forecast_pv
179             point_net[day_idx, issue_idx] = forecast_load - forecast_pv
180             residual[day_idx, issue_idx] = actual_net - point_net[day_idx, issue_idx]
181     return residual, point_pv, point_net

```

```

182
183
184 def risk_uplift(residual_cache: np.ndarray, day_idx: int, issue_idx: int) ->
    ↳ tuple[np.ndarray, int, int]:
185     first = max(1, day_idx - RESIDUAL_WINDOW_DAYS)
186     candidates = np.arange(first, day_idx, dtype=int)
187     valid = candidates[np.isfinite(residual_cache[candidates, issue_idx]).all(axis=1)]
188     if len(valid) == 0:
189         raise ValueError("发布时间对应的历史残差为空")
190     return np.quantile(residual_cache[valid, issue_idx], RISK_QUANTILE, axis=0),
    ↳ int(valid[0]), int(valid[-1])
191
192
193 def metric(actual: np.ndarray, forecast: np.ndarray) -> dict[str, float]:
194     error = forecast - actual
195     return {
196         "mae_kw": float(np.mean(np.abs(error))),
197         "rmse_kw": float(np.sqrt(np.mean(error**2))),
198         "wape": float(np.sum(np.abs(error)) / np.sum(np.abs(actual))),
199         "bias_kw": float(np.mean(error)),
200     }
201
202
203 def run_strategy(
204     *,
205     name: str,
206     update_hours: list[int],
207     interpolation: str,
208     load_kw: np.ndarray,
209     pv_kw: np.ndarray,
210     forecast3_kw: np.ndarray,
211     dates: list[date],
212     price: np.ndarray,
213     residual_cache: np.ndarray,
214     raw_endpoint_labels: list[str],
215     keep_detail: bool,
216     actual_price_by_date: np.ndarray | None = None,
217     planning_price_kind: str | None = None,
218 ) -> dict[str, Any]:
219     soc = SOC_INITIAL
220     daily_rows: list[dict[str, Any]] = []
221     detail_rows: list[dict[str, Any]] = []
222     storage_rows: list[dict[str, Any]] = []
223     event_rows: list[dict[str, Any]] = []
224     version_rows: list[dict[str, Any]] = []
225     workbook_plans: list[dict[str, Any]] = []
226     max_plan_balance = max_plan_state = max_link = 0.0
227     max_actual_balance = max_actual_state = max_crossday_gap = 0.0
228     min_soc = max_soc = soc
229     plan_simultaneous = secondary_plans = 0
230     previous_close: float | None = None
231
232     for day_idx in range(EVAL_START, len(dates)):
233         day = dates[day_idx]
234         if actual_price_by_date is None:
235             plan_price_today = price
236             plan_price_tomorrow = price
237             settlement_price = price
238         else:
239             if planning_price_kind in {"weekday_mean_28d",
    ↳ "weekday_mean_28d_intraday_scale",
    ↳ "weekday_mean_28d_intraday_scale_ar1"}:
240                 plan_price_today = weekday_mean(actual_price_by_date, dates, day_idx,
    ↳ day)
241                 plan_price_tomorrow = weekday_mean(actual_price_by_date, dates,
    ↳ day_idx, day + timedelta(days=1))
242             elif planning_price_kind == "persistence":
243                 plan_price_today = actual_price_by_date[day_idx - 1]
244                 plan_price_tomorrow = actual_price_by_date[day_idx - 1]
245             else:
246                 raise ValueError("实时价格模式必须明确 planning_price_kind")
247             settlement_price = actual_price_by_date[day_idx]
248             forecast_load_0, forecast_pv_0 = issue_forecast(
249                 load_kw=load_kw, pv_kw=pv_kw, forecast3_kw=forecast3_kw, dates=dates,
250                 day_idx=day_idx, issue_idx=0, interpolation=interpolation,

```

```

251 )
252 uplift_0, residual_first, residual_last = risk_uplift(residual_cache, day_idx,
    ↪ 0)
253 tomorrow_load = q2.weekday_weighted_forecast(load_kw, dates, day_idx, day +
    ↪ timedelta(days=1), "linear")
254 tomorrow_pv = np.mean(pv_kw[max(0, day_idx - 5) : day_idx], axis=0)
255 net_48h_kwh = np.concatenate([
256     forecast_load_0 - forecast_pv_0 + uplift_0,
257     tomorrow_load - tomorrow_pv + uplift_0,
258 ]) / 6.0
259 q0_plan = solve_deterministic_plan(
260     net_48h_kwh, np.concatenate([plan_price_today, plan_price_tomorrow]), soc,
    ↪ SOC_INITIAL
261 )
262 q0 = q0_plan.q[:N].copy()
263 max_plan_balance = max(max_plan_balance, q0_plan.balance_max_abs_kwh)
264 max_plan_state = max(max_plan_state, q0_plan.state_max_abs_kwh)
265 plan_simultaneous += q0_plan.simultaneous_positive_slots
266 secondary_plans += int(q0_plan.secondary_used)
267
268 effective = q0.copy()
269 used_issue = np.zeros(N, dtype=int)
270 active_load_forecast = forecast_load_0.copy()
271 active_pv_forecast = forecast_pv_0.copy()
272 active_uplift = uplift_0.copy()
273 active_price_forecast = plan_price_today.copy()
274 charge = np.zeros(N)
275 discharge = np.zeros(N)
276 emergency = np.zeros(N)
277 surplus = np.zeros(N)
278 soc_path = np.zeros(N + 1)
279 soc_path[0] = soc
280 boundaries = sorted([0, *[hour * 6 for hour in update_hours], N])
281 current_soc = soc
282
283 if keep_detail:
284     for slot in range(N):
285         version_rows.append({
286             "date": day.isoformat(), "issue_time": "00:00", "issue_slot": 0,
287             "target_slot": slot + 1, "interval": natural_slot_label(slot)[2],
288             "original_plan_kwh": float(q0[slot]), "new_effective_plan_kwh":
    ↪ float(q0[slot]),
289             "change_kwh": 0.0, "train_end_date": dates[day_idx -
    ↪ 1].isoformat(),
290         })
291
292 for segment_idx in range(len(boundaries) - 1):
293     start = boundaries[segment_idx]
294     end = boundaries[segment_idx + 1]
295     if start > 0:
296         issue_hour = start // 6
297         issue_idx = ISSUE_HOURS.index(issue_hour)
298         forecast_load, forecast_pv = issue_forecast(
299             load_kw=load_kw, pv_kw=pv_kw, forecast3_kw=forecast3_kw,
    ↪ dates=dates,
300             day_idx=day_idx, issue_idx=issue_idx, interpolation=interpolation,
301         )
302         uplift, residual_first, residual_last = risk_uplift(residual_cache,
    ↪ day_idx, issue_idx)
303         effective_net_kwh = (forecast_load - forecast_pv + uplift) / 6.0
304         adjustable_count = N - start
305         original_horizon = np.concatenate([q0[start:], np.zeros(start)])
306         if actual_price_by_date is None:
307             price_horizon = np.concatenate([plan_price_today[start:],
    ↪ plan_price_tomorrow[:start]])
308         else:
309             price_horizon, _ = causal_price_forecast_horizon(
310                 actual_price_by_date, dates, day_idx, issue_hour,
    ↪ str(planning_price_kind)
311             )
312         adjusted = solve_adjusted_plan(
313             effective_net_kwh,
314             price_horizon,
315             current_soc,
316             SOC_INITIAL,

```

```

317         original_horizon,
318         adjustable_count,
319     )
320     max_plan_balance = max(max_plan_balance, adjusted.balance_max_abs_kwh)
321     max_plan_state = max(max_plan_state, adjusted.state_max_abs_kwh)
322     max_link = max(max_link, adjusted.adjustment_link_max_abs_kwh)
323     plan_simultaneous += adjusted.simultaneous_positive_slots
324     secondary_plans += int(adjusted.secondary_used)
325     effective[start:] = adjusted.purchase[:adjustable_count]
326     used_issue[start:] = issue_hour
327     active_load_forecast[start:] = forecast_load[:adjustable_count]
328     active_pv_forecast[start:] = forecast_pv[:adjustable_count]
329     active_uplift[start:] = uplift[:adjustable_count]
330     active_price_forecast[start:] = price_horizon[:adjustable_count]
331     if keep_detail:
332         for slot in range(start, N):
333             version_rows.append({
334                 "date": day.isoformat(), "issue_time":
335                 ↪ f"{issue_hour:02d}:00", "issue_slot": start,
336                 "target_slot": slot + 1, "interval":
337                 ↪ natural_slot_label(slot)[2],
338                 "original_plan_kwh": float(q0[slot]),
339                 ↪ "new_effective_plan_kwh": float(effective[slot]),
340                 "change_kwh": float(effective[slot] - q0[slot]),
341                 ↪ "train_end_date": dates[day_idx - 1].isoformat(),
342             })
343
344     executed = execute_fixed_plan(
345         load_kw[day_idx, start:end] / 6.0,
346         pv_kw[day_idx, start:end] / 6.0,
347         effective[start:end],
348         current_soc,
349     )
350     charge[start:end] = executed.charge
351     discharge[start:end] = executed.discharge
352     emergency[start:end] = executed.emergency
353     surplus[start:end] = executed.surplus
354     soc_path[start : end + 1] = executed.soc
355     current_soc = float(executed.soc[-1])
356
357     load_kwh = load_kw[day_idx] / 6.0
358     pv_kwh = pv_kw[day_idx] / 6.0
359     balance = effective + pv_kwh + discharge + emergency - load_kwh - charge -
360     ↪ surplus
361     state_residual = soc_path[1:] - soc_path[:-1] - ETA_CHARGE * charge + discharge
362     ↪ / ETA_DISCHARGE
363     max_actual_balance = max(max_actual_balance, float(np.max(np.abs(balance))))
364     max_actual_state = max(max_actual_state, float(np.max(np.abs(state_residual))))
365     min_soc = min(min_soc, float(np.min(soc_path)))
366     max_soc = max(max_soc, float(np.max(soc_path)))
367     if previous_close is not None:
368         max_crossday_gap = max(max_crossday_gap, abs(float(soc_path[0]) -
369         ↪ previous_close))
370
371     upward = np.maximum(effective - q0, 0.0)
372     downward = np.maximum(q0 - effective, 0.0)
373     base_cost_slot = settlement_price * q0
374     down_credit_slot = 0.5 * settlement_price * downward
375     up_cost_slot = 1.5 * settlement_price * upward
376     normal_settlement_slot = base_cost_slot - down_credit_slot + up_cost_slot
377     emergency_cost_slot = EMERGENCY_MULTIPLIER * settlement_price * emergency
378     total_cost_slot = normal_settlement_slot + emergency_cost_slot
379     daily = {
380         "date": day.isoformat(),
381         "train_end_date": dates[day_idx - 1].isoformat(),
382         "residual_history_start_date": dates[residual_first].isoformat(),
383         "residual_history_end_date": dates[residual_last].isoformat(),
384         "soc_open_kwh": float(soc_path[0]),
385         "soc_close_kwh": float(soc_path[-1]),
386         "original_purchase_kwh": float(np.sum(q0)),
387         "effective_purchase_kwh": float(np.sum(effective)),
388         "upward_adjustment_kwh": float(np.sum(upward)),
389         "downward_adjustment_kwh": float(np.sum(downward)),
390         "emergency_purchase_kwh": float(np.sum(emergency)),
391         "base_plan_cost_yuan": float(np.sum(base_cost_slot)),

```

```

385     "downward_credit_yuan": float(np.sum(down_credit_slot)),
386     "upward_adjustment_cost_yuan": float(np.sum(up_cost_slot)),
387     "normal_settlement_cost_yuan": float(np.sum(normal_settlement_slot)),
388     "emergency_cost_yuan": float(np.sum(emergency_cost_slot)),
389     "total_cost_yuan": float(np.sum(total_cost_slot)),
390     "charge_kwh": float(np.sum(charge)),
391     "discharge_kwh": float(np.sum(discharge)),
392     "surplus_discard_kwh": float(np.sum(surplus)),
393 }
394 daily_rows.append(daily)
395
396 if keep_detail:
397     workbook_plans.append({
398         "date": day.isoformat(),
399         "original_purchase_kwh": [float(value) for value in q0],
400         "effective_purchase_kwh": [float(value) for value in effective],
401         "original_total_kwh": daily["original_purchase_kwh"],
402         "effective_total_kwh": daily["effective_purchase_kwh"],
403         "total_cost_yuan": daily["total_cost_yuan"],
404     })
405     for slot in range(N):
406         start_text, end_text, label = natural_slot_label(slot)
407         detail_rows.append({
408             "date": day.isoformat(), "slot": slot + 1,
409             "source_endpoint_label": raw_endpoint_labels[slot],
410             "interval_start": start_text, "interval_end": end_text,
411             ↪ "interval_label": label,
412             "executed_plan_issue_time": f"{used_issue[slot]:02d}:00",
413             "train_end_date": dates[day_idx - 1].isoformat(),
414             "forecast_load_kw": float(active_load_forecast[slot]),
415             "forecast_pv_kw": float(active_pv_forecast[slot]),
416             "forecast_point_net_kw": float(active_load_forecast[slot] -
417             ↪ active_pv_forecast[slot]),
418             "risk_uplift_kw": float(active_uplift[slot]),
419             "forecast_price_yuan_per_kwh": float(active_price_forecast[slot]),
420             "actual_price_yuan_per_kwh": float(settlement_price[slot]),
421             "actual_load_kw": float(load_kw[day_idx, slot]),
422             "actual_pv_kw": float(pv_kw[day_idx, slot]),
423             "load_kwh": float(load_kwh[slot]), "pv_kwh": float(pv_kwh[slot]),
424             "original_purchase_kwh": float(q0[slot]),
425             "effective_purchase_kwh": float(effective[slot]),
426             "upward_adjustment_kwh": float(upward[slot]),
427             "downward_adjustment_kwh": float(downward[slot]),
428             "charge_bus_kwh": float(charge[slot]), "discharge_bus_kwh":
429             ↪ float(discharge[slot]),
430             "emergency_purchase_kwh": float(emergency[slot]),
431             "surplus_discard_kwh": float(surplus[slot]),
432             "soc_open_kwh": float(soc_path[slot]), "soc_close_kwh":
433             ↪ float(soc_path[slot + 1]),
434             "base_plan_cost_yuan": float(base_cost_slot[slot]),
435             "downward_credit_yuan": float(down_credit_slot[slot]),
436             "upward_adjustment_cost_yuan": float(up_cost_slot[slot]),
437             "normal_settlement_cost_yuan": float(normal_settlement_slot[slot]),
438             "emergency_cost_yuan": float(emergency_cost_slot[slot]),
439             "total_cost_yuan": float(total_cost_slot[slot]),
440             "balance_residual_kwh": float(balance[slot]),
441             "state_residual_kwh": float(state_residual[slot]),
442         })
443     for block in range(6):
444         left, right = block * 24, (block + 1) * 24
445         storage_rows.append({
446             "date": day.isoformat(), "block": block + 1,
447             "interval": f"{block * 4}:00-{(block + 1) * 4}:00",
448             "charge_kwh": float(np.sum(charge[left:right])),
449             "discharge_kwh": float(np.sum(discharge[left:right])),
450             "soc_open_day_kwh": float(soc_path[0]), "soc_close_day_kwh":
451             ↪ float(soc_path[-1]),
452         })
453     for event_id, (event_start, event_end, amount) in
454     ↪ enumerate(maximal_positive_events(emergency), start=1):
455         event_rows.append({
456             "date": day.isoformat(), "event_id": event_id,
457             "start_slot": event_start + 1, "end_slot_exclusive": event_end + 1,
458             "interval": f"{natural_slot_label(event_start)[0]}-{natural_slot_l
459             ↪ abel(event_end - 1)[1]}",

```

```

452         "purchase_kwh": amount,
453     })
454
455     previous_close = float(soc_path[-1])
456     soc = previous_close
457
458     aggregate = {
459         "strategy": name, "update_hours": update_hours, "interpolation": interpolation,
460         "days": len(daily_rows),
461         "original_purchase_kwh": float(sum(row["original_purchase_kwh"] for row in
462             ↪ daily_rows)),
463         "effective_purchase_kwh": float(sum(row["effective_purchase_kwh"] for row in
464             ↪ daily_rows)),
465         "upward_adjustment_kwh": float(sum(row["upward_adjustment_kwh"] for row in
466             ↪ daily_rows)),
467         "downward_adjustment_kwh": float(sum(row["downward_adjustment_kwh"] for row in
468             ↪ daily_rows)),
469         "emergency_purchase_kwh": float(sum(row["emergency_purchase_kwh"] for row in
470             ↪ daily_rows)),
471         "base_plan_cost_yuan": float(sum(row["base_plan_cost_yuan"] for row in
472             ↪ daily_rows)),
473         "downward_credit_yuan": float(sum(row["downward_credit_yuan"] for row in
474             ↪ daily_rows)),
475         "upward_adjustment_cost_yuan": float(sum(row["upward_adjustment_cost_yuan"] for
476             ↪ row in daily_rows)),
477         "normal_settlement_cost_yuan": float(sum(row["normal_settlement_cost_yuan"] for
478             ↪ row in daily_rows)),
479         "emergency_cost_yuan": float(sum(row["emergency_cost_yuan"] for row in
480             ↪ daily_rows)),
481         "total_cost_yuan": float(sum(row["total_cost_yuan"] for row in daily_rows)),
482         "charge_kwh": float(sum(row["charge_kwh"] for row in daily_rows)),
483         "discharge_kwh": float(sum(row["discharge_kwh"] for row in daily_rows)),
484         "surplus_discard_kwh": float(sum(row["surplus_discard_kwh"] for row in
485             ↪ daily_rows)),
486         "soc_initial_kwh": SOC_INITIAL, "soc_final_kwh": soc,
487         "soc_min_kwh": min_soc, "soc_max_kwh": max_soc,
488         "max_plan_balance_residual_kwh": max_plan_balance,
489         "max_plan_state_residual_kwh": max_plan_state,
490         "max_adjustment_link_residual_kwh": max_link,
491         "plan_simultaneous_positive_slots": plan_simultaneous,
492         "plan_secondary_count": secondary_plans,
493         "max_actual_balance_residual_kwh": max_actual_balance,
494         "max_actual_state_residual_kwh": max_actual_state,
495         "max_crossday_soc_gap_kwh": max_crossday_gap,
496     }
497
498     return {
499         "aggregate": aggregate, "daily": daily_rows, "detail": detail_rows,
500         "storage": storage_rows, "events": event_rows, "versions": version_rows,
501         "workbook_plans": workbook_plans,
502     }
503
504 def write_csv(path: Path, rows: list[dict[str, Any]]) -> None:
505     if not rows:
506         raise ValueError(f"拒绝写入无表头空CSV: {path.name}")
507     with path.open("w", encoding="utf-8-sig", newline="") as stream:
508         writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
509         writer.writeheader()
510         writer.writerows(rows)
511
512 def main() -> None:
513     RESULTS.mkdir(parents=True, exist_ok=True)
514     VALIDATION.mkdir(parents=True, exist_ok=True)
515     INTERMEDIATE.mkdir(parents=True, exist_ok=True)
516     fixed = load_attachment1()
517     annual = load_attachment2()
518     forecasts = load_attachment3()
519     if annual["dates"] != forecasts["dates"]:
520         raise ValueError("附件2与附件3日期轴不一致")
521     dates = annual["dates"]
522     load_kw, pv_kw = annual["load_kw"], annual["pv_kw"]
523     forecast3_kw = forecasts["forecast_kw"]
524     price = fixed["price_yuan_per_kwh"]
525

```

```

516 cache_by_interpolation = {}
517 forecast_diagnostics = {}
518 for interpolation in ("linear", "step_hold"):
519     residual, point_pv, point_net = build_forecast_caches(load_kw, pv_kw,
520     ↪ forecast3_kw, dates, interpolation)
521     cache_by_interpolation[interpolation] = residual
522     diagnostics = {}
523     for issue_idx, issue_hour in enumerate(ISSUE_HOURS):
524         valid_days = np.arange(EVAL_START, len(dates) - 1)
525         actual_pv = np.asarray([actual_target(pv_kw, idx, issue_hour) for idx in
526         ↪ valid_days])
527         actual_net = np.asarray([actual_target(load_kw, idx, issue_hour) -
528         ↪ actual_target(pv_kw, idx, issue_hour) for idx in valid_days])
529         diagnostics[f"{issue_hour:02d}:00"] = {
530             "pv": metric(actual_pv, point_pv[valid_days, issue_idx]),
531             "net": metric(actual_net, point_net[valid_days, issue_idx]),
532         }
533     forecast_diagnostics[interpolation] = diagnostics
534
535 specs = [
536     ("q3_0only", [], "linear", False),
537     ("q3_0_6", [6], "linear", False),
538     ("q3_0_6_12", [6, 12], "linear", False),
539     ("q3_all_main", [6, 12, 18], "linear", True),
540     ("q3_all_step_hold", [6, 12, 18], "step_hold", False),
541 ]
542 runs = {}
543 for name, update_hours, interpolation, keep_detail in specs:
544     print(json.dumps({"event": "q3_strategy_start", "strategy": name},
545     ↪ ensure_ascii=False), flush=True)
546     runs[name] = run_strategy(
547         name=name, update_hours=update_hours, interpolation=interpolation,
548         load_kw=load_kw, pv_kw=pv_kw, forecast3_kw=forecast3_kw, dates=dates,
549         price=price, residual_cache=cache_by_interpolation[interpolation],
550         raw_endpoint_labels=annual["raw_endpoint_labels"], keep_detail=keep_detail,
551     )
552     print(json.dumps({"event": "q3_strategy_complete", **runs[name]["aggregate"]},
553     ↪ ensure_ascii=False), flush=True)
554
555 main_run = runs["q3_all_main"]
556 paths = {
557     "detail": RESULTS / "q3_detail.csv",
558     "daily": RESULTS / "q3_daily.csv",
559     "storage": RESULTS / "q3_storage_4h.csv",
560     "events": RESULTS / "q3_emergency_events.csv",
561     "versions": RESULTS / "q3_plan_versions.csv",
562 }
563 write_csv(paths["detail"], main_run["detail"])
564 write_csv(paths["daily"], main_run["daily"])
565 write_csv(paths["storage"], main_run["storage"])
566 write_csv(paths["versions"], main_run["versions"])
567 if main_run["events"]:
568     write_csv(paths["events"], main_run["events"])
569 else:
570     paths["events"].write_text("date,event_id,start_slot,end_slot_exclusive,interv
571     ↪ al,purchase_kwh\n", encoding="utf-8-sig")
572
573 aggregates = {name: run["aggregate"] for name, run in runs.items()}
574 ordered = ["q3_0only", "q3_0_6", "q3_0_6_12", "q3_all_main"]
575 incremental = []
576 for previous, current in zip(ordered[:-1], ordered[1:]):
577     incremental.append({
578         "from": previous, "to": current,
579         "cost_change_yuan": aggregates[current]["total_cost_yuan"] -
580         ↪ aggregates[previous]["total_cost_yuan"],
581         "cost_saving_yuan": aggregates[previous]["total_cost_yuan"] -
582         ↪ aggregates[current]["total_cost_yuan"],
583         "emergency_change_kwh": aggregates[current]["emergency_purchase_kwh"] -
584         ↪ aggregates[previous]["emergency_purchase_kwh"],
585     })
586 selected_dates = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
587 selected_events = {target: [row for row in main_run["events"] if row["date"] ==
588     ↪ target] for target in selected_dates}
589 summary = {
590     "schema_version": 1, "status": "SOLVED_AND_VALIDATED", "question": "Q3",

```

```

581     "evaluation_period": {"start": dates[EVAL_START].isoformat(), "end":
    ↪ dates[-1].isoformat(), "days": 334},
582     "frozen_basis": {
583         "information": "PV forecasts arrive at 00/06/12/18; only unexecuted slots
    ↪ may be revised; load forecast is held to the 00:00 seasonal model",
584         "settlement": "final effective normal purchase is settled once relative to
    ↪ the 00:00 original plan",
585         "storage": "causal ten-minute execution with cross-day SOC continuity",
586         "known_conflict": "A01 remains and result workbook uses visibly disclosed
    ↪ corrected labels",
587     },
588     "model_settings": {
589         "pv_interpolation_main": "causal piecewise linear from observed issue-time
    ↪ boundary to lead-hour forecasts",
590         "pv_interpolation_sensitivity": "step hold",
591         "load_forecast": "prior-28-day same-weekday linearly weighted load fixed
    ↪ from 00:00 for all issue times",
592         "residual_risk": "same-issue trailing-56-day slotwise q80 of combined
    ↪ net-load residual",
593         "q0_horizon": "48 hours; attachment3 covers first 24 hours and seasonal
    ↪ PV/load extends the virtual second day",
594         "adjustment_horizon": "24 hours from each issue; current-day remaining
    ↪ slots use adjustment tariff, virtual next-day slots use normal tariff",
595     },
596     "forecast_diagnostics": forecast_diagnostics,
597     "strategies": aggregates,
598     "incremental_update_value": incremental,
599     "selected_date_emergency_events": selected_events,
600     "files": {key: str(value.relative_to(PROJECT_ROOT)) for key, value in
    ↪ paths.items()},
601 }
602 summary_path = RESULTS / "q3_summary.json"
603 summary_path.write_text(json.dumps(summary, ensure_ascii=False, indent=2),
    ↪ encoding="utf-8")
604
605 agg = main_run["aggregate"]
606 past_lock_violations = sum(1 for row in main_run["versions"] if row["issue_slot"] >
    ↪ 0 and row["target_slot"] <= row["issue_slot"])
607 information_violations = sum(1 for row in main_run["daily"] if
    ↪ row["train_end_date"] >= row["date"] or row["residual_history_end_date"] >=
    ↪ row["date"])
608 simultaneous_actual = sum(1 for row in main_run["detail"] if row["charge_bus_kwh"]
    ↪ > POSITIVE_TOL_KWH and row["discharge_bus_kwh"] > POSITIVE_TOL_KWH)
609 emergency_charge_overlap = sum(1 for row in main_run["detail"] if
    ↪ row["emergency_purchase_kwh"] > POSITIVE_TOL_KWH and row["charge_bus_kwh"] >
    ↪ POSITIVE_TOL_KWH)
610 max_actual_charge = max(float(row["charge_bus_kwh"]) for row in main_run["detail"])
611 max_actual_discharge = max(float(row["discharge_bus_kwh"]) for row in
    ↪ main_run["detail"])
612 daily_cost = float(sum(row["total_cost_yuan"] for row in main_run["daily"]))
613 pass_flags = {
614     "rows": len(main_run["detail"]) == 334 * 144 and len(main_run["storage"]) ==
    ↪ 334 * 6,
615     "past_lock": past_lock_violations == 0,
616     "information_cutoff": information_violations == 0,
617     "actual_balance": agg["max_actual_balance_residual_kwh"] <= SOLVER_TOL_KWH,
618     "actual_state": agg["max_actual_state_residual_kwh"] <= SOLVER_TOL_KWH,
619     "soc_bounds": agg["soc_min_kwh"] >= SOC_MIN - SOLVER_TOL_KWH and
    ↪ agg["soc_max_kwh"] <= SOC_MAX + SOLVER_TOL_KWH,
620     "storage_power": max_actual_charge <= FLOW_MAX_KWH + SOLVER_TOL_KWH and
    ↪ max_actual_discharge <= FLOW_MAX_KWH + SOLVER_TOL_KWH,
621     "crossday_soc": agg["max_crossday_soc_gap_kwh"] <= SOLVER_TOL_KWH,
622     "mutual_exclusion_actual": simultaneous_actual == 0,
623     "emergency_not_charging": emergency_charge_overlap == 0,
624     "plan_balance": agg["max_plan_balance_residual_kwh"] <= SOLVER_TOL_KWH,
625     "plan_state": agg["max_plan_state_residual_kwh"] <= SOLVER_TOL_KWH,
626     "adjustment_link": agg["max_adjustment_link_residual_kwh"] <= SOLVER_TOL_KWH,
627     "plan_mutual_exclusion": agg["plan_simultaneous_positive_slots"] == 0,
628     "cost_reconciliation": abs(daily_cost - agg["total_cost_yuan"]) <= 1e-6,
629 }
630 validation = {
631     "schema_version": 1, "question": "Q3", "status": "PASS" if
    ↪ all(pass_flags.values()) else "FAIL",
632     "pass_flags": pass_flags,

```

```

633     "metrics": {"past_lock_violations": past_lock_violations,
        ↪ "information_violations": information_violations,
634             "simultaneous_actual_slots": simultaneous_actual,
        ↪ "emergency_charge_overlap_slots": emergency_charge_overlap,
635             "storage_power_limit_kw": 5000.0,
        ↪ "storage_flow_limit_kwh_per_slot": FLOW_MAX_KWH,
636             "max_actual_charge_kwh_per_slot": max_actual_charge,
        ↪ "max_actual_discharge_kwh_per_slot": max_actual_discharge,
        ↪ "daily_cost_recompute_yuan": daily_cost, **agg},
637     "all_strategy_aggregates": aggregates,
638     "environment": {"python": sys.version, "python_executable": sys.executable,
639                     ↪ "platform": platform.platform(),
640                     ↪ "numpy": np.__version__, "scipy": scipy.__version__, "solver":
        ↪ "scipy.optimize.linprog(method='highs')"},
641 }
642 validation_path = VALIDATION / "q3_validation.json"
643 validation_path.write_text(json.dumps(validation, ensure_ascii=False, indent=2),
        ↪ encoding="utf-8")
644
645 workbook_payload = {
646     "schema_version": 1, "question": "Q3",
647     "time_label_mapping":
        ↪ "natural_day_00:00_to_24:00_corrected_from_shifted_official_template",
648     "plans": main_run["workbook_plans"], "storage_four_hour": main_run["storage"],
649     "emergency_events": main_run["events"],
650 }
651 workbook_payload_path = INTERMEDIATE / "q3_workbook_payload.json"
652 workbook_payload_path.write_text(json.dumps(workbook_payload, ensure_ascii=False,
        ↪ indent=2), encoding="utf-8")
653 run_manifest = {
654     "question": "Q3", "status": validation["status"],
655     "input_sha256": {"attachment1": fixed["sha256"], "attachment2":
        ↪ annual["sha256"], "attachment3": forecasts["sha256"]},
656     "code_sha256": {name: sha256(Path(__file__).with_name(name)) for name in
        ↪ ("common_data.py", "dispatch_core.py", "q3_solve.py")},
657     "outputs_sha256": {str(path.relative_to(PROJECT_ROOT)): sha256(path) for path
        ↪ in [*paths.values(), summary_path, validation_path,
        ↪ workbook_payload_path]},
658 }
659 (INTERMEDIATE / "q3_run_manifest.json").write_text(json.dumps(run_manifest,
        ↪ ensure_ascii=False, indent=2), encoding="utf-8")
660 print(json.dumps({"status": validation["status"], "main_total_cost_yuan":
        ↪ agg["total_cost_yuan"],
661                 "q3_0only_cost_yuan": aggregates["q3_0only"]["total_cost_yuan"],
662                 "saving_vs_0only_yuan": aggregates["q3_0only"]["total_cost_yuan"]
        ↪ - agg["total_cost_yuan"],
663                 "main_emergency_kwh": agg["emergency_purchase_kwh"],
        ↪ "main_final_soc_kwh": agg["soc_final_kwh"]},
        ↪ ensure_ascii=False))
664 if validation["status"] != "PASS":
665     raise RuntimeError("Q3主结果未通过约束核验")
666
667
668 if __name__ == "__main__":
669     main()

```

附录 C-4：问题四波动电价求解程序

该程序读取实际电价数据，形成日前电价预测，并在波动电价下分别实现仅 0:00 制定计划和允许日内调整的购电—储能优化。

问题四波动电价优化

源文件：code/q4_solve.py

```

1  from __future__ import annotations
2
3  import csv
4  import json
5  import platform
6  import sys

```

```

7 from datetime import date, timedelta
8 from pathlib import Path
9 from typing import Any
10
11 import numpy as np
12 import scipy
13
14 from common_data import PROJECT_ROOT, load_attachment1, load_attachment2,
15     ↳ load_attachment3, load_attachment4, sha256
16 from dispatch_core import POSITIVE_TOL_KWH, SOC_MAX, SOC_MIN, SOLVER_TOL_KWH
17 import q2_solve as q2
18 import q3_solve as q3
19
20 RESULTS = PROJECT_ROOT / "results"
21 VALIDATION = PROJECT_ROOT / "validation"
22 INTERMEDIATE = PROJECT_ROOT / "intermediate"
23 EVAL_START = 31
24 N = 144
25
26
27 def write_csv(path: Path, rows: list[dict[str, Any]]) -> None:
28     if not rows:
29         raise ValueError(f"拒绝写入无表头空CSV: {path.name}")
30     with path.open("w", encoding="utf-8-sig", newline="") as stream:
31         writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
32         writer.writeheader()
33         writer.writerows(rows)
34
35
36 def metric(actual: np.ndarray, forecast: np.ndarray) -> dict[str, float]:
37     error = forecast - actual
38     return {
39         "mae_yuan_per_kwh": float(np.mean(np.abs(error))),
40         "rmse_yuan_per_kwh": float(np.sqrt(np.mean(error**2))),
41         "wape": float(np.sum(np.abs(error)) / np.sum(np.abs(actual))),
42         "bias_yuan_per_kwh": float(np.mean(error)),
43     }
44
45
46 def price_issue_diagnostics(price: np.ndarray, dates: list[date]) -> dict[str, Any]:
47     diagnostics: dict[str, Any] = {}
48     for issue_hour in (0, 6, 12, 18):
49         start = issue_hour * 6
50         actual_rows: list[np.ndarray] = []
51         persistence_rows: list[np.ndarray] = []
52         weekday_rows: list[np.ndarray] = []
53         scaled_rows: list[np.ndarray] = []
54         scaled_ar1_rows: list[np.ndarray] = []
55         scales: list[float] = []
56         for day_idx in range(EVAL_START, len(dates) - 1):
57             actual_rows.append(np.concatenate([price[day_idx, start:], price[day_idx +
58                 ↳ 1, :start]]))
59             persistence, _ = q3.causal_price_forecast_horizon(price, dates, day_idx,
60                 ↳ issue_hour, "persistence")
61             weekday, _ = q3.causal_price_forecast_horizon(price, dates, day_idx,
62                 ↳ issue_hour, "weekday_mean_28d")
63             scaled, scale = q3.causal_price_forecast_horizon(price, dates, day_idx,
64                 ↳ issue_hour, "weekday_mean_28d_intraday_scale")
65             scaled_ar1, _ = q3.causal_price_forecast_horizon(price, dates, day_idx,
66                 ↳ issue_hour, "weekday_mean_28d_intraday_scale_ar1")
67             persistence_rows.append(persistence)
68             weekday_rows.append(weekday)
69             scaled_rows.append(scaled)
70             scaled_ar1_rows.append(scaled_ar1)
71             scales.append(scale)
72         actual = np.asarray(actual_rows)
73         diagnostics[f"{issue_hour:02d}:00"] = {
74             "evaluation_days_with_complete_24h_target": len(actual_rows),
75             "persistence": metric(actual, np.asarray(persistence_rows)),
76             "weekday_mean_28d": metric(actual, np.asarray(weekday_rows)),
77             "weekday_mean_28d_intraday_scale": metric(actual, np.asarray(scaled_rows)),
78             "weekday_mean_28d_intraday_scale_ar1": metric(actual,
79                 ↳ np.asarray(scaled_ar1_rows)),
80             "scale_range": [float(np.min(scales)), float(np.max(scales))],

```

```

75     }
76     return diagnostics
77
78
79 def fixed_reprice_q2(path: Path, price: np.ndarray, dates: list[date]) -> dict[str,
    ↪ float]:
80     date_idx = {value.isoformat(): idx for idx, value in enumerate(dates)}
81     normal = emergency = 0.0
82     with path.open("r", encoding="utf-8-sig", newline="") as stream:
83         for row in csv.DictReader(stream):
84             p = float(price[date_idx[row["date"]], int(row["slot"]) - 1])
85             normal += p * float(row["normal_purchase_kwh"])
86             emergency += 5.0 * p * float(row["emergency_purchase_kwh"])
87     return {"normal_cost_yuan": normal, "emergency_cost_yuan": emergency,
    ↪ "total_cost_yuan": normal + emergency}
88
89
90 def fixed_reprice_q3(path: Path, price: np.ndarray, dates: list[date]) -> dict[str,
    ↪ float]:
91     date_idx = {value.isoformat(): idx for idx, value in enumerate(dates)}
92     base = credit = upward = emergency = 0.0
93     with path.open("r", encoding="utf-8-sig", newline="") as stream:
94         for row in csv.DictReader(stream):
95             p = float(price[date_idx[row["date"]], int(row["slot"]) - 1])
96             base += p * float(row["original_purchase_kwh"])
97             credit += 0.5 * p * float(row["downward_adjustment_kwh"])
98             upward += 1.5 * p * float(row["upward_adjustment_kwh"])
99             emergency += 5.0 * p * float(row["emergency_purchase_kwh"])
100     normal = base - credit + upward
101     return {
102         "base_plan_cost_yuan": base,
103         "downward_credit_yuan": credit,
104         "upward_adjustment_cost_yuan": upward,
105         "normal_settlement_cost_yuan": normal,
106         "emergency_cost_yuan": emergency,
107         "total_cost_yuan": normal + emergency,
108     }
109
110
111 def save_q2_run(run: dict[str, Any]) -> dict[str, Path]:
112     paths = {
113         "detail": RESULTS / "q4_2_detail.csv",
114         "daily": RESULTS / "q4_2_daily.csv",
115         "storage": RESULTS / "q4_2_storage_4h.csv",
116         "events": RESULTS / "q4_2_emergency_events.csv",
117     }
118     for key in ("detail", "daily", "storage"):
119         write_csv(paths[key], run[key])
120     if run["events"]:
121         write_csv(paths["events"], run["events"])
122     else:
123         paths["events"].write_text("date,event_id,start_slot,end_slot_exclusive,interv
    ↪ al,purchase_kwh\n", encoding="utf-8-sig")
124     return paths
125
126
127 def save_q3_run(run: dict[str, Any]) -> dict[str, Path]:
128     paths = {
129         "detail": RESULTS / "q4_3_detail.csv",
130         "daily": RESULTS / "q4_3_daily.csv",
131         "storage": RESULTS / "q4_3_storage_4h.csv",
132         "events": RESULTS / "q4_3_emergency_events.csv",
133         "versions": RESULTS / "q4_3_plan_versions.csv",
134     }
135     for key in ("detail", "daily", "storage", "versions"):
136         write_csv(paths[key], run[key])
137     if run["events"]:
138         write_csv(paths["events"], run["events"])
139     else:
140         paths["events"].write_text("date,event_id,start_slot,end_slot_exclusive,interv
    ↪ al,purchase_kwh\n", encoding="utf-8-sig")
141     return paths
142
143

```

```

144 def validate_aggregate(question: str, agg: dict[str, Any], detail: list[dict[str,
    ↳ Any]], days: int) -> dict[str, Any]:
145     simultaneous = sum(
146         float(row["charge_bus_kwh"]) > POSITIVE_TOL_KWH and
    ↳ float(row["discharge_bus_kwh"]) > POSITIVE_TOL_KWH
147         for row in detail
148     )
149     emergency_charge = sum(
150         float(row["emergency_purchase_kwh"]) > POSITIVE_TOL_KWH and
    ↳ float(row["charge_bus_kwh"]) > POSITIVE_TOL_KWH
151         for row in detail
152     )
153     max_charge = max(float(row["charge_bus_kwh"]) for row in detail)
154     max_discharge = max(float(row["discharge_bus_kwh"]) for row in detail)
155     info_violations = sum(row["train_end_date"] >= row["date"] for row in detail)
156     flags = {
157         "detail_rows": len(detail) == days * N,
158         "information_cutoff": info_violations == 0,
159         "actual_balance": agg["max_actual_balance_residual_kwh"] <= SOLVER_TOL_KWH,
160         "actual_state": agg["max_actual_state_residual_kwh"] <= SOLVER_TOL_KWH,
161         "soc_bounds": agg["soc_min_kwh"] >= SOC_MIN - SOLVER_TOL_KWH and
    ↳ agg["soc_max_kwh"] <= SOC_MAX + SOLVER_TOL_KWH,
162         "storage_power": max_charge <= q3.FLOW_MAX_KWH + SOLVER_TOL_KWH and
    ↳ max_discharge <= q3.FLOW_MAX_KWH + SOLVER_TOL_KWH,
163         "crossday_soc": agg["max_crossday_soc_gap_kwh"] <= SOLVER_TOL_KWH,
164         "mutual_exclusion_actual": simultaneous == 0,
165         "emergency_not_charging": emergency_charge == 0,
166         "plan_balance": agg["max_plan_balance_residual_kwh"] <= SOLVER_TOL_KWH,
167         "plan_state": agg["max_plan_state_residual_kwh"] <= SOLVER_TOL_KWH,
168         "plan_mutual_exclusion": agg["plan_simultaneous_positive_slots"] == 0,
169     }
170     return {
171         "schema_version": 1,
172         "question": question,
173         "status": "PASS" if all(flags.values()) else "FAIL",
174         "pass_flags": flags,
175         "metrics": {"information_violations": info_violations,
    ↳ "simultaneous_actual_slots": simultaneous,
    ↳ "emergency_charge_overlap_slots": emergency_charge,
176             "storage_power_limit_kw": 5000.0,
    ↳ "storage_flow_limit_kwh_per_slot": q3.FLOW_MAX_KWH,
177             "max_charge_kwh_per_slot": max_charge,
    ↳ "max_discharge_kwh_per_slot": max_discharge, **agg},
178         "environment": {"python": sys.version, "python_executable": sys.executable,
    ↳ "platform": platform.platform(), "numpy": np.__version__, "scipy":
    ↳ scipy.__version__, "solver": "scipy.optimize.linprog(method='highs')"},
179     }
180
181
182 def main() -> None:
183     RESULTS.mkdir(parents=True, exist_ok=True)
184     VALIDATION.mkdir(parents=True, exist_ok=True)
185     INTERMEDIATE.mkdir(parents=True, exist_ok=True)
186     fixed = load_attachment1()
187     annual = load_attachment2()
188     pv_forecast = load_attachment3()
189     realtime = load_attachment4()
190     if not (annual["dates"] == pv_forecast["dates"] == realtime["dates"]):
191         raise ValueError("附件2、3、4日期轴不一致")
192     dates = annual["dates"]
193     load_kw, pv_kw = annual["load_kw"], annual["pv_kw"]
194     actual_net = load_kw - pv_kw
195     price = realtime["price_yuan_per_kwh"]
196
197     price_weekday = q2.make_historical_weekday_cache(price, dates)
198     eval_slice = slice(EVAL_START, len(dates))
199     price_diagnostics = {
200         "range_yuan_per_kwh": [float(np.min(price)), float(np.max(price))],
201         "mean_yuan_per_kwh": float(np.mean(price)),
202         "persistence_1d": metric(price[eval_slice], price[EVAL_START - 1 : len(dates) -
    ↳ 1]),
203         "weekday_mean_28d": metric(price[eval_slice], price_weekday[eval_slice]),
204         "issue_horizon_24h": price_issue_diagnostics(price, dates),
205     }
206

```

```

207 q2_config = json.loads((INTERMEDIATE /
    ↪ "q2_main_config.json").read_text(encoding="utf-8"))
208 q2_kind = str(q2_config["selected_forecast_kind"])
209 _, _, q2_point_net = q2.make_point_forecast_cache(q2_kind, load_kw, pv_kw, dates)
210 print(json.dumps({"event": "q4_2_start", "price_model": "persistence"},
    ↪ ensure_ascii=False), flush=True)
211 q42_persistence = q2.run_strategy(
212     name="q4_2_price_persistence", forecast_kind=q2_kind, quantile=0.8,
213     terminal_mode="fixed_6000_at_48h", load_kw=load_kw, pv_kw=pv_kw, dates=dates,
214     price=fixed["price_yuan_per_kwh"], actual_net_kw=actual_net,
    ↪ weekday_net_cache_kw=q2_point_net,
215     raw_endpoint_labels=annual["raw_endpoint_labels"], keep_detail=False,
216     actual_price_by_date=price, planning_price_kind="persistence",
217 )
218 print(json.dumps({"event": "q4_2_start", "price_model": "weekday_mean_28d"},
    ↪ ensure_ascii=False), flush=True)
219 q42 = q2.run_strategy(
220     name="q4_2_main", forecast_kind=q2_kind, quantile=0.8,
221     terminal_mode="fixed_6000_at_48h", load_kw=load_kw, pv_kw=pv_kw, dates=dates,
222     price=fixed["price_yuan_per_kwh"], actual_net_kw=actual_net,
    ↪ weekday_net_cache_kw=q2_point_net,
223     raw_endpoint_labels=annual["raw_endpoint_labels"], keep_detail=True,
224     actual_price_by_date=price, planning_price_kind="weekday_mean_28d",
225 )
226 q42_paths = save_q2_run(q42)
227
228 cache_linear, _, _ = q3.build_forecast_caches(load_kw, pv_kw,
    ↪ pv_forecast["forecast_kw"], dates, "linear")
229 q43_specs = [
230     ("q4_3_0only", [], "weekday_mean_28d_intraday_scale_ar1", False),
231     ("q4_3_0_6", [6], "weekday_mean_28d_intraday_scale_ar1", False),
232     ("q4_3_0_6_12", [6, 12], "weekday_mean_28d_intraday_scale_ar1", False),
233     ("q4_3_all_main", [6, 12, 18], "weekday_mean_28d_intraday_scale_ar1", True),
234     ("q4_3_all_price_scale_only", [6, 12, 18], "weekday_mean_28d_intraday_scale",
    ↪ False),
235     ("q4_3_all_price_frozen_midnight", [6, 12, 18], "weekday_mean_28d", False),
236     ("q4_3_all_price_persistence", [6, 12, 18], "persistence", False),
237 ]
238 q43_runs: dict[str, dict[str, Any]] = {}
239 for name, updates, price_kind, keep in q43_specs:
240     print(json.dumps({"event": "q4_3_start", "strategy": name},
    ↪ ensure_ascii=False), flush=True)
241     q43_runs[name] = q3.run_strategy(
242         name=name, update_hours=updates, interpolation="linear", load_kw=load_kw,
    ↪ pv_kw=pv_kw,
243         forecast3_kw=pv_forecast["forecast_kw"], dates=dates,
    ↪ price=fixed["price_yuan_per_kwh"],
244         residual_cache=cache_linear,
    ↪ raw_endpoint_labels=annual["raw_endpoint_labels"], keep_detail=keep,
245         actual_price_by_date=price, planning_price_kind=price_kind,
246     )
247 q43 = q43_runs["q4_3_all_main"]
248 q43_paths = save_q3_run(q43)
249
250 fixed_q2 = fixed_reprice_q2(RESULTS / "q2_detail.csv", price, dates)
251 fixed_q3 = fixed_reprice_q3(RESULTS / "q3_detail.csv", price, dates)
252 q42_validation = validate_aggregate("Q4-2", q42["aggregate"], q42["detail"], 334)
253 q43_validation = validate_aggregate("Q4-3", q43["aggregate"], q43["detail"], 334)
254 if q43["aggregate"]["max_adjustment_link_residual_kwh"] > SOLVER_TOL_KWH:
255     q43_validation["pass_flags"]["adjustment_link"] = False
256     q43_validation["status"] = "FAIL"
257 else:
258     q43_validation["pass_flags"]["adjustment_link"] = True
259
260 q42_workbook = {
261     "schema_version": 1, "question": "Q4-2",
262     "time_label_mapping":
    ↪ "natural_day_00:00_to_24:00_corrected_from_shifted_official_template",
263     "plans": q42["plans"], "storage_four_hour": q42["storage"], "emergency_events":
    ↪ q42["events"],
264 }
265 q43_workbook = {
266     "schema_version": 1, "question": "Q4-3",
267     "time_label_mapping":
    ↪ "natural_day_00:00_to_24:00_corrected_from_shifted_official_template",

```

```

268         "plans": q43["workbook_plans"], "storage_four_hour": q43["storage"],
        ↪     "emergency_events": q43["events"],
269     }
270     q42_payload_path = INTERMEDIATE / "q4_2_workbook_payload.json"
271     q43_payload_path = INTERMEDIATE / "q4_3_workbook_payload.json"
272     q42_payload_path.write_text(json.dumps(q42_workbook, ensure_ascii=False, indent=2),
        ↪     encoding="utf-8")
273     q43_payload_path.write_text(json.dumps(q43_workbook, ensure_ascii=False, indent=2),
        ↪     encoding="utf-8")
274
275     ordered = ["q4_3_0only", "q4_3_0_6", "q4_3_0_6_12", "q4_3_all_main"]
276     incremental = []
277     for before, after in zip(ordered[:-1], ordered[1:]):
278         a, b = q43_runs[before]["aggregate"], q43_runs[after]["aggregate"]
279         incremental.append({"from": before, "to": after, "cost_saving_yuan":
        ↪     a["total_cost_yuan"] - b["total_cost_yuan"], "emergency_change_kwh":
        ↪     b["emergency_purchase_kwh"] - a["emergency_purchase_kwh"]})
280     selected = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
281     summary = {
282         "schema_version": 1, "status": "SOLVED_AND_VALIDATED" if
        ↪     q42_validation["status"] == q43_validation["status"] == "PASS" else "FAIL",
283         "question": "Q4", "evaluation_period": {"start": dates[EVAL_START].isoformat(),
        ↪     "end": dates[-1].isoformat(), "days": 334},
284         "frozen_basis": {"future_actual_price": "settlement only; never used in
        ↪     planning", "q4_2_information": "Q2 permissions without attachment 3",
        ↪     "q4_3_information": "Q3 permissions with attachment 3",
        ↪     "settlement_price_index": "actual delivery-slot price", "known_conflict":
        ↪     "A01 remains"},
285         "price_forecast_diagnostics": price_diagnostics,
286         "model_settings": {"q2_forecast_interface": q2_kind,
        ↪     "q4_2_main_price_forecast": "prior-28-day same-weekday mean formed causally
        ↪     at midnight", "price_baseline": "previous-day same-slot persistence",
        ↪     "q4_3_intraday_price_forecast": "at 00:00 use the prior-28-day same-weekday
        ↪     profile; at 06:00/12:00/18:00 scale the causal 24-hour historical profile
        ↪     by the observed-prefix ratio and add a decaying AR(1) correction from the
        ↪     last observed residual; AR(1) is re-estimated from prior-28-day causal
        ↪     residuals"},
287         "fixed_strategy_repricing": {"q2_fixed_decisions": fixed_q2,
        ↪     "q3_fixed_decisions": fixed_q3},
288         "q4_2": {"main": q42["aggregate"], "price_persistence":
        ↪     q42_persistence["aggregate"], "saving_vs_fixed_q2_decisions_yuan":
        ↪     fixed_q2["total_cost_yuan"] - q42["aggregate"]["total_cost_yuan"],
        ↪     "saving_vs_price_persistence_yuan":
        ↪     q42_persistence["aggregate"]["total_cost_yuan"] -
        ↪     q42["aggregate"]["total_cost_yuan"], "selected_date_events": {d: [r for r
        ↪     in q42["events"] if r["date"] == d] for d in selected}},
289         "q4_3": {"strategies": {name: run["aggregate"] for name, run in
        ↪     q43_runs.items()}, "incremental_update_value": incremental,
        ↪     "saving_vs_fixed_q3_decisions_yuan": fixed_q3["total_cost_yuan"] -
        ↪     q43["aggregate"]["total_cost_yuan"], "saving_vs_price_persistence_yuan":
        ↪     q43_runs["q4_3_all_price_persistence"]["aggregate"]["total_cost_yuan"] -
        ↪     q43["aggregate"]["total_cost_yuan"],
        ↪     "saving_vs_frozen_midnight_price_forecast_yuan":
        ↪     q43_runs["q4_3_all_price_frozen_midnight"]["aggregate"]["total_cost_yuan"]
        ↪     - q43["aggregate"]["total_cost_yuan"], "selected_date_events": {d: [r for r
        ↪     in q43["events"] if r["date"] == d] for d in selected}},
290         "files": {"q4_2": {k: str(v.relative_to(PROJECT_ROOT)) for k, v in
        ↪     q42_paths.items()}, "q4_3": {k: str(v.relative_to(PROJECT_ROOT)) for k, v
        ↪     in q43_paths.items()}},
291     }
292     summary_path = RESULTS / "q4_summary.json"
293     summary_path.write_text(json.dumps(summary, ensure_ascii=False, indent=2),
        ↪     encoding="utf-8")
294     q42_validation_path = VALIDATION / "q4_2_validation.json"
295     q43_validation_path = VALIDATION / "q4_3_validation.json"
296     q42_validation_path.write_text(json.dumps(q42_validation, ensure_ascii=False,
        ↪     indent=2), encoding="utf-8")
297     q43_validation_path.write_text(json.dumps(q43_validation, ensure_ascii=False,
        ↪     indent=2), encoding="utf-8")
298
299     manifest = {
300         "question": "Q4", "status": summary["status"],

```

```

301     "input_sha256": {"attachment1": fixed["sha256"], "attachment2":
    ↪ annual["sha256"], "attachment3": pv_forecast["sha256"], "attachment4":
    ↪ realtime["sha256"]},
302     "code_sha256": {name: sha256(Path(__file__).with_name(name)) for name in
    ↪ ("common_data.py", "dispatch_core.py", "q2_solve.py", "q3_solve.py",
    ↪ "q4_solve.py")},
303     "outputs_sha256": {str(path.relative_to(PROJECT_ROOT)): sha256(path) for path
    ↪ in [*q42_paths.values(), *q43_paths.values(), summary_path,
    ↪ q42_validation_path, q43_validation_path, q42_payload_path,
    ↪ q43_payload_path]},
304 }
305 (INTERMEDIATE / "q4_run_manifest.json").write_text(json.dumps(manifest,
    ↪ ensure_ascii=False, indent=2), encoding="utf-8")
306 print(json.dumps({"status": summary["status"], "q4_2_total_cost_yuan":
    ↪ q42["aggregate"]["total_cost_yuan"], "q4_3_total_cost_yuan":
    ↪ q43["aggregate"]["total_cost_yuan"], "q4_2_fixed_reprice_yuan":
    ↪ fixed_q2["total_cost_yuan"], "q4_3_fixed_reprice_yuan":
    ↪ fixed_q3["total_cost_yuan"]}, ensure_ascii=False))
307 if summary["status"] != "SOLVED_AND_VALIDATED":
308     raise RuntimeError("Q4核验未通过")
309
310
311 if __name__ == "__main__":
312     main()

```